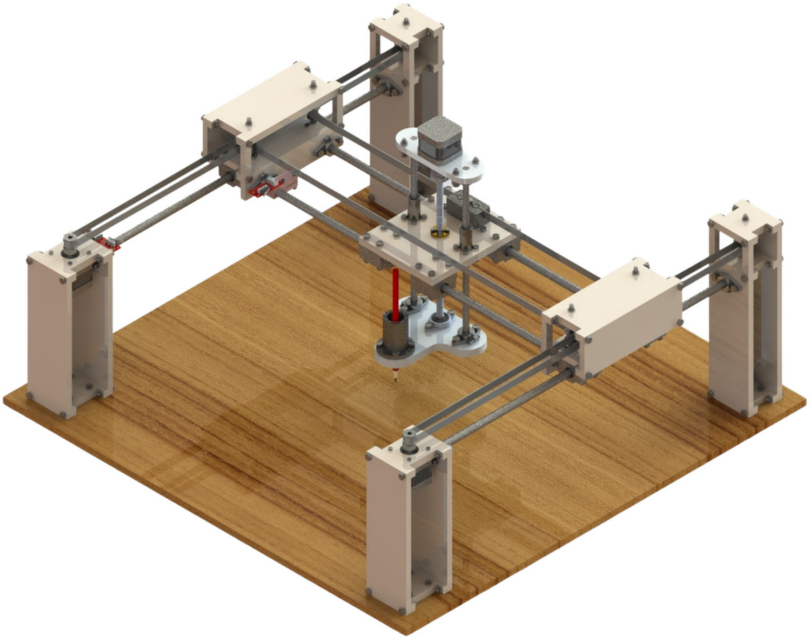
University Mohammed VI Polytechnic

EMINES School of Industrial Management

Rapport Final

Groupe 14 – Class 2030



Samy Youssoufine
Riham Boughrara
Rita Tamma
Soufiane Bourghel

Juin 2026

Remerciements

Nous tenons en premier lieu à remercier l'équipe pédagogique de l'EMINES pour l'encadrement, les ressources mises à disposition et les précieux conseils prodigués tout au long de ce deuxième projet d'ingénierie.

Nous adressons une reconnaissance toute particulière aux responsables et techniciens du FabLab. Leur expertise technique et leur disponibilité nous ont été indispensables pour surmonter les défis liés à la fabrication.

Enfin, ce projet est le fruit d'un travail d'équipe intense. Nous nous remercions mutuellement pour l'implication, la rigueur et la bonne humeur qui ont permis de mener ce projet à son terme.



Résumé

Ce rapport synthétise tout le travail réalisé dans le cadre du deuxième projet d'ingénierie de la CPI-1A. Il est structuré suivant l'ordre chronologique des étapes de développement, présentant les choix techniques, les problèmes rencontrés et les solutions apportées. Il est composé de cinq parties principales :

- **Analyse fonctionnelle** — Cette section détaille les besoins et les exigences du projet, les dimensionnements techniques effectués, ainsi que les diagrammes et outils SysML utilisés pour modéliser le système.
- **Conception** — Cette partie présente la conception du système sous SolidWorks, ainsi que les différents choix techniques et les justifications associées.
- **Fabrication et assemblage** — Cette partie détaille le processus de fabrication des pièces et l'assemblage du système.
- **Algorithmique et logique de commande** — Cette section décrit l'implémentation du DDA en entiers pour la réalisation de trajectoires linéaires, ainsi que les autres dispositifs de commande et de synchronisation des moteurs (profil trapézoïdal, etc.).
- **Interface homme-machine** — Cette section présente toutes les étapes de conception et d'élaboration de l'interface graphique permettant de contrôler le système, ainsi que ses diverses fonctionnalités.
- **Tests unitaires et validation** — Cette partie présente les tests effectués sur le système, les résultats obtenus, ainsi que les ajustements réalisés pour atteindre les performances fixées par le CdCF.

Table des matières

1 Analyse fonctionnelle 2
1.1 Présentation générale du projet 2
1.2 Diagramme des cas d'utilisation (UC) 2
1.3 Diagramme des exigences (Requirements) 3
1.4 Diagrammes BDD 4
1.5 Diagrammes IBD 7
1.6 Schémas cinématiques 9
1.7 Schéma d'architecture 10
1.8 Caractérisation du mouvement de la courroie 11
1.9 Modélisation cinématique du porte-stylet (axe Z) 13
1.10 Études de dimensionnement 14
2 Conception et Mécanique 22
2.1 Maquette SolidWorks 22
2.2 Pièces conçues 24
2.3 Solutions de liaison glissière 32
2.4 Solutions d'encastrement 32
3 Fabrication et assemblage 35
3.1 Synthèse des pièces fabriquées 35
3.2 Procédés de fabrication utilisés 35
3.3 Gamme d'usinage détaillée d'une pièce représentative 36
3.4 Chaîne numérique de fabrication 36
3.5 Assemblage mécanique 36
3.6 Montage de l'axe Z 37
3.7 Problèmes rencontrés et solutions apportées 37
3.8 Montage final 38
3.9 Écarts par rapport à la CAO initiale 38

4 Algorithmique et Logique de commande 40
4.1 Architecture générale du code embarqué 40
4.2 Analyse du premier étage — Communication et Parsing 42
4.3 Analyse du deuxième étage — Planification et sécurité 44
4.4 Analyse du troisième étage — Exécution du mouvement (DDA) 49
4.5 Informations additionnelles 52
4.6 Gestion du stylet (Axe Z) 56
4.7 Procédures de Homing/Remise à zéro 56
4.8 Intégration à l'UI et Handshake 58
5 Interface Homme-Machine 60
5.1 Identité Graphique 60
5.2 Architecture Technique 61
5.3 Accessibilité et Ergonomie (UX) 62
5.4 Fonctionnalités et Logique Opérationnelle 63
5.5 Méthodologie de Conception Assistée par IA 65
6 Tests et validation 68
6.1 Protocoles des tests unitaires et d'intégration 68
6.2 Synthèse des résultats et anomalies observées 69
6.3 Solutions et correctifs appliqués 69
6.4 Évaluation des performances du système final 70
Conclusion Générale 72
Annexe 74

Table des figures

1 Diagramme des cas d'utilisation du système de dessin sur plan 3
2 Diagramme des exigences du projet de traceur sur plan 4
3 Diagramme BDD du système de dessin sur plan 5
4 Diagramme BDD du sous-système de déplacement en plan XY 6
5 Diagramme BDD du sous-système de déplacement selon Z 7
6 Diagramme IBD du système de dessin sur plan 8
7 Diagramme IBD du sous-système de déplacement en plan XY 8
8 Diagramme IBD du sous-système de déplacement selon Z 9
9 Schéma cinématique en 3D du système 9
10 Schéma d'architecture du système 10
11 Détail de l'architecture selon Z du système 10
12 Paramétrage cinématique du système d'entraînement 11
13 Analyse d'un déplacement élémentaire du chariot de M_i vers M_f et répercussion sur les longueurs de courroie. 11
14 Décomposition vectorielle du vecteur vitesse 13
15 Schéma de modélisation dynamique du système de courroie dans le plan XY (simplifié pour l'étude de dimensionnement) 15
16 Modélisation du mouvement d'une vis-écrou par un solide sur un plan incliné 18
17 Maquette 3D de l'assemblage final du système cartésien. 22
18 Vue du sous-ensemble SE0 – Base 23
19 Vue du sous-ensemble SE1 – Mobile horizontal 23
20 Vue du sous-ensemble SE2 – Mobile vertical 24
21 Bâti (HPL, marron). 24
22 Colonnes de support moteur (HPL, vertes). 25
23 Colonnes de support poulie libre (HPL, vertes). 26
24 Supports moteur et poulie libre. 27

25 Supports inférieurs de liaison des colonnes. 27
26 Plaques de liaison et intermédiaires du sous-assemblage douilles (teintes mauve/lavande). 28
27 Pièce centrale du chariot (HPL, intégrée au SE2 gris). 29
28 Supports de l'axe vertical (SE2 gris). 29
29 Composants du porte-stylet (impression 3D). 30
30 Support de fin de course (impression 3D). 30
31 Pièce d'ancrage de la courroie (impression 3D). 31
32 Rails de guidage cylindriques (acier). 31
33 Tiges filetées (acier). 31
34 Diagramme de flux de la chaîne numérique de conversion des données géométriques. 36
35 Arborescence des fonctions et procédures du système cartésien traceur de plan 40
36 Diagramme de flux du système cartésien traceur de plan 41
37 Algorigramme du premier étage de communication et d'analyse syntaxique. . 42
38 Comparaison globale des trajectoires rectilignes générées via l'interpolation cartésienne et l'interpolation moteur. 45
39 Principe de sécurisation spatiale sur le plan XY. Toute trajectoire traversant les frontières de l'enveloppe déclenche une interruption matérielle immédiate, couplée à un recalcul borné des coordonnées absolues pour préserver le référentiel. 46
40 Profil cinématique généré par la fonction `rampedHP`. Le démarrage à V_{min} évite les pertes de pas liées au couple de détente du moteur. 48
41 Extrait de l'algorigramme d'aiguillage cinématique au sein de la boucle d'asservissement temporelle DDA. 49
42 Modulation de la largeur d'impulsion matérielle traduisant la consigne trapézoïdale de vitesse. L'échelle des impulsions représentées est exagérée et ne correspond pas exactement à celles envoyées au moteur en pratique (à but illustratif uniquement). 49
43 Analyse géométrique de l'erreur de quantification spatiale du traceur. L'injection de la constante `total/2` convertit l'opération de troncature en un arrondi au plus proche, centrant l'erreur sur $[-\frac{1}{2}, +\frac{1}{2}]$. 50
44 Chronogramme dualisé de l'Analyseur Différentiel Numérique (DDA) sur douze seaux d'intégration fractionnelle de capacité arbitraire $= 100$. L'évaluation simultanée de `eM1` et `eM2` met en évidence la **distribution asynchrone** des impulsions physiques envoyées aux drivers moteurs. 51
45 Algorigramme de fonctionnement de la procédure de Homing planaire . . . 57
46 Exemples de boutons à "aspect mécanique" (ombre portée sans flou) 61

47 Illustration de l'aplatissement d'une courbe de Bézier. Les segments rouges représentent les instructions linéaires (G1) générées. 63

48 Profil de mouvement cinématique lors d'une transition entre deux chemins vectoriels isolés. 63

49 Modélisation de la sécurité spatiale logicielle interdisant la sortie du stylet de la zone de dessin (sans échelle). 64

50 Vue du jumeau numérique sur l'interface graphique. 65

51 Conception de l'interface sur Photopea (le brouillon initial prévoit une interface mobile). 66

52 Création du prompt. 66

53 Diagramme simplifié expliquant les drifts. 68

54 Diagramme et graphique simplifiés expliquant les overshoots. 68

55 Comparaison de l'overshoot en profil rectangulaire et en profil trapézoïdal. En pointillés rouge (resp. vert) : le profil de vitesse rectangulaire (resp. trapézoïdal). En trait continu rouge (resp. vert) : la position $x(t)$ résultante et l'overshoot (resp. sans overshoot). 69

56 Nouvelle pièce de maintien du stylet. 70

57 Tracé SVG original à convertir 71

58 Résultat de la conversion (tracés parasites visibles sur R et O) 71

59 Schéma électrique simplifié du système entier 75

Liste des tableaux

1 Récapitulatif des spécifications fonctionnelles et contraintes du projet 2
2 Lois de transformation cinématique selon l'axe de déplacement du chariot . 13
3 Récapitulatif des énergies cinétiques de chaque composant du système de courroie 15
4 Paramètres utilisés pour le dimensionnement des moteurs du système de courroie 17
5 Paramètres pour l'application numérique du couple moteur 20
6 Récapitulatif des couples moteurs minimaux requis pour les différents systèmes d'entraînement du traceur de plan 20
7 Tableau des liaisons d'encastrement. 33
8 Synthèse des pièces fabriquées et procédés associés 35
9 Procédés de fabrication utilisés et descriptions associées 35
10 Solutions d'assemblage utilisées et descriptions fonctionnelles 37
11 Problèmes rencontrés et actions correctives associées 37
12 Architecture fonctionnelle des deux couches de surveillance du traceur . . . 46
13 Fonctions de communication et télémétrie (Interface Série). 52
14 Utilitaires et conversions mathématiques (Helpers). 53
15 Couche d'abstraction matérielle et sécurité (HAL). 53
16 Moteur d'exécution cinématique (Core). 53
17 Cycle de vie et configuration. 54
18 Récapitulatif des paramètres de configuration matériels et des constantes cinématiques d'asservissement. 55
19 Registre d'utilisation des outils d'IA 76

1 ANALYSE FONCTIONNELLE

++++++++++[>+++++++>++++++++++>++++++++++>++++++++++>++++++
++++>++++++++++>++++++++++>+++>++++++++++>++++++++++>+++
++++++++>++++++++++>++++++++++>++++++++++>++++++++++>++++
+++++++>++++++++++>++++++++++>++++++++++<<<<<<<<<<<<<<<<<<<-
]>-----.>.>---.>--.>+.>-----.>+.>++.>++.>+.>.>-.>----.>-----.
>+.>..>+.>--..>+.

1.1 Présentation générale du projet

Le deuxième projet d'ingénierie consiste à réaliser un système de positionnement (cartésien) à trois degrés de liberté (X, Y, Z) pour un effecteur. La translation selon l'axe Z est indépendante, tandis que les translations dans le plan X-Y sont couplées par un système de courroies H8. L'application choisie pour ce projet est le dessin sur plan.

ID	Critère	Description	Niveau/ valeur	Unité	Flexibilité 0-1-2
FS1 *Déplacer un effecteur (Dessin sur plan dans ce cas)*	**Déplacement plan**	Portée par degré de liberté en plan	400	mm	1
	Déplacement hors plan	Portée minimum hors plan	100	mm	1
	Vitesse	Plage vitesses en plan	0–80	mm/s	0
	Accélération	Accélération maximale en plan	100	mm/s^2	0
	Précision	Erreur linéaire par ddl	0.2	mm	0
FS2 *S'adapter au réseau électrique*	**Energie**	Mode d'alimentation	Secteur	N/A	0
FC *S'intégrer au lieu de travail*	**Encombrement**	Gabarit maximal L × l × H	75 × 75 × 45	cm^3	0

TABLEAU 1 – Récapitulatif des spécifications fonctionnelles et contraintes du projet

1.2 Diagramme des cas d'utilisation (UC)

Ce diagramme recense les services rendus par le traceur à l'utilisateur. Il permet d'isoler la fonction principale (*la réalisation d'un tracé*) et de définir les interactions entre l'opérateur et le système.

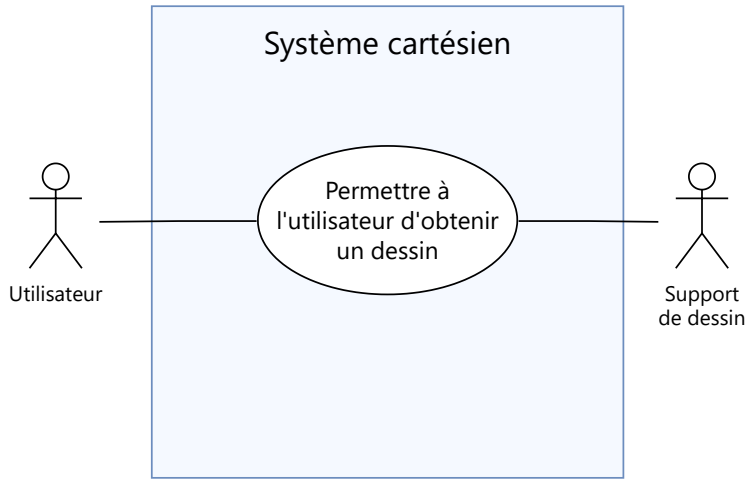


FIGURE 1 – Diagramme des cas d'utilisation du système de dessin sur plan

1.3 Diagramme des exigences (Requirements)

Ce diagramme hiérarchise de manière visuelle les exigences fonctionnelles et contraintes du projet. Il permet aussi de visualiser les relations de dépendance entre les différentes exigences.

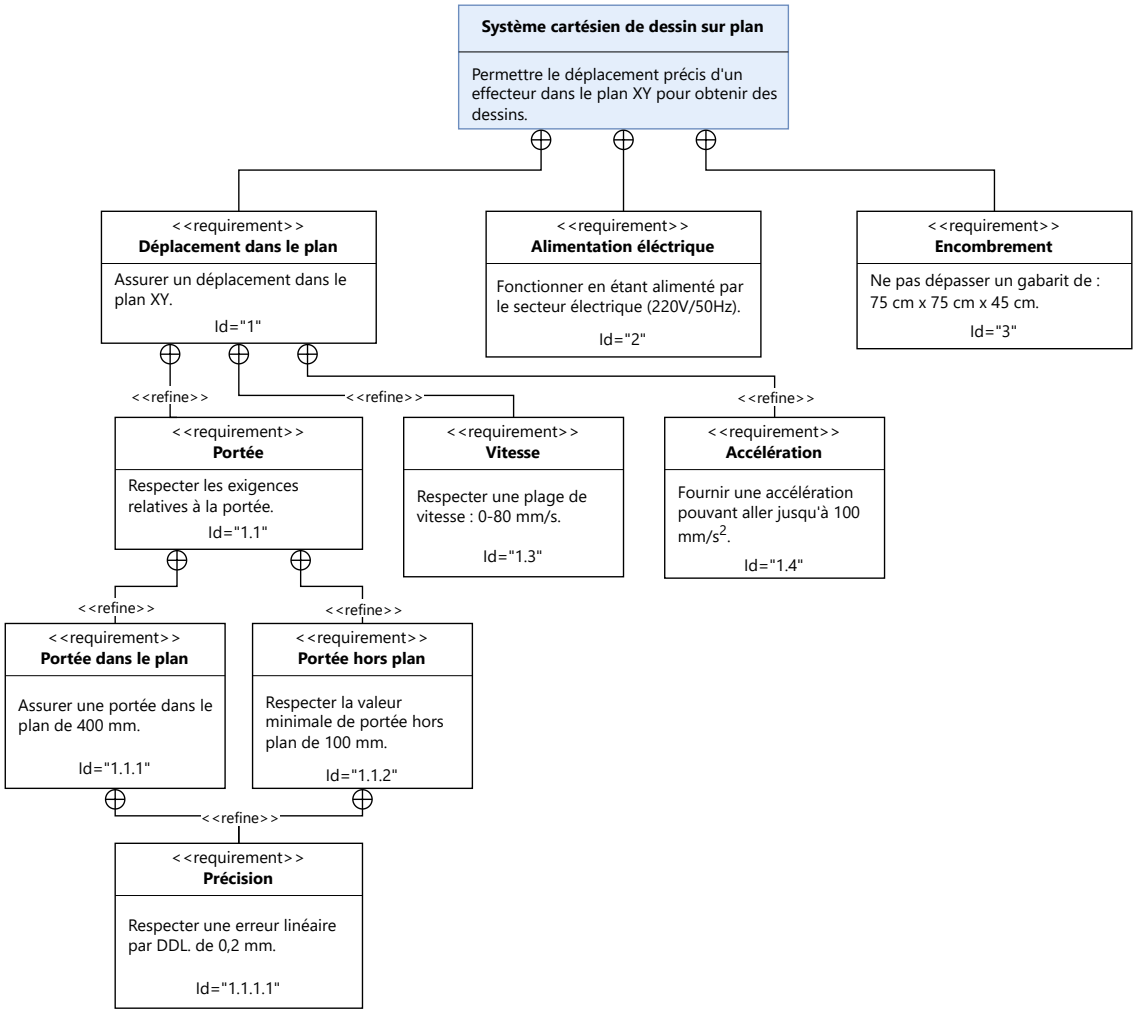


FIGURE 2 – Diagramme des exigences du projet de traceur sur plan

1.4 Diagrammes BDD

Ces diagrammes présentent l'architecture physique du système. Ils permettent de visualiser la décomposition du robot en sous-ensembles (énergie, traitement, action) et de définir les caractéristiques mécaniques de chaque composant.

La figure 3 présente la structure globale du système, tandis que les figures 4 et 5 détaillent respectivement les sous-systèmes de déplacement en plan XY et de déplacement selon Z.

1.4.1 Diagramme BDD du système global

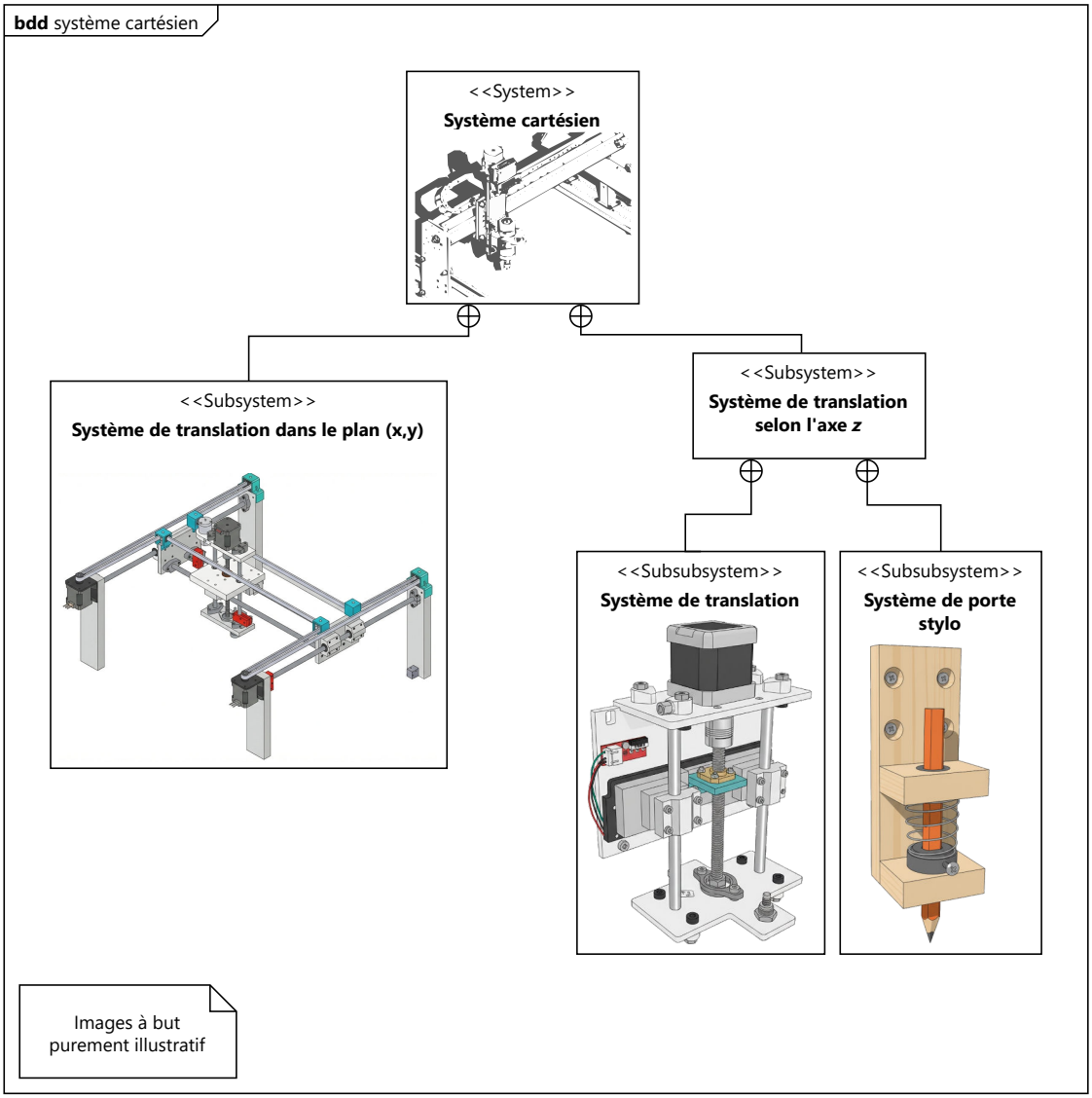


FIGURE 3 – Diagramme BDD du système de dessin sur plan

1.4.2 Diagramme BDD du sous-système de déplacement en plan XY

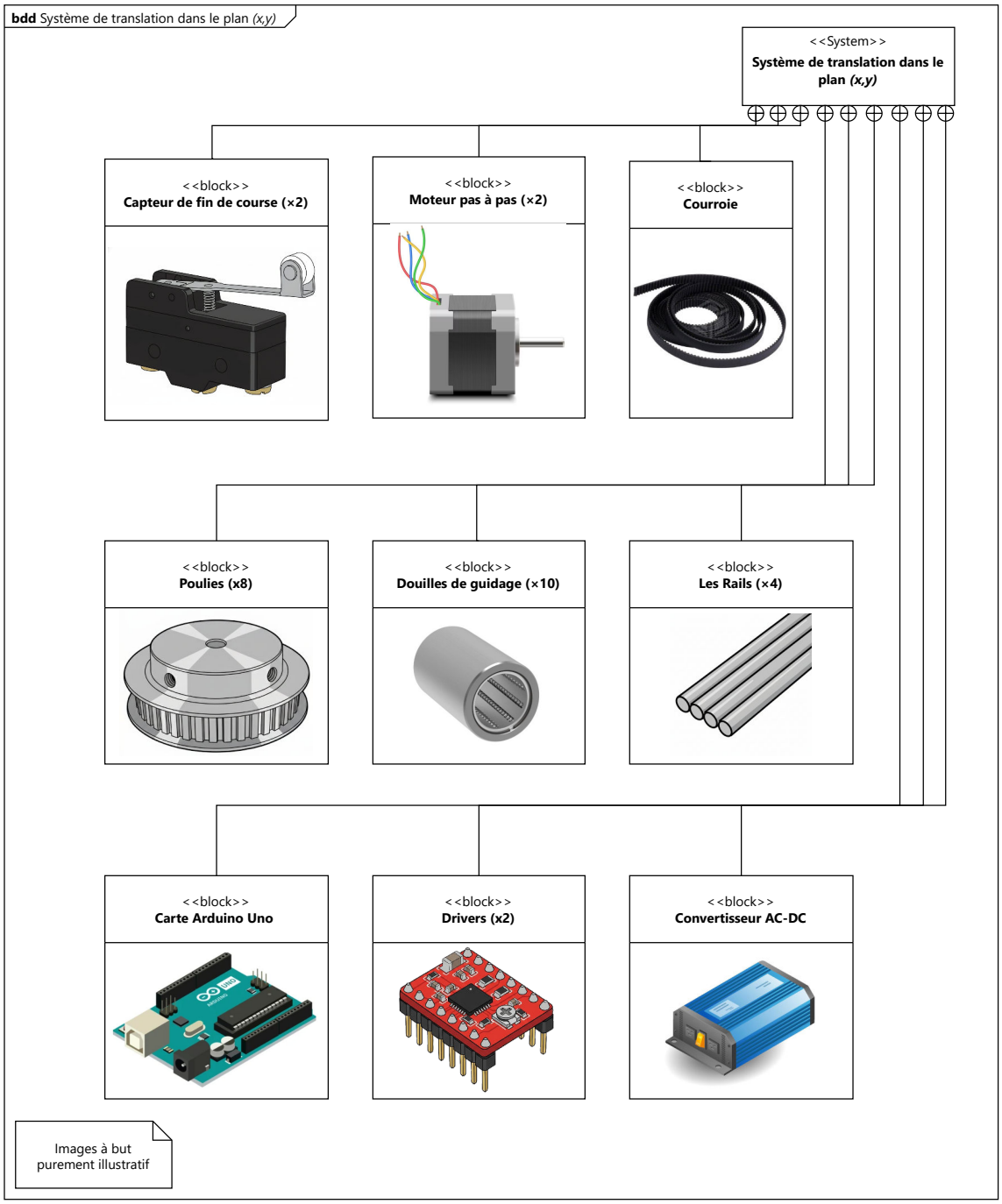


FIGURE 4 – Diagramme BDD du sous-système de déplacement en plan XY

1.4.3 Diagramme BDD du sous-système de déplacement selon Z

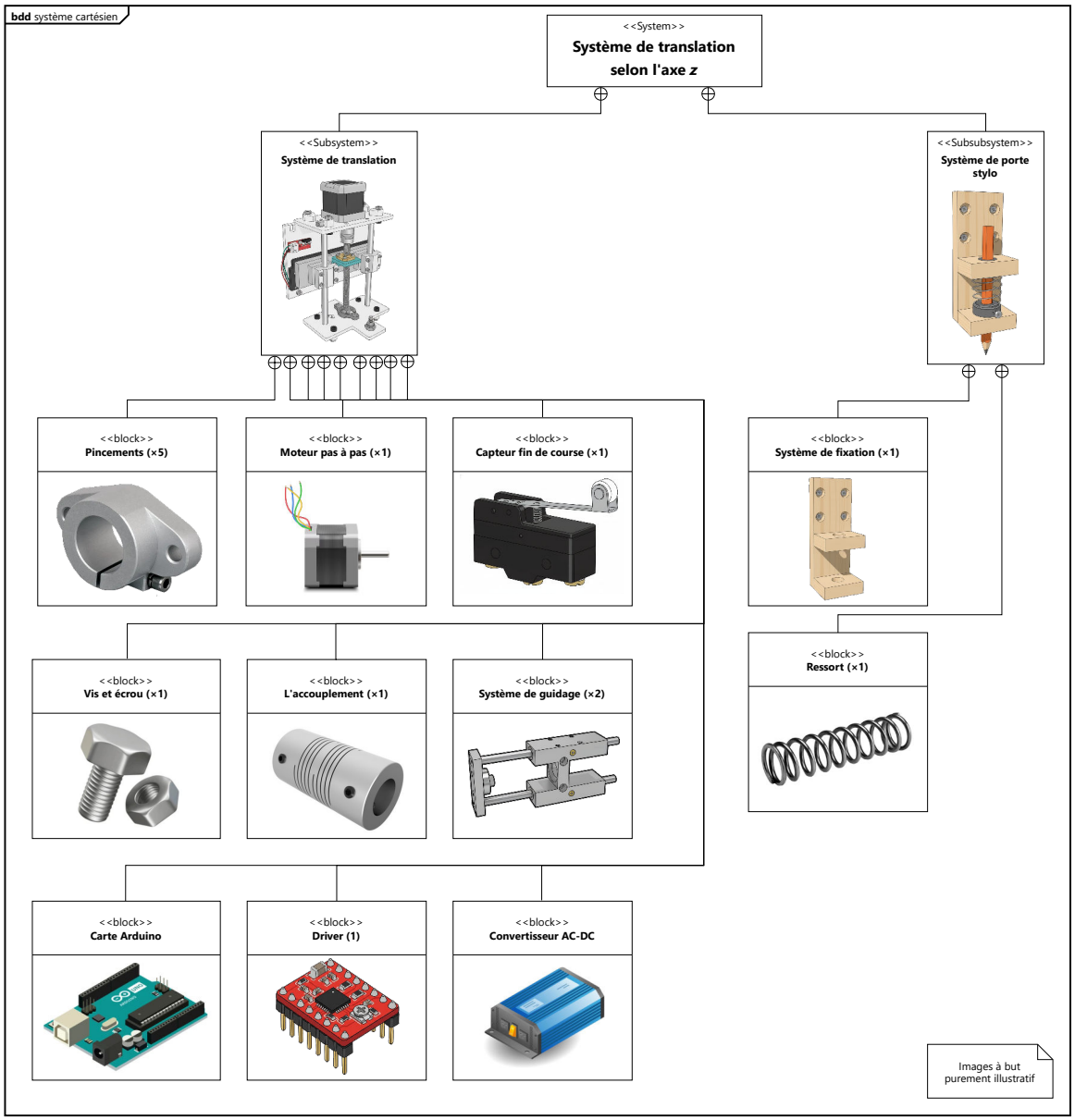


FIGURE 5 – Diagramme BDD du sous-système de déplacement selon Z

1.5 Diagrammes IBD

Ces diagrammes présentent les interactions fonctionnelles entre les différents composants du système. Ils permettent de visualiser les flux d'énergie et d'information au sein du système.

1.5.1 Diagramme IBD du système global

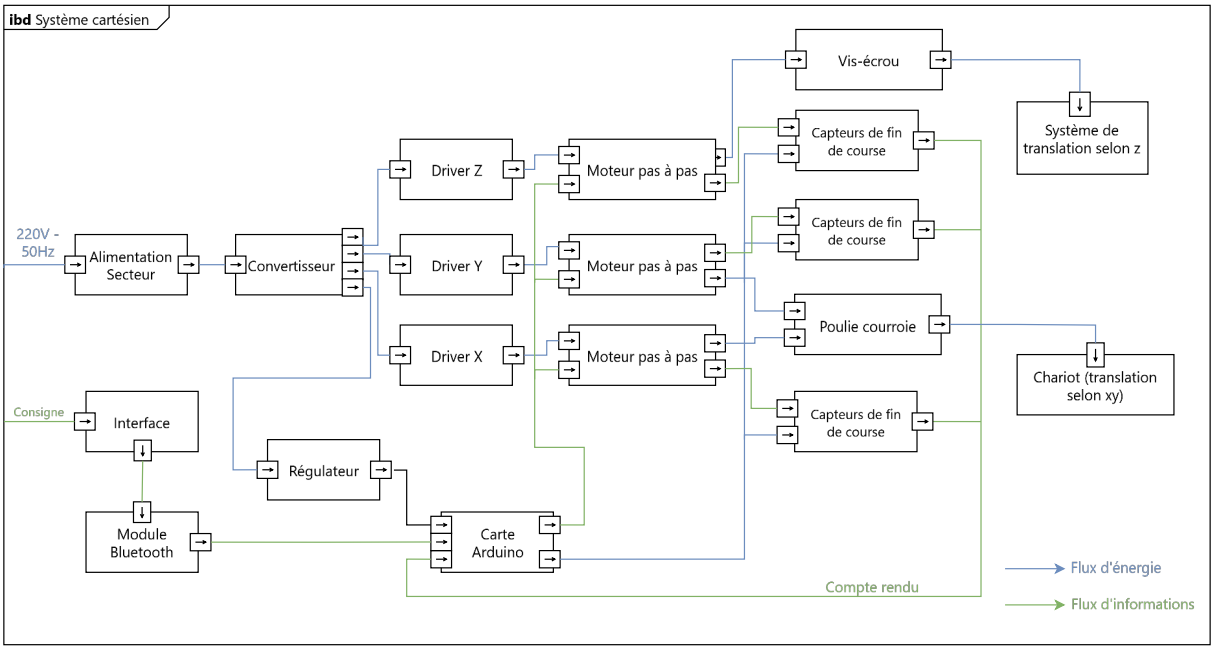


FIGURE 6 – Diagramme IBD du système de dessin sur plan

1.5.2 Diagramme IBD du sous-système de déplacement en plan XY

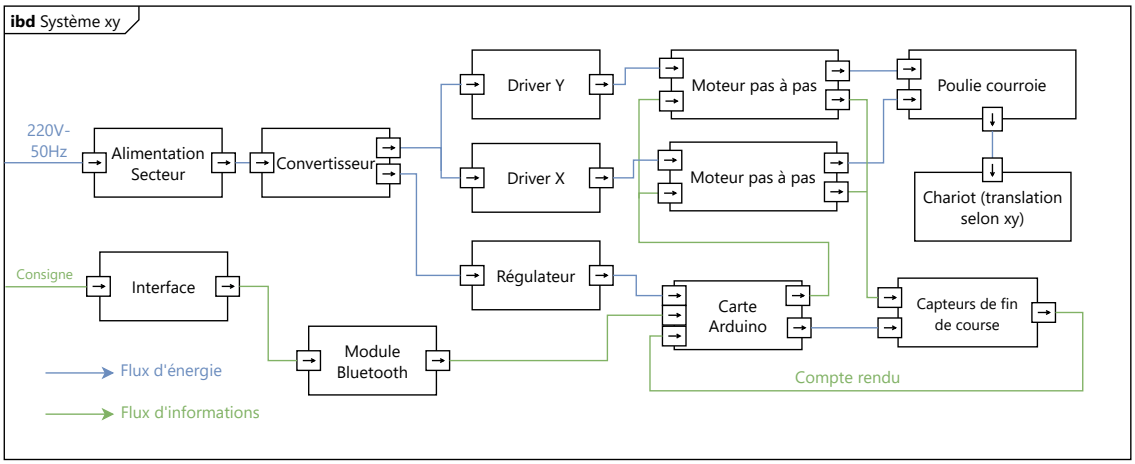


FIGURE 7 – Diagramme IBD du sous-système de déplacement en plan XY

1.5.3 Diagramme IBD du sous-système de déplacement selon Z

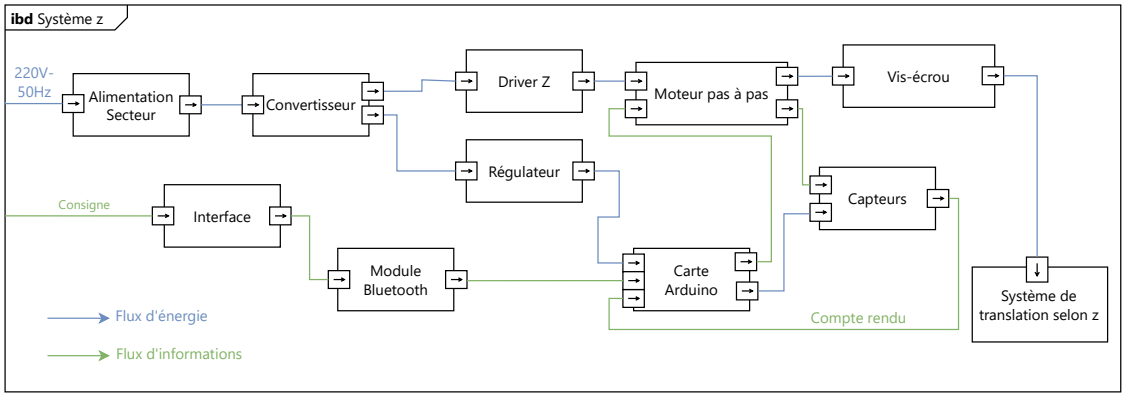


FIGURE 8 – Diagramme IBD du sous-système de déplacement selon Z

1.6 Schémas cinématiques

Ces schémas permettent de modéliser les mouvements relatifs de différentes parties du système entre elles. Ceci se fait par le biais de symboles normalisés qui représentent des degrés de liberté connus.

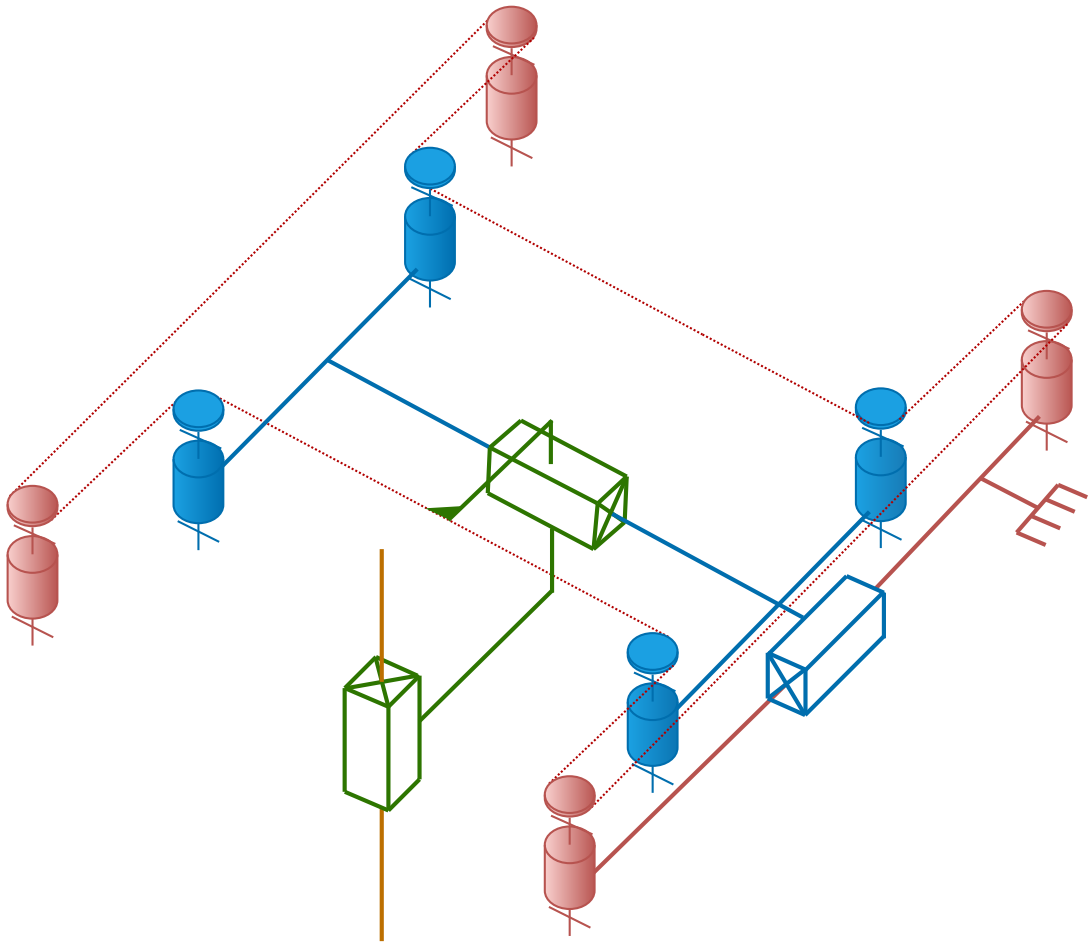


FIGURE 9 – Schéma cinématique en 3D du système

1.7 Schéma d'architecture

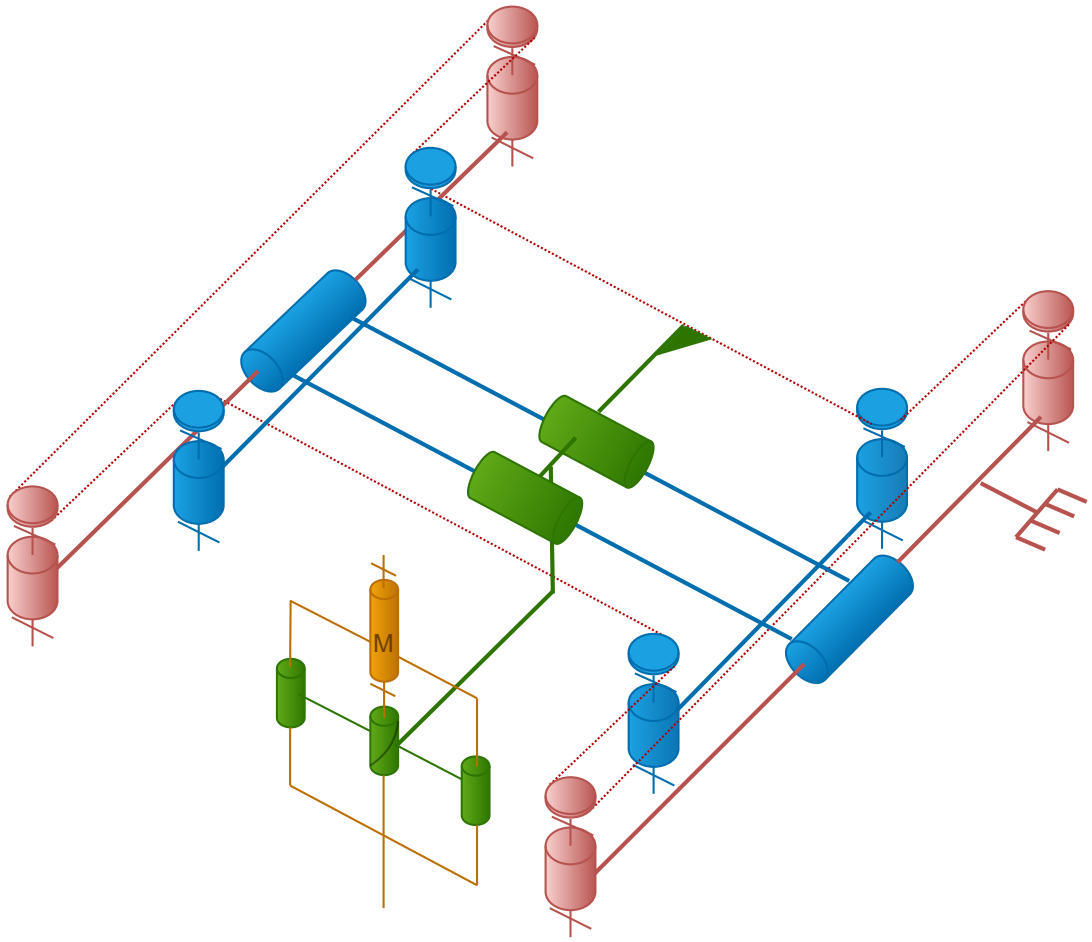


FIGURE 10 – Schéma d'architecture du système

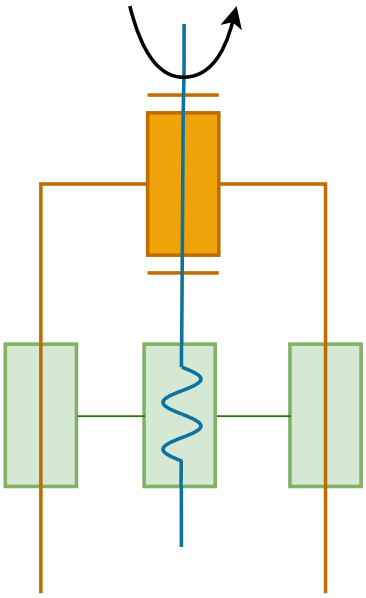


FIGURE 11 – Détail de l'architecture selon Z du système

1.8 Caractérisation du mouvement de la courroie

Paramétrage et hypothèses L'étude cinématique repose sur la modélisation de la courroie comme un lien parfaitement inextensible. La transmission de puissance au niveau des poulies motrices (P_1) et (P_2), de rayon identique R, s'effectue par roulement sans glissement. L'espace de travail du chariot est rapporté au repère orthonormé direct $(O, \vec{u}_x, \vec{u}_y)$. Le chariot est modélisé par son centre d'inertie M, dont les coordonnées (x, y) définissent la position du chariot.

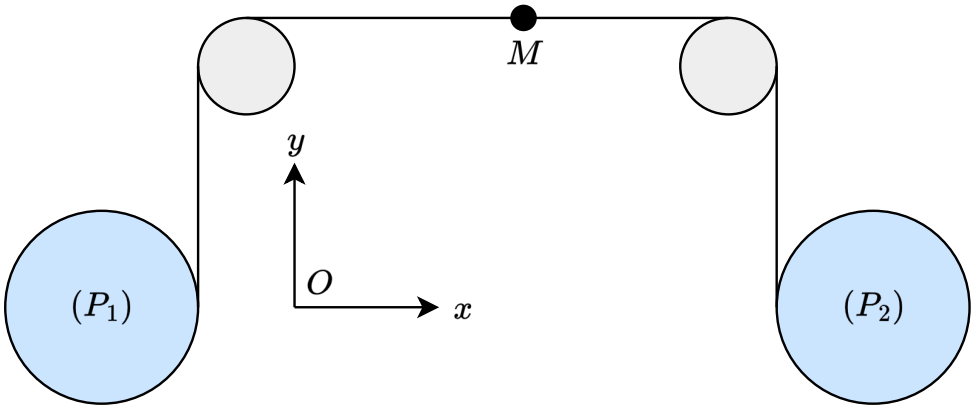


FIGURE 12 – Paramétrage cinématique du système d'entraînement

Fermeture géométrique et couplage Soient $l_1(x, y)$ et $l_2(x, y)$ les longueurs géométriques des brins de courroie reliant le chariot aux poulies (P_1) et (P_2). L'analyse d'un déplacement du chariot d'un point M_i vers M_f (Fig. 13) permet de visualiser l'influence des variations opérationnelles Δx et Δy sur la géométrie des brins.

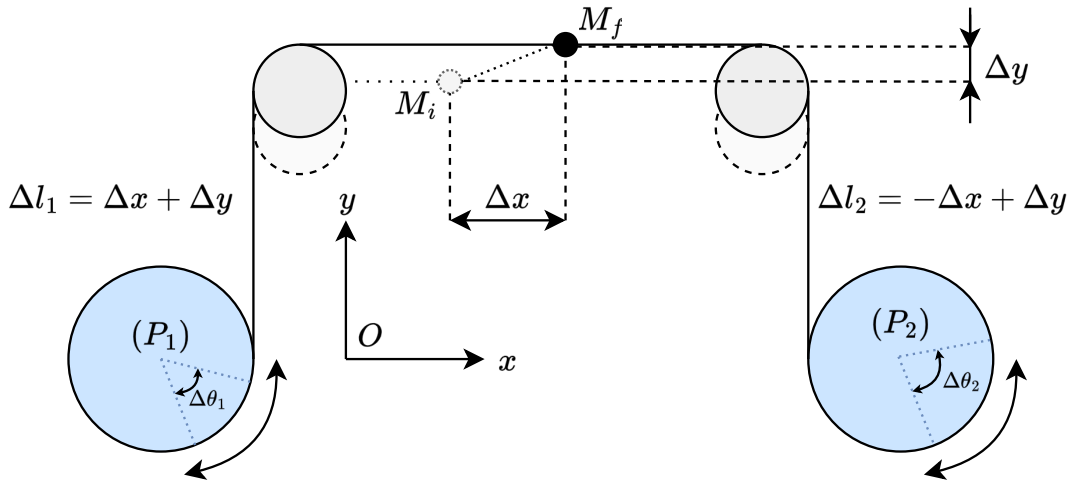


FIGURE 13 – Analyse d'un déplacement élémentaire du chariot de M_i vers M_f et répercussion sur les longueurs de courroie.

L'analyse de la construction du système permet d'exprimer la longueur totale de chaque brin de courroie en fonction de la position (x, y) du chariot. Soient $l_1(x, y)$ et $l_2(x, y)$ ces longueurs. En isolant les segments variables des segments de longueur fixe (enroulements sur

les poulies folles et tronçons constants, notés C_1 et C_2), le parcours des courroies impose les relations macroscopiques suivantes :

$$l_1(x,y) = x + y + C_1 \quad \text{et} \quad l_2(x,y) = -x + y + C_2 \tag{1}$$

Pour un déplacement élémentaire du chariot défini par les variations dx et dy, la différenciation de ces équations linéaires fournit directement les variations de longueur associées pour chaque brin :

$$dl_1 = dx + dy \quad \text{et} \quad dl_2 = -dx + dy \tag{2}$$

La condition de roulement sans glissement lie physiquement ces variations aux rotations angulaires $d\theta_1$ et $d\theta_2$ des moteurs. Par convention, une rotation trigonométrique ($d\theta_1 > 0$) libère une longueur de courroie $Rd\theta_1$ sur le brin 1. À l'inverse, une rotation $d\theta_2 > 0$ absorbe de la courroie sur le brin 2. En identifiant ces relations aux variations géométriques précédemment établies, nous obtenons les équations de couplage :

$$\begin{cases} Rd\theta_1 = dx + dy \\ -Rd\theta_2 = -dx + dy \end{cases} \implies \begin{cases} Rd\theta_1 = dx + dy \\ Rd\theta_2 = dx - dy \end{cases} \tag{3}$$

Par intégration temporelle, nous obtenons les relations macroscopiques :

$$\boxed{\Delta x + \Delta y = R\Delta\theta_1} \quad \text{et} \quad \boxed{\Delta x - \Delta y = R\Delta\theta_2} \tag{4}$$

Superposition cinématique (alternative)

Ces relations de couplage peuvent également être démontrées par le principe de superposition, autorisé par la linéarité du système. En bloquant arbitrairement l'une des deux poulies, on peut aboutir à un système d'équations pour $(\Delta x, \Delta y)$.

Modèle cinématique global La résolution du système (4) permet d'isoler les déplacements linéaires :

$$\begin{cases} \Delta x = \frac{R}{2}(\Delta\theta_1 + \Delta\theta_2) \\ \Delta y = \frac{R}{2}(\Delta\theta_1 - \Delta\theta_2) \end{cases} \tag{5}$$

Étude d'un déplacement d'angle φ Pour un déplacement rectiligne dans le plan orienté d'un angle φ par rapport à l'axe $\vec{u}_x$, le vecteur vitesse opérationnelle $\vec{V}$ s'exprime par ses projections orthogonales.

L'expression vectorielle de la vitesse est :

$$\vec{V} = V\cos(\varphi)\vec{u}_x + V\sin(\varphi)\vec{u}_y \tag{6}$$

L'injection des composantes $\dot{x} = V\cos(\varphi)$ et $\dot{y} = V\sin(\varphi)$ dans le modèle cinématique inverse permet d'établir les lois de commande articulaires :

$$\begin{cases} R\dot{\theta}_1 = V(\cos(\varphi) + \sin(\varphi)) \\ R\dot{\theta}_2 = V(\cos(\varphi) - \sin(\varphi)) \end{cases} \tag{7}$$

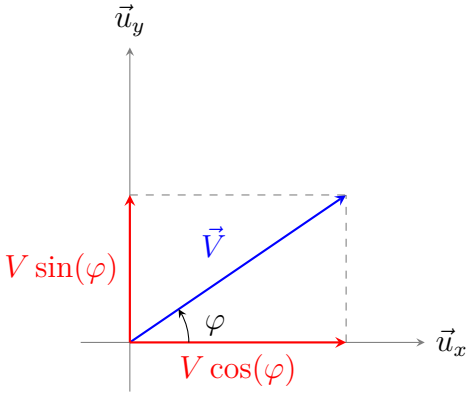


FIGURE 14 – Décomposition vectorielle du vecteur vitesse

Par intégration temporelle pour un déplacement de norme ΔL, les variations angulaires requises sont :

$$\boxed{R\Delta\theta_1 = \Delta L(\cos(\varphi) + \sin(\varphi))} \quad \text{et} \quad \boxed{R\Delta\theta_2 = \Delta L(\cos(\varphi) - \sin(\varphi))} \tag{8}$$

Nature du mouvement	**Relation articulaire ($\Delta\theta_1, \Delta\theta_2$)**	**Relation opérationnelle ($\Delta x, \Delta y$)**
Mouvement pur selon $\vec{u}_x$	$\Delta\theta_1 = \Delta\theta_2$	$\Delta y = 0$
Mouvement pur selon $\vec{u}_y$	$\Delta\theta_1 = -\Delta\theta_2$	$\Delta x = 0$
Diagonale d'angle φ	$R\Delta\theta_1 = \Delta L(\cos(\varphi) + \sin(\varphi))$ $R\Delta\theta_2 = \Delta L(\cos(\varphi) - \sin(\varphi))$	$\Delta x = \Delta L\cos(\varphi)$ $\Delta y = \Delta L\sin(\varphi)$

TABLEAU 2 – Lois de transformation cinématique selon l'axe de déplacement du chariot

1.9 Modélisation cinématique du porte-stylet (axe Z)

L'étude cinématique de l'axe vertical repose sur la transformation du mouvement de rotation du moteur en une translation rectiligne par un système vis-écrou. L'espace de travail est rapporté à l'axe vertical $(O, \vec{u}_z)$. Le porte-stylet est modélisé par son centre d'inertie M_z, dont la coordonnée z définit la position opérationnelle. Le système est caractérisé par son pas de vis p (déplacement linéaire pour un tour complet).

On note θ_z l'angle de rotation du moteur de l'axe Z et p le pas de la vis, c'est-à-dire le déplacement linéaire obtenu pour un tour complet de la vis.

Pour un tour complet du moteur, la vis effectue une rotation de 2π radians et le porte-stylet se déplace d'une distance égale au pas p :

$$\Delta\theta_z = 2\pi \quad \Longrightarrow \quad \Delta z = p \tag{9}$$

Par proportionnalité, pour une rotation quelconque $\Delta\theta_z$, le déplacement vertical correspondant est :

$$\Delta z = \frac{p}{2\pi}\Delta\theta_z \tag{10}$$

Inversement, pour imposer un déplacement vertical Δz, la rotation moteur nécessaire est :

$$\Delta\theta_z = \frac{2\pi}{p}\Delta z \tag{11}$$

En fonction de la position verticale initiale z_0, la position du porte-stylet peut donc s'écrire :

$$\boxed{z = z_0 + \frac{p}{2\pi}\Delta\theta_z} \tag{12}$$

Ce modèle géométrique permet ainsi de relier directement l'angle de rotation du moteur de l'axe Z au déplacement vertical du porte-stylet.

1.10 Études de dimensionnement

1.10.1 Dimensionnement des poulies (résolution)

La résolution du système est directement conditionnée par **la précision angulaire des moteurs pas à pas et le rayon R des poulies motrices**. Les moteurs utilisés présentent une résolution native de $1,8°$ par pas, soit $N = 200$ pas par tour. La variation angulaire minimale d'un moteur est donc :

$$\delta\theta = \frac{2\pi}{N} = \frac{\pi}{100}\ \text{rad} \tag{13}$$

D'après le modèle cinématique, un déplacement pur selon l'axe $\vec{u}_x$ (où $\Delta\theta_1 = \Delta\theta_2 = \Delta\theta$) est lié à la rotation motrice par la relation $\Delta x = R\Delta\theta$. La résolution linéaire du traceur δx est donc définie par :

$$\delta x = R \cdot \delta\theta \tag{14}$$

Le cahier des charges impose une précision linéaire $\delta x \leq 0,2\,\text{mm}$. Nous en déduisons la condition sur le rayon maximal de la poulie :

$$R \leq \frac{\delta x}{\delta\theta} \implies \boxed{R_{max} = \frac{0,2 \cdot 100}{\pi} \approx 6,36\,\text{mm}} \tag{15}$$

1.10.2 Dimensionnement des moteurs du système de courroie (axes X et Y)

L'objectif de cette étude est de déterminer le couple moteur nécessaire C pour assurer le déplacement du chariot selon les axes X et Y à travers le système de courroie.

Choix du système et hypothèses Le système étudié (présenté dans la figure 15) est constitué des sous-ensembles suivants :

- le chariot de masse m_c,
- la traverse de masse m_t,
- l'ensemble des 8 poulies de masse m_p chacune (2 motrices et 6 folles),
- les courroies de masse négligeable.

Les hypothèses de travail sont les suivantes :

- les courroies sont parfaitement inextensibles,
- la transmission de puissance s'effectue par roulement sans glissement au niveau des poulies,

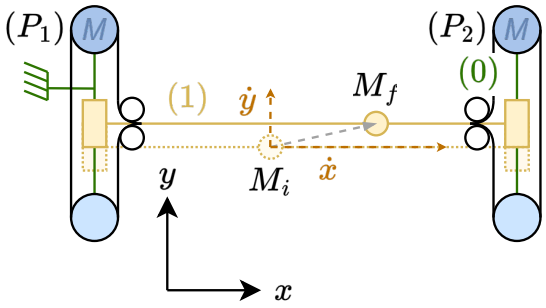


FIGURE 15 – Schéma de modélisation dynamique du système de courroie dans le plan XY (simplifié pour l'étude de dimensionnement)

Les forces de frottement sont modélisées par des forces de frottement sec F_x et F_y pour le mouvement du chariot sur la traverse (1) et le mouvement de la traverse sur la traverse (0) respectivement.

Bilan cinématique L'énergie cinétique du système par rapport au référentiel galiléen (le bâti) est la somme des énergies cinétiques de chaque composant :

Composant	**Énergie cinétique**	**Expression en fonction de $\dot{x}$ et $\dot{y}$**
Chariot seul	$E_{c,c} = \frac{1}{2}m_c(\dot{x}^2 + \dot{y}^2)$	
Traverse + Poulies associées	$E_{c,t} = \frac{1}{2}(m_t + 4m_p)\dot{y}^2$	
Poulies	$E_{c,p} = 8\left(\frac{1}{2}J_{O,z}(\dot{\theta}_1^2 + \dot{\theta}_2^2)\right)$	$E_{c,p} = \frac{8J_{O,z}}{R^2}\left(\dot{x}^2 + \dot{y}^2\right)$

TABLEAU 3 – Récapitulatif des énergies cinétiques de chaque composant du système de courroie

L'énergie cinétique totale du système est donc :

$$E_c = E_{c,c} + E_{c,t} + E_{c,p} = \frac{1}{2}m_c(\dot{x}^2 + \dot{y}^2) + \frac{1}{2}(m_t + 4m_p)\dot{y}^2 + \frac{8J_{O,z}}{R^2}(\dot{x}^2 + \dot{y}^2) \tag{16}$$

Bilan des puissances et efforts mécaniques La puissance des actions mécaniques appliquées au système se divise en deux catégories :

- les puissances motrices P_m fournies par les moteurs au niveau des poulies motrices,
- les puissances résistantes P_r dues aux forces de frottement.

La puissance motrice fournie par chacun des moteurs est donnée par :

$$P_{m,1} = C_1\dot{\theta}_1 \quad \text{et} \quad P_{m,2} = C_2\dot{\theta}_2 \tag{17}$$

Ainsi, l'expression de la puissance motrice totale est :

$$P_m = P_{m,1} + P_{m,2} = C_1\dot{\theta}_1 + C_2\dot{\theta}_2 \tag{18}$$

La puissance dissipée par les forces de frottement est donnée par :

$$P_r = -F_x\dot{x} - F_y\dot{y} \tag{19}$$

Application du théorème de la puissance cinétique Le théorème stipule que la somme des puissances motrices et résistantes est égale à la dérivée de l'énergie cinétique du système :

$$P_m + P_r = \frac{dE_c}{dt} \tag{20}$$

L'équation globale de la dynamique du système s'écrit donc :

$$C_1\frac{\dot{x}+\dot{y}}{R} + C_2\frac{\dot{x}-\dot{y}}{R} - F_x\dot{x} - F_y\dot{y} = m_c\dot{x}\ddot{x} + m_c\dot{y}\ddot{y} + (m_t + 4m_p)\dot{y}\ddot{y} + \frac{8J_{O,z}}{R^2}(\dot{x}\ddot{x} + \dot{y}\ddot{y}) \tag{21}$$

Détermination du couple minimal Pour dimensionner les moteurs, il faut évaluer le cas de charge le plus contraignant, à savoir un mouvement en diagonale ($\dot{x} = \dot{y} = V$) avec une accélération maximale ($\ddot{x} = \ddot{y} = a_{max}$). Pour un mouvement strictement diagonal, les relations cinématiques imposent $\dot{\theta}_2 = 0$. La puissance motrice totale est fournie par un seul moteur (le moteur 1). En injectant ces conditions dans l'équation de la dynamique, on trouve :

$$C_1\frac{2V}{R} - F_xV - F_yV = 2m_cVa_{max} + 2(m_t + 4m_p)Va_{max} + \frac{16J_{O,z}}{R^2}Va_{max} \tag{22}$$

Cela permet d'isoler le couple moteur minimal nécessaire pour assurer le déplacement du chariot selon les axes X et Y :

$$\boxed{C_1 = \frac{R}{2}\left[2m_ca_{max} + 2(m_t + 4m_p)a_{max} + \frac{16J_{O,z}}{R^2}a_{max} + (F_x + F_y)\right]} \tag{23}$$

Paramètre	Valeur
m_c (masse du chariot)	1 kg
m_t (masse de la traverse)	1 kg
m_p (masse d'une poulie)	$0,2$ kg
R (rayon des poulies)	$6,36$ mm
$J_{O,z}$ (moment d'inertie d'une poulie)	$= \frac{1}{2} m_p R^2 \approx 4 \times 10^{-6}$ kg·m^2
F_x (force de frottement chariot-traverse)	1 N
F_y (force de frottement traverse-bâti)	1 N
V (vitesse maximale)	$0,08$ m/s
a_{max} (accélération maximale)	$0,1$ m/s^2

TABLEAU 4 – Paramètres utilisés pour le dimensionnement des moteurs du système de courroie

En injectant les valeurs numériques des paramètres dans l'expression du couple moteur, on trouve :

$$C_{\text{min}} \approx 8,64 \times 10^{-3} \text{ N·m} \tag{24}$$

Ce couple moteur minimal doit être pris en compte lors du choix des moteurs pour assurer un fonctionnement fiable et performant du système de courroie.

1.10.3 Dimensionnement du moteur du système vis-écrou (axe Z)

L'objectif de cette étude est de déterminer le couple moteur nécessaire C_z pour assurer le déplacement vertical du porte-stylet selon l'axe Z à travers le système vis-écrou. Le cas évalué est le cas le plus contraignant pour le moteur, à savoir une montée verticale du porte-stylet.

Choix du repère et du système On choisit l'axe Z vertical orienté vers le haut. Le système étudié est l'ensemble mobile selon l'axe Z, de masse m_z.

Les forces appliquées au système sont les suivantes :

- la force motrice transmise par la vis : $\vec{F}_m = F_m \, \vec{e}_z$,
- le poids : $\vec{P} = m_z \vec{g} = -m_z g \, \vec{e}_z$,
- les forces résistantes (frottements) : $\vec{F}_{R,z} = F_{R,z} \, \vec{e}_z$.

En appliquant le principe fondamental de la dynamique selon l'axe Z, on établit l'équation du mouvement :

$$\sum F_z = m_z a_z \implies F_m - m_z g - F_{R,z} = m_z a_z \tag{25}$$

On peut donc en déduire l'expression de la force motrice nécessaire pour assurer une accélération a_z selon l'axe Z :

$$\boxed{F_m = m_z(g + a_z) + F_{R,z}} \tag{26}$$

Conversion force-couple (vis-écrou) Nous devons maintenant déterminer le couple moteur C_z nécessaire pour générer la force motrice F_m à travers le système vis-écrou. On définit

les paramètres suivants :

- p le pas de la vis (m/tour),
- η le rendement global du système vis–écrou,
- C_z le couple moteur appliqué sur l'axe Z.

Le travail fourni par le moteur pour un tour de vis est :

$$W_m = 2\pi C_z \tag{27}$$

Le travail utile transmis à la charge est :

$$W_u = F_m\, p \tag{28}$$

En intégrant le rendement η du système, on établit la relation entre le travail moteur et le travail utile :

$$W_u = \eta W_m \tag{29}$$

En remplaçant les expressions de W_u et W_m, on trouve :

$$F_m p = \eta(2\pi C_z) \tag{30}$$

On en déduit **l'expression du couple moteur nécessaire** pour assurer le déplacement selon l'axe Z :

$$C_z = \frac{F_m\, p}{2\pi\eta} \tag{31}$$

En remplaçant F_m par son expression en fonction de la masse, de l'accélération et des forces résistantes (équation 26), on trouve :

$$\boxed{C_z = \frac{\left[m_z(g + a_z) + F_{R,z}\right] p}{2\pi\eta}} \tag{32}$$

Détermination du rendement η du système vis-écrou Nous devons maintenant déterminer le rendement η du système vis-écrou.

On assimile le mouvement d'une vis-écrou à celui d'un solide se déplaçant sur un plan incliné obtenu en déroulant un tour de filet sur le diamètre moyen d_m.
Sur un tour de vis, l'avance axiale est égale au pas p et la longueur parcourue sur la génératrice vaut πd_m. L'angle d'hélice λ vérifie donc :

$$\tan\lambda = \frac{p}{\pi d_m} \tag{33}$$

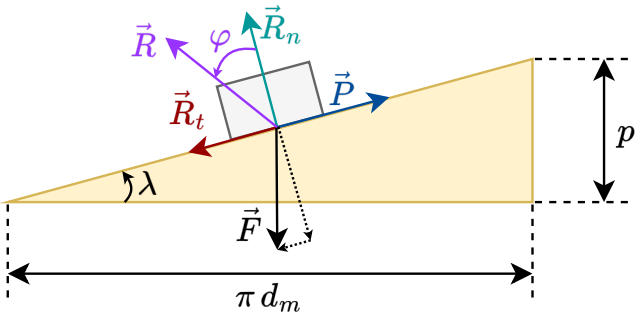


FIGURE 16 – Modélisation du mouvement d'une vis-écrou par un solide sur un plan incliné

En l'absence de frottement, l'effort tangent P_0 (sans frottements) nécessaire pour soulever une charge axiale F est :

$$P_0 = F \tan \lambda. \tag{34}$$

Nous allons maintenant déterminer l'effort avec frottement. On introduit l'angle de frottement φ défini par :

$$\tan \varphi = \mu, \tag{35}$$

où μ est le coefficient de frottement entre la vis et l'écrou.

Pour la montée de la charge, l'effort tangent réel devient :

$$P = F \tan(\lambda + \varphi). \tag{36}$$

On utilise la formule trigonométrique de l'addition pour la tangente :

$$\tan(\lambda + \varphi) = \frac{\tan \lambda + \tan \varphi}{1 - \tan \lambda \tan \varphi}, \tag{37}$$

Pour ensuite obtenir l'expression de l'effort réel P en fonction de la charge F, de l'angle d'hélice λ et du coefficient de frottement μ :

$$P = F \frac{\tan \lambda + \mu}{1 - \mu \tan \lambda}. \tag{38}$$

Le rendement η est défini comme le rapport entre l'effort utile idéal et l'effort réel :

$$\eta = \frac{P_0}{P}. \tag{39}$$

En substituant les expressions précédentes, on trouve l'expression suivante :

$$\eta = \frac{F \tan \lambda}{F \dfrac{\tan \lambda + \mu}{1 - \mu \tan \lambda}} = \frac{\tan \lambda (1 - \mu \tan \lambda)}{\tan \lambda + \mu}. \tag{40}$$

Dans le cas des vis à faible angle d'hélice, on a :

$$\mu \tan \lambda \ll 1 \implies 1 - \mu \tan \lambda \approx 1 \tag{41}$$

On obtient alors l'expression approchée du rendement :

$$\boxed{\eta \approx \frac{\tan \lambda}{\tan \lambda + \mu}} \tag{42}$$

Expression finale du couple moteur En remplaçant le rendement η (équation 42) dans l'expression du couple moteur (équation 32), on trouve :

$$C_z = \frac{\left[m_z(g + a_z) + F_{R,z}\right] p}{2\pi} \cdot \frac{\tan \lambda + \mu}{\tan \lambda} \tag{43}$$

Paramètre	Valeur
Masse m_z	$0,5$ kg
Accélération a_z	$0,1$ m/s^2
Force de frottement $F_{R,z}$	1 N
Pas de la vis p	2 mm
Angle d'hélice λ	$5°$
Coefficient de frottement μ	$0,1$

TABLEAU 5 – Paramètres pour l'application numérique du couple moteur

En injectant les valeurs numériques des paramètres (Tableau 5), on trouve :

$$C_z \approx 4,130 \times 10^{-3} \text{ N} \cdot \text{m} \qquad (44)$$

Cette estimation du couple moteur nécessaire pour assurer le déplacement selon l'axe Z nous permettra de sélectionner un moteur adapté lors de la phase de conception.

Impact de l'inertie de rotation de la vis

L'influence de l'inertie de rotation de la vis a été évaluée. Étant donné les faibles accélérations ($0,1$ m/s^2) et les dimensions de la vis, le couple dynamique (lié aux inerties du système) reste négligeable devant le couple statique $\frac{F_m\, p}{2\pi\eta}$.
Par conséquent, le dimensionnement du moteur peut être réalisé en se basant uniquement sur le couple statique nécessaire pour soulever la charge, sans prendre en compte les effets dynamiques.

1.10.4 Récapitulatif des résultats de dimensionnement

Moteur	Couple minimal requis
Moteurs du système de courroie (axes X et Y)	$C_{\text{min}} \approx 8,64 \times 10^{-3}$ N·m
Moteur du système vis-écrou (axe Z)	$C_z \approx 4,130 \times 10^{-3}$ N·m

TABLEAU 6 – Récapitulatif des couples moteurs minimaux requis pour les différents systèmes d'entraînement du traceur de plan

2 CONCEPTION ET MÉCANIQUE

```
++++++++++[>+++++++>++++++++++>++++++++++>++++++++++>++++++
++++>++++++++++>++++++++++>++++++++++>++++++++++>+++++++
++++>+++>++++++++++>++++++++++>+++>++++++++>++++++++++++++
+++++>++++++++++++++++>++++++++++>++++++++++>++++++++++>+++
++++++++>++++++++++>++++++++++>++++++++++<<<<<<<<<<<<<<<<<<
<<<<<<<-]>---.>+.>.>-.>+.>++.>----.>-----.>+.>.>++.>+.>----.>
++.>---.>-----.>-.>-.>---.>.>-----.>+++.>---.>+.
```

2.1 Maquette SolidWorks

2.1.1 Vue d'ensemble

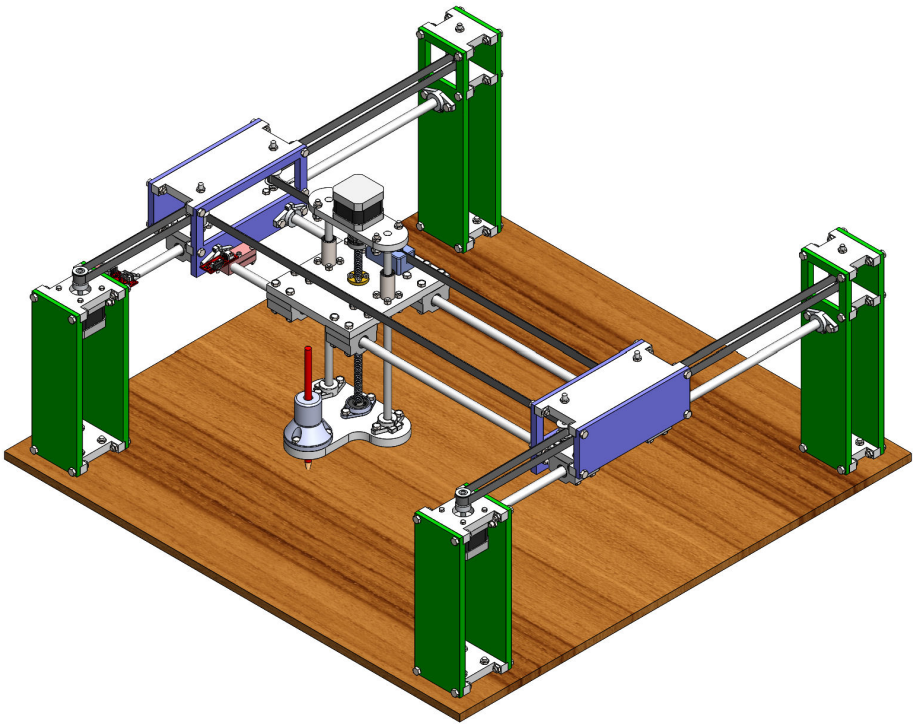


FIGURE 17 – Maquette 3D de l'assemblage final du système cartésien.

Le système est décomposé en trois sous-ensembles principaux, repérables par leurs couleurs dans la CAO :

- **SE0 –Ensemble de base** (bâti marron et colonnes de support vertes, Fig. 18) : structure statique comprenant bâti, colonnes de support moteur/poulies, poulies fixes, axes de guidage fixes et capteurs de fin de course.
- **SE1 –Ensemble mobile horizontal** (pièces de traverse mauve/lavande, Fig. 19) : axes de guidage mobiles, poulies mobiles, courroies et capteur de fin de course. Responsable du mouvement dans le plan XY.
- **SE2 –Ensemble mobile vertical** (classe d'équivalence de translation selon Z grise, Fig. 20) : chariot, axe Z (moteur, vis-écrou) et capteur de fin de course associé.

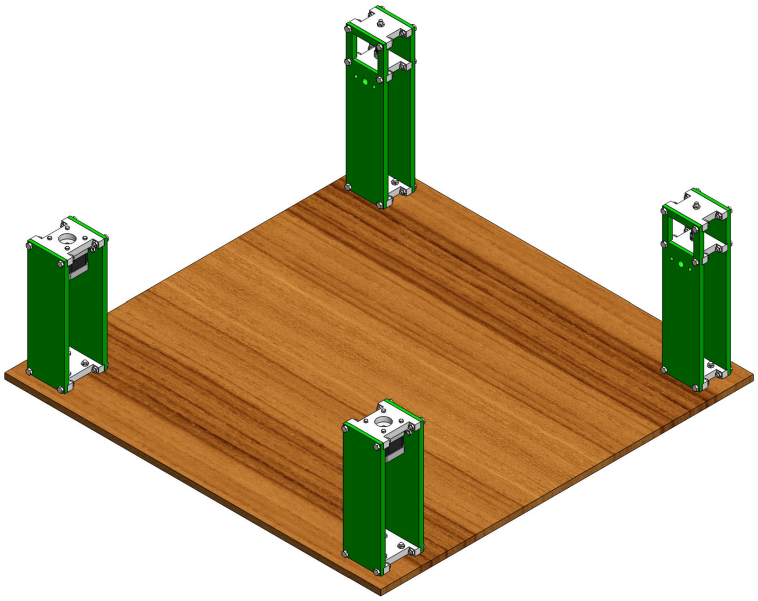


FIGURE 18 – Vue du sous-ensemble SE0 – Base

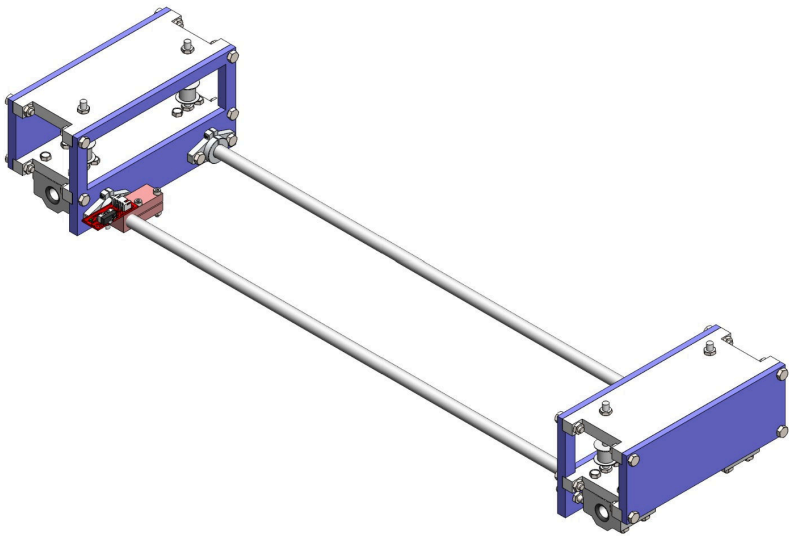


FIGURE 19 – Vue du sous-ensemble SE1 – Mobile horizontal

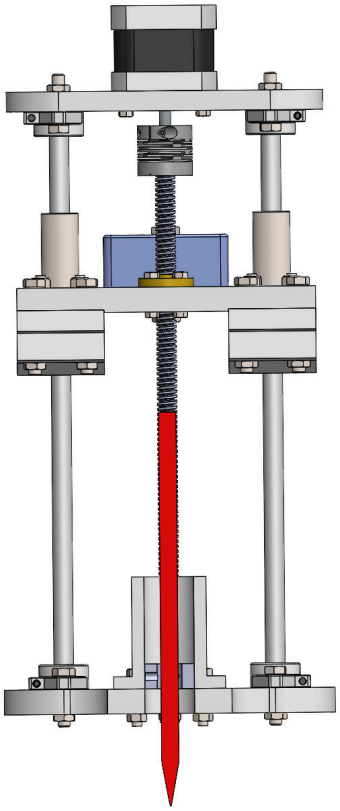


FIGURE 20 – Vue du sous-ensemble SE2 – Mobile vertical

2.2 Pièces conçues

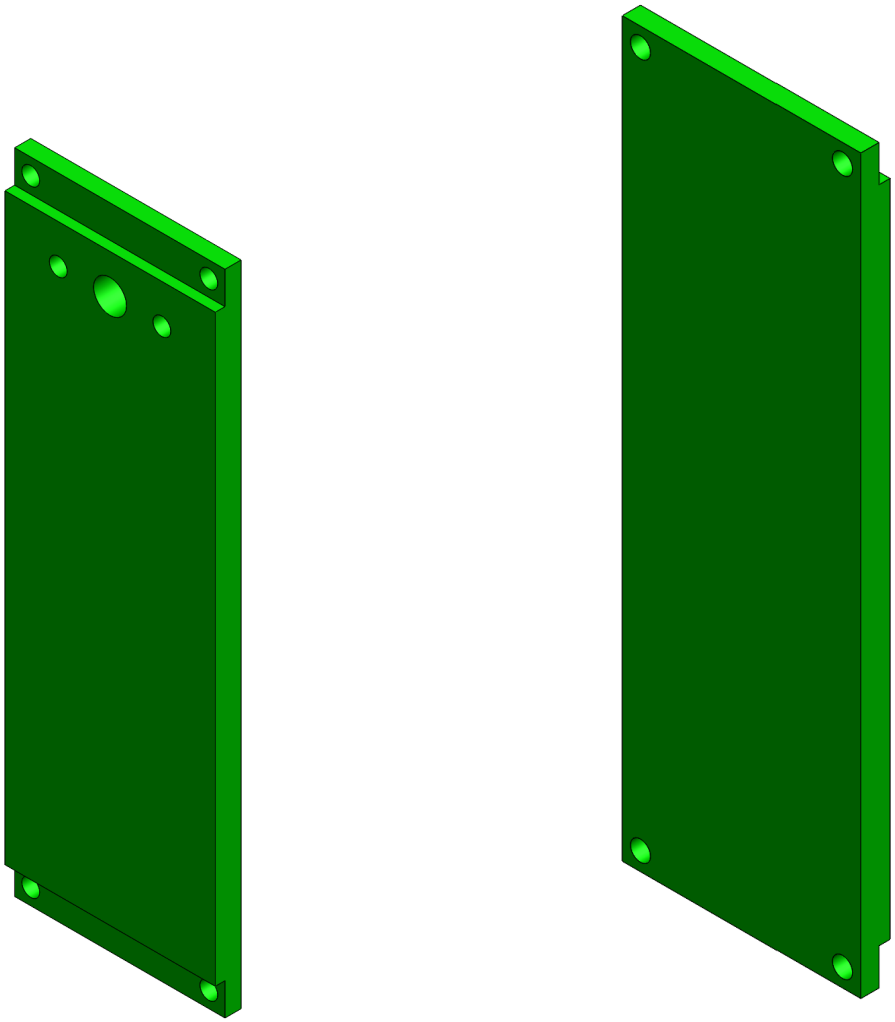
2.2.1 Pièces structurelles



FIGURE 21 – Bâti (HPL, marron).

Le bâti, de teinte marron dans la maquette, constitue la pièce maîtresse. Fabriqué en HPL, il accueille les colonnes de support et assure la rigidité de l'ensemble tout en intégrant un

boîtier pour l'électronique.

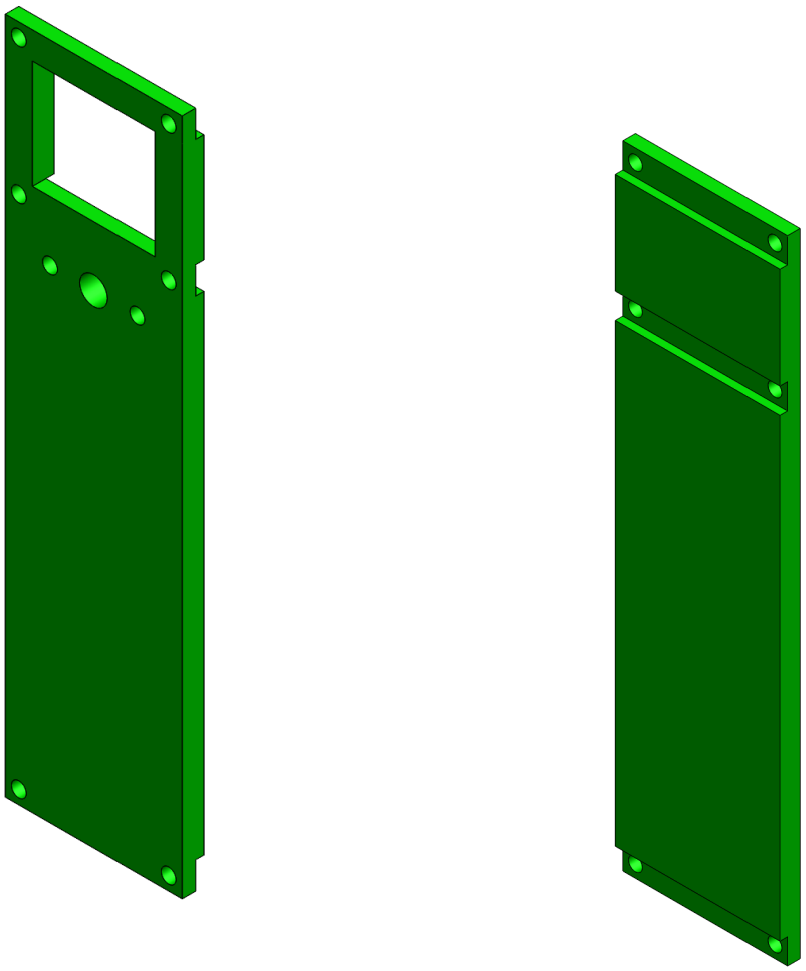


(a) Version percée (b) Version sans perçage

FIGURE 22 – Colonnes de support moteur (HPL, vertes).

Les colonnes moteur, colorées en vert, existent en deux versions :

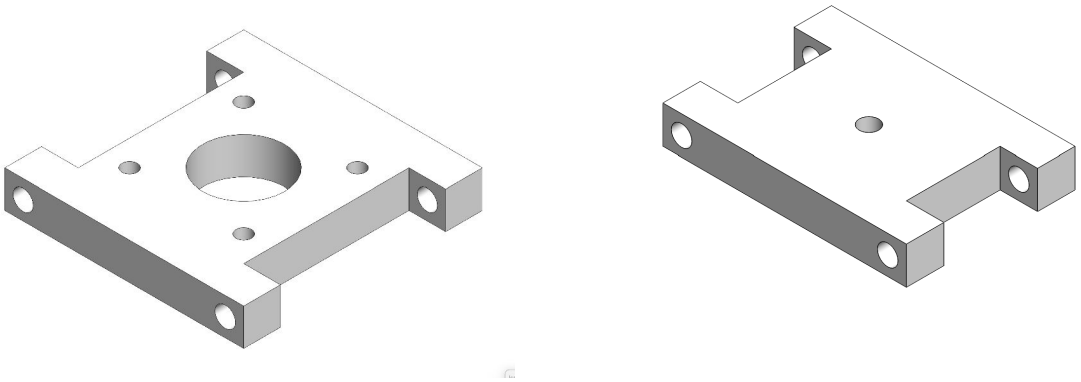
— **Percée** : fixation du moteur + encastrement des axes de guidage cylindriques par pincement. Présence de chambrages pour les pièces intermédiaires.
— **Sans perçage** : identique mais sans les perçages de pincement des axes.



(a) Version percée (b) Version sans perçage

FIGURE 23 – Colonnes de support poulie libre (HPL, vertes).

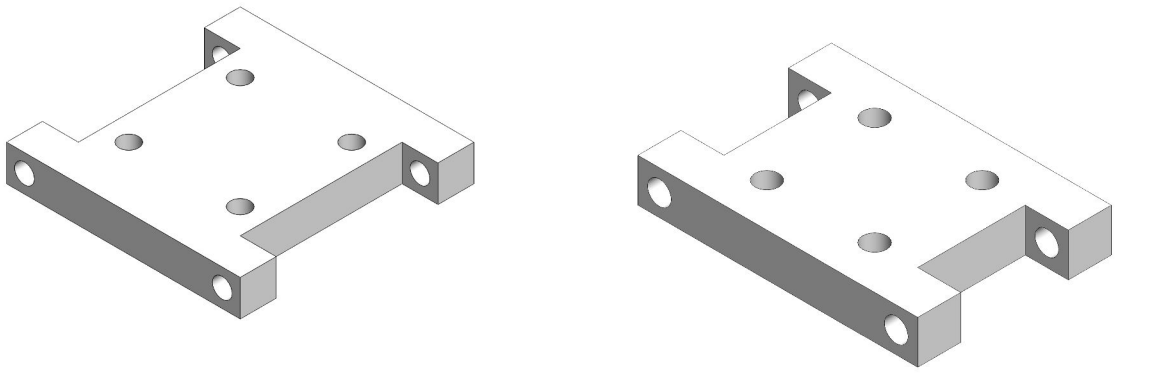
Même principe que les colonnes moteur, avec une version percée pour le pincement de l'axe et une version sans.



(a) Support moteur (PVC) (b) Support poulie libre (PVC)

FIGURE 24 – Supports moteur et poulie libre.

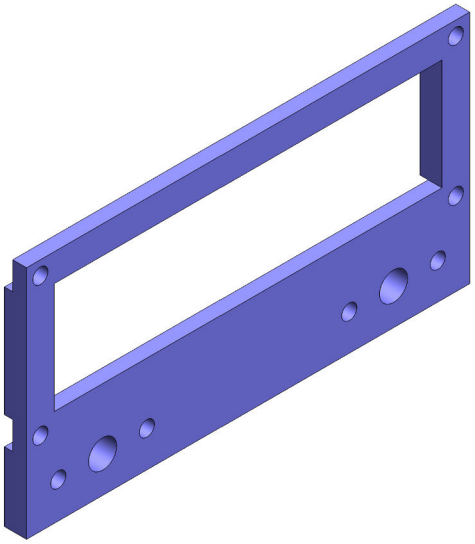
— **Support moteur** : maintient le moteur entre deux colonnes vertes, fixé par 4+4 vis.
— **Support poulie libre** : fixe la poulie libre en translation, rotation libre sur tige filetée, fixation par 4 vis.



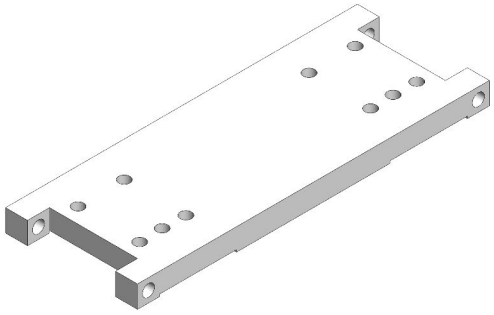
(a) Colonne moteur (HPL) (b) Colonne poulie libre (PVC)

FIGURE 25 – Supports inférieurs de liaison des colonnes.

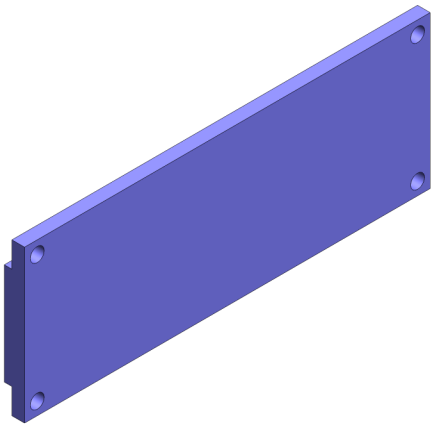
Ces pièces solidarisent les paires de colonnes entre elles, sans contact avec les axes de guidage.



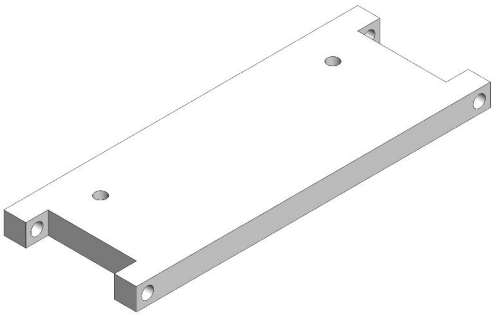
(a) Plaque côté traverse (HPL)



(b) Plaque intermédiaire (PVC)



(c) Plaque opposée (HPL)



(d) Plaque intermédiaire supérieure (PVC)

FIGURE 26 – Plaques de liaison et intermédiaires du sous-assemblage douilles (teintes mauve/lavande).

Ces quatre plaques, teintées dans les tons mauve/lavande, forment l'ossature de SE1 :

- **Plaque côté traverse** : liaison entre axes de guidage et sous-ensemble glissière, chambrage central pour la plaque intermédiaire et évidement pour la courroie.
- **Plaque intermédiaire** : encastre les douilles, relie les plaques latérales, comporte 2 perçages pour tiges filetées de poulies.
- **Plaque opposée** : fixe la plaque intermédiaire supérieure et la plaque intermédiaire douilles, sans contact avec les axes.
- **Plaque intermédiaire supérieure** : relie les deux plaques latérales, sans perçages pour douilles.

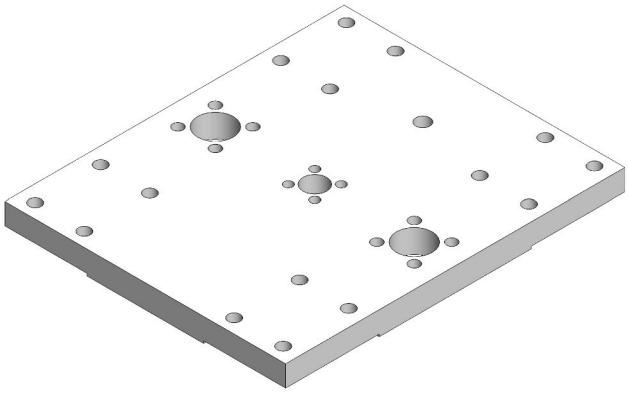
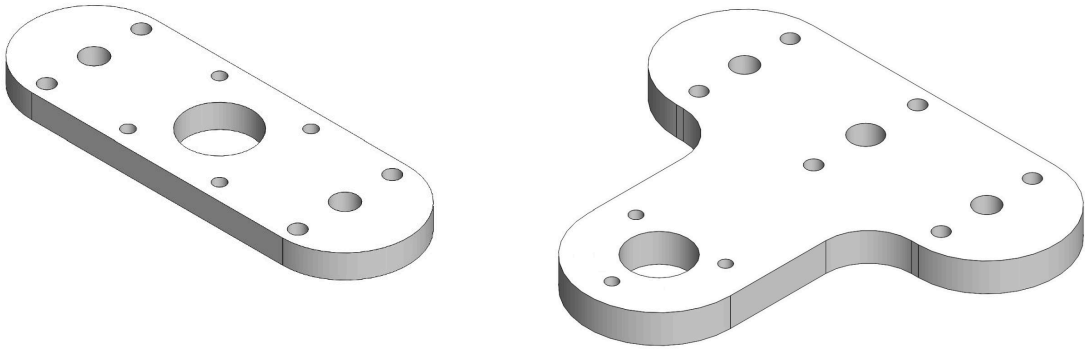


FIGURE 27 – Pièce centrale du chariot (HPL, intégrée au SE2 gris).

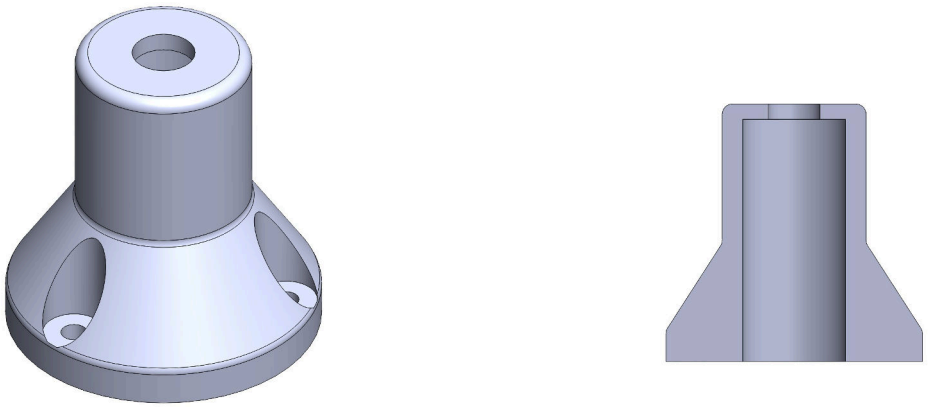
Elle supporte l'axe Z et comporte : 4 chambrages pour douilles latérales, 3 grands perçages centraux (central pour écrou, latéraux pour douilles cylindriques), 4 perçages pour le support de fin de course et 1 perçage pour l'ancrage courroie.



(a) Support moteur Z (HPL) (b) Support inférieur Z (HPL)

FIGURE 28 – Supports de l'axe vertical (SE2 gris).

— **Support moteur Z** : longue pièce avec évidement central pour l'arbre moteur, fixation moteur par 4 vis, 6 perçages latéraux pour pincements des rails Z.
— **Support inférieur Z** : 3 ensembles de perçages alignés (central pour palier à roulement, latéraux pour pincement des rails), évidement pour le passage du stylet.



(a) Porte-stylet externe (avec nervures) (b) Coupe du porte-stylet

FIGURE 29 – Composants du porte-stylet (impression 3D).

Suite aux problèmes de fléchissement analysés en Section 6.2, le porte-stylet a été redessiné :

- **Ressort à raideur plus faible** (asservissement en position),
- **Nervures circulaires** sur la pièce externe pour mieux encaisser les efforts latéraux.

Ces modifications sont détaillées dans la section “ Tests et validation ” (plus particulièrement en 6.3).

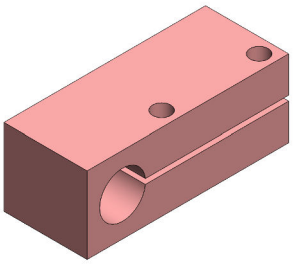


FIGURE 30 – Support de fin de course (impression 3D).

Pièce simple réalisant un pincement autour d’un rail cylindrique. Le capteur et le serrage sont fixés par vis-écrou.

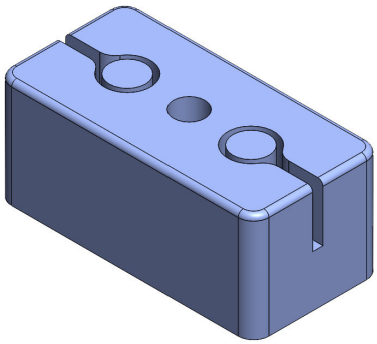


FIGURE 31 – Pièce d'ancrage de la courroie (impression 3D).

Deux passages latéraux pour enrouler et diviser la courroie. Largeur déterminée empiriquement (environ 2,3 mm). Un perçage central assure la fixation au chariot.

2.2.2 Pièces de guidage et de transmission



FIGURE 32 – Rails de guidage cylindriques (acier).

Deux longueurs sont disponibles :

- **Rails courts (300 mm)** pour l'axe vertical Z,
- **Rails longs (500 mm)** pour les axes horizontaux X et Y.



FIGURE 33 – Tiges filetées (acier).

— **Tige filetée pour poulies libres** : maintien en rotation des poulies.
— **Tige filetée pour l'axe** Z (300 mm) : transformation vis-écrou de la rotation du moteur en translation verticale.

2.3 Solutions de liaison glissière

Le guidage linéaire repose sur l'association **rails cylindriques + douilles à billes**, identique sur les trois axes. Le roulement des billes assure un mouvement fluide et une précision adaptée au prototype. Deux rails parallèles avec 3 ou 4 douilles suppriment tous les degrés de liberté sauf la translation désirée.



Le principal risque est l'hyperstatisme : un mauvais alignement des rails peut provoquer un arc-boutement. Le montage a été réalisé avec soin pour l'éviter.

2.4 Solutions d'encastrement

Le tableau 7 récapitule les liaisons fixes. Certaines solutions ont été fusionnées car elles partagent les mêmes MIP et MAP (à la taille de vis près).

Pièce 1	Pièce 2	MIP	MAP	Q
Support capteur fin de course	Plaque chariot	Appui plan	Vis-écrou	1
Douilles	Plaque chariot	Chambrage	Vis-écrou	8
Porte stylo	Support moteur Z	Chambrage	Vis-écrou	1
Support moteur Z inférieur	Rails Z	Pincement	Pincement	2
Plaque chariot	Rails Z	Douilles	Douilles	2
Support moteur Z	Rails Z	Pincement	Pincement	2
Douilles	Plaque traverse intermédiaire	Chambrage	Vis-écrou	2
Tige Y	Plaque traverse	Pincement	Pincement	4
Tige X	Colonne	Pincement	Pincement	4
Support colonne	Colonne	Chambrage	Vis-écrou	20
Support moteur	Moteur	Appui plan	Vis-écrou	3
Plaque opposée traverse	Plaque traverse intermédiaire	Chambrage	Vis-écrou	2
Plaque traverse	Plaque traverse intermédiaire	Chambrage	Vis-écrou	2
Plaque opposée traverse	Plaque traverse intermédiaire sup.	Chambrage	Vis-écrou	2
Plaque traverse	Plaque traverse intermédiaire sup.	Chambrage	Vis-écrou	2
Tige filetée	Plaque traverse intermédiaire sup.	Écrou	Écrou	4
Porte stylo interne	Porte stylo externe	Ressort + nervures	Vis de pression	1
Tige filetée	Support moteur Z	Palier roulement	Palier roulement	1
Tige filetée	Support	Écrou	Écrou	4
Tige filetée	Plaque traverse intermédiaire	Écrou	Écrou	4
Tige filetée	Plaque chariot	Écrou cylindrique	Écrou cylindrique	1
Ancrage courroie	Plaque chariot	Appui plan	Vis-écrou	1

TABLEAU 7 – Tableau des liaisons d'encastrement.

3 FABRICATION ET ASSEMBLAGE

```
++++++++++[>+++++++>++++++++++>++++++++++>++++++++++>+++++++
++++>++++++++++>++++++++++>++++++++++>++++++++++>+++++++++
++>++++++++++>+++>++++++++++>++++++++++>+++>++++++++++>+++
+++++++++>++++++++++>++++++++++>++++++++++>++++++++++>+++++
+++++>++++++++++>++++++++++<<<<<<<<<<<<<<<<<<<<<<-]>.>---.>
--.>++++.>-----.>-.>---.>----.>-----.>+.>.>++.>+.>----.>++.>-
--.>-----..>+.>-.>--.>--.>---.>+++.>+.
```

3.1 Synthèse des pièces fabriquées

La réalisation du traceur a nécessité la fabrication de plusieurs pièces mécaniques assurant les fonctions de guidage, de support, de liaison et de positionnement des différents sous-ensembles. Les principaux matériaux utilisés sont le HPL, le PVC et les pièces imprimées en 3D.

TABLEAU 8 – Synthèse des pièces fabriquées et procédés associés

Pièce	Matériau	Procédé	Fonction
Supports inférieur et supérieur de colonne moteur	PVC	Fraisage CNC	Maintien et positionnement du moteur
Colonnes moteur	HPL	Fraisage CNC	Structure porteuse et guidage des rails
Plaque intermédiaire des douilles	PVC	Fraisage CNC	Support et guidage du chariot
Support inférieur de l'axe Z	HPL	Fraisage CNC	Maintien du porte-stylo
Support moteur de l'axe Z	HPL	Fraisage CNC	Positionnement du moteur Z
Plaque centrale du chariot	HPL	Fraisage CNC	Support principal du système XY
Support de stylo	Plastique	Impression 3D	Maintien du stylo et intégration du ressort

3.2 Procédés de fabrication utilisés

La fabrication des pièces structurelles a principalement reposé sur l'usinage CNC, permettant d'obtenir une bonne précision géométrique et une excellente répétabilité. Les différentes opérations réalisées sont résumées dans le tableau suivant.

TABLEAU 9 – Procédés de fabrication utilisés et descriptions associées

Procédé	Description
Fraisage CNC	Découpe des profils extérieurs des pièces
Perçage	Réalisation des trous de fixation et de guidage
Évidement	Création des logements et chambres d'assemblage
Ébavurage	Suppression des bavures et amélioration des états de surface
Contrôle dimensionnel	Vérification des dimensions critiques avant assemblage
Impression 3D	Fabrication du support de stylo de l'axe Z

3.3 Gamme d'usinage détaillée d'une pièce représentative

La plaque intermédiaire assurant la fixation des douilles est retenue comme pièce représentative du processus de fabrication.

1. Préparation du modèle CAO sous SolidWorks.
2. Export du contour et des perçages au format DXF.
3. Génération des trajectoires d'usinage sous ArtCAM.
4. Positionnement et bridage de la plaque sur la machine CNC.
5. Prise d'origine et réglage des paramètres d'usinage.
6. Réalisation des perçages de fixation.
7. Usinage des évidements et logements fonctionnels.
8. Contournage extérieur de la pièce.
9. Ébavurage manuel et contrôle dimensionnel.

Les principaux équipements utilisés sont :

— Fraiseuse CNC ;
— Logiciels SolidWorks et ArtCAM ;
— Perceuse d'atelier ;
— Pied à coulisse pour le contrôle dimensionnel ;
— Outils d'ébavurage et de finition.

3.4 Chaîne numérique de fabrication

La transition entre la conception et la fabrication suit la chaîne numérique formalisée par la figure 34.

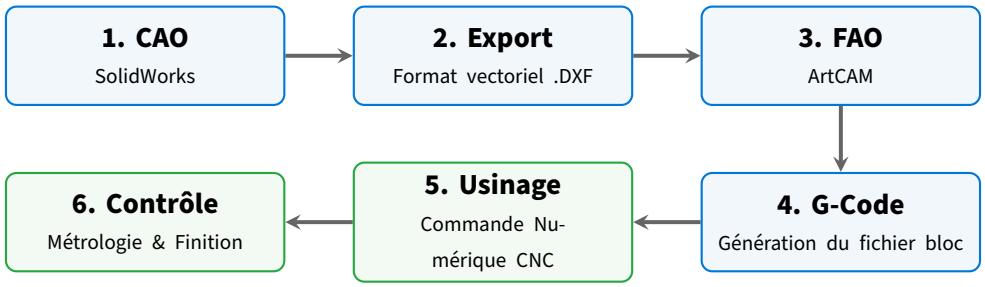


FIGURE 34 – Diagramme de flux de la chaîne numérique de conversion des données géométriques.

3.5 Assemblage mécanique

L'assemblage du système repose sur une combinaison de liaisons par encastrement, guidage linéaire et fixations vissées.

TABLEAU 10 – Solutions d'assemblage utilisées et descriptions fonctionnelles

Type de liaison	Description
Fixations vissées M4	Assemblage principal des supports
Fixations vissées M5	Renforcement de certaines liaisons structurelles
Guidage par douilles	Translation du chariot sur les axes de guidage
Système vis-écrou	Transformation du mouvement pour l'axe Z
Courroie crantée	Transmission du mouvement des moteurs XY

3.6 Montage de l'axe Z

Des améliorations ont été apportées lors du montage final de l'axe Z afin d'augmenter la stabilité du système de maintien du stylo.

- Ajout d'un perçage dans l'anneau interne du support de stylo afin d'améliorer sa rigidité et sa fixation.
- Intégration d'un ressort au-dessus de cet anneau pour maintenir un effort constant sur le stylo.
- Fabrication du support de stylo par impression 3D afin d'obtenir une géométrie adaptée aux contraintes fonctionnelles.

Cette solution permet d'assurer un meilleur contact du stylo avec la surface de travail tout en absorbant les petites irrégularités du support d'écriture.

3.7 Problèmes rencontrés et solutions apportées

Au cours de la fabrication et de l'assemblage, plusieurs difficultés ont été rencontrées.

TABLEAU 11 – Problèmes rencontrés et actions correctives associées

Problème rencontré	Solution apportée
Perçages incomplets ou imprécis	Reprise manuelle et contrôle dimensionnel
Coins arrondis dus au fraisage	Ébavurage et ajustement manuel
Mauvais alignement de certaines pièces	Ajustement géométrique et reprise de montage
Frottement au niveau des douilles	Démontage complet puis réalignement de l'ensemble
Courroie insuffisamment tendue	Retension et refixation sans remplacement de la courroie
Mauvais alignement des perçages du bâti	Reprise des perçages après marquage des positions réelles
Interférences géométriques locales	Ébavurage et ajustement des logements

3.8 Montage final

Après validation de l'ensemble mécanique, les composants électroniques ont été installés sur le bâti :

- Fixation des moteurs pas à pas ;
- Montage des capteurs de fin de course ;
- Installation de la carte de commande ;
- Mise en place de l'alimentation électrique ;
- Raccordement et organisation du câblage.

Une phase finale de vérification a permis de valider le bon fonctionnement du système, le déplacement fluide du chariot et le fonctionnement de l'axe Z.

3.9 Écarts par rapport à la CAO initiale

Certaines modifications ont été introduites au cours de la réalisation afin de prendre en compte les contraintes de fabrication et d'assemblage :

- Ajout d'éléments d'appui pour améliorer la mise en position ;
- Adaptation de certaines géométries aux limites des outils de fraisage ;
- Ajout du ressort dans le système porte-stylo ;
- Modification du support de stylo par impression 3D ;
- Ajustement local de certaines interfaces d'assemblage.

Ces adaptations ont permis d'améliorer la robustesse, la précision et la fiabilité globale du traceur.

4 ALGORITHMIQUE ET LOGIQUE DE COMMANDE

++++++++++[>+++++++>++++++++++>++++++++++>++++++++++>++++++
+++++>++++++++++>++++++++++>++++++++++>++++++++++>++++++++
++++>++++++++++>++++++++++>++++++++++>+++>++++++++++>+++++
+++++++>+++>++++++++>++++++++++>++++++++++>++++++++++>+++++
++++++>++++++++++>++++++++++>+++>++++++++++>++++++++++>+++>
++++++++++>++++++++++>++++++++++>++++++++++>++++++++++>+++
+++++++>++++++++++<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<-]>-----
.>--.>+++.>+.>++++.>-----.>----.>++++.>-.>-----.>+++.>---.>+.
>++.>+.>----.>++.>----.>+.>+++.>-----.>+++.>---.>+.>++.>.>+.>
++.>-.>+.>-..>---.>.>.>+.

4.1 Architecture générale du code embarqué

L'architecture logicielle du système cartésien traceur de plan est conçue comme un système de pilotage en temps réel. **Sa mission est de transformer une consigne textuelle asynchrone** (envoyée par l'utilisateur via flux de données série) en **un train d'impulsions physiques synchronisées** pour les moteurs pas à pas.

Le système doit être capable de :

- gérer les procédures de mise à zéro (homing) pour les axes X, Y et Z,
- gérer les mouvements dans le plan XY et selon l'axe Z,
- gérer la sécurité via les capteurs de fin de course, avec des garde-fous en cas de défaillance des capteurs,
- synchroniser le mouvement des moteurs pour tracer des lignes droites,
- gérer la vitesse de déplacement pour limiter les vibrations et les pertes de pas.

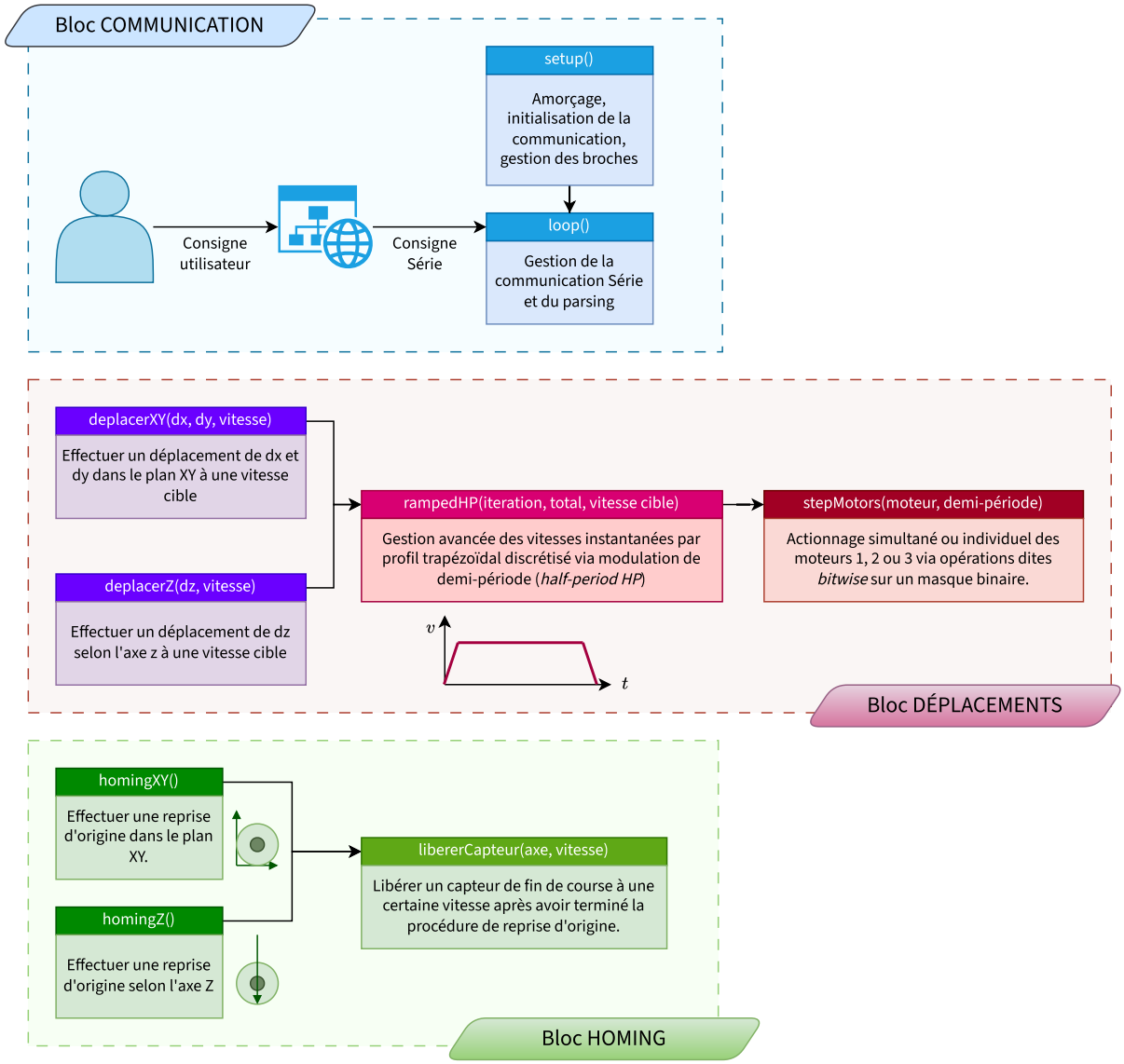


FIGURE 35 – Arborescence des fonctions et procédures du système cartésien traceur de plan

Le système a été conçu pour être modulaire. Chaque composant logiciel est encapsulé dans

un module indépendant (dans la majorité des cas, une fonction ou procédure adéquate), mais qui se basent sur des constantes et fonctions *utilitaires*/globales partagées.

Le système peut être représenté comme pipeline sous forme de diagramme de flux (Fig. 36).

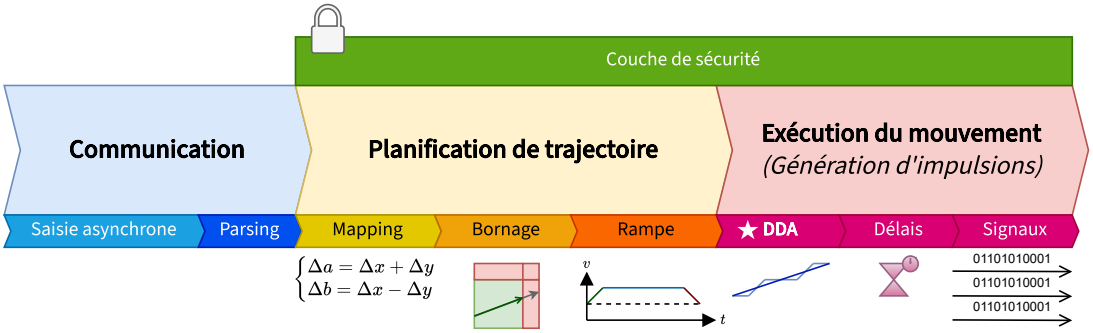


FIGURE 36 – Diagramme de flux du système cartésien traceur de plan

Chaque étage de cette pipeline logicielle remplit un rôle spécifique :

1. **Premier étage – Communication et Parsing :** Cet étage assure l'acquisition des données via liaison série UART à 115220 bauds (pour éviter la *Starvation* et s'assurer que la communication est au moins aussi rapide que l'exécution mécanique), et opère de manière totalement asynchrone par rapport au reste du système.
2. **Deuxième étage – Planification et Sécurité :** Une fois la consigne reçue et validée par le premier étage, le planificateur prépare l'exécution matérielle du mouvement en trois étapes :
   (a) **Mapping cinématique :** À partir des lois cinématiques obtenues lors de l'analyse fonctionnelle, le système convertit les instructions $(x, y), (z)$ données dans l'espace cartésien en instructions "pas" dans le référentiel moteur.

   $$\text{Référentiel Cartésien } (x, y), (z) \rightarrow \text{Référentiel Moteur } (p_1, p_2)$$

   (b) **Bornage dynamique :** Avant toute mise en mouvement, le système vérifie si la cible appartient à l'enveloppe de travail définie en amont dans le code. Toute les coordonnées abérrantes sont tronquées adéquatement pour ne pas dépasser cette limite, et l'utilisateur en est informé.
   (c) **Pré-calcul de la rampe :** Les paramètres du profil trapézoïdal de vitesse (dont l'implémentation est nécessaire pour garantir des tracés corrects) sont calculés ici pour libérer des cycles d'horloge lors de l'exécution matérielle (et garantir un mouvement des moteurs fluide).
3. **Troisième étage – Exécution du mouvement :** L'exécution du mouvement est ensuite réalisée après synchronisation de la vitesse instantanée (et dynamiquement ajustée par profil trapézoïdal) des moteurs par DDA en entiers.

Lors de l'exécution du mouvement, le système **continue à interroger** les capteurs de fins de course et à surveiller la communication Série pour une commande d'arrêt d'urgence logiciel. Les positions spatiales sont –évidemment– immédiatemment recalculées pour éviter tout décalage du référentiel lors de la reprise du mouvement.

4.2 Analyse du premier étage — Communication et Parsing

Le premier étage du pipeline logiciel repose sur la boucle principale (`loop()`). Son fonctionnement s'articule autour d'un mécanisme de scrutation (*polling*) non bloquant du port série, garantissant que le processeur reste disponible pour la gestion des interruptions et la synchronisation matérielle.

Cet étage remplit trois fonctions critiques : la sécurité (arrêt d'urgence), le routage des macros simples, et l'analyse syntaxique (*parsing*) des consignes de déplacement complexes. Il est représenté visuellement sous forme d'algorigramme dans la figure 37 ci-dessous.

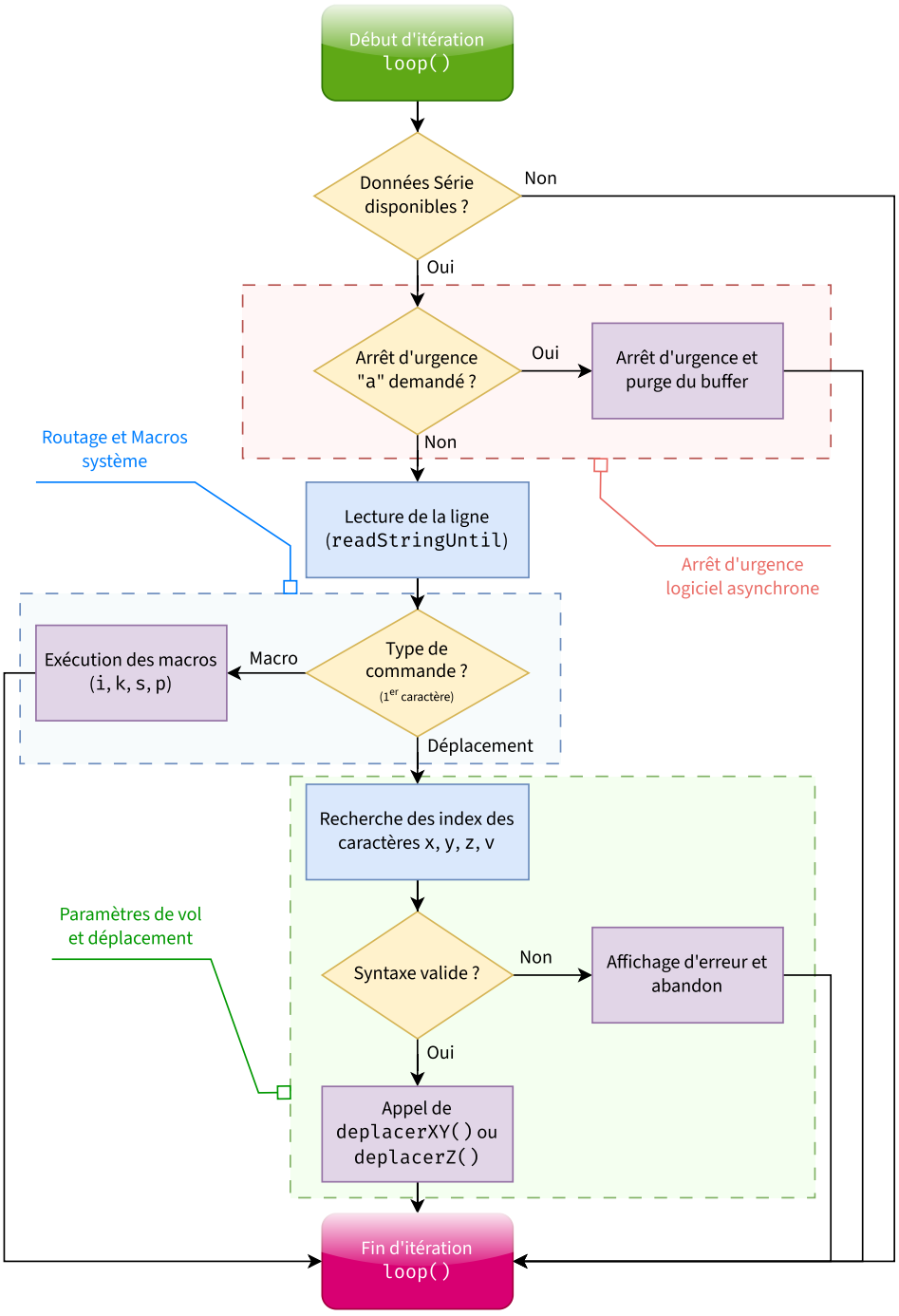


FIGURE 37 – Algorigramme du premier étage de communication et d'analyse syntaxique.

Nous pouvons détailler le fonctionnement de chacune des trois fonctions critiques remplies par cet étage comme suit :

1. Arrêt d'Urgence logiciel asynchrone Pour éviter qu'une commande d'urgence ne soit mise en attente derrière une longue chaîne de caractères non entièrement reçue (ce qui nécessiterait l'appel bloquant à `readStringUntil()`), le code utilise la méthode `Serial.peek()`. Cette fonction permet d'inspecter le premier octet du buffer d'entrée sans le consommer.

```
if (Serial.peek() == 'a' || Serial.peek() == 'A') {
    Serial.read(); // Consomme le caractère d'urgence
    Serial.println(F("!!! ARRET !!!"));
    afficherPosition();
    while(Serial.available()) Serial.read(); // Purge le reste du buffer
    return;
}
```

Listing 1 – Mécanisme d'arrêt d'urgence par *peeking*.

Si une demande d'arrêt d'urgence est détectée, le système purge instantanément le buffer pour empêcher l'exécution d'anciennes commandes.

2. Routage et Macros système Si aucune urgence n'est détectée, la ligne est lue puis rognées (`trim()`). Le premier caractère est isolé et converti en minuscule pour définir le routage/redirection vers une procédure dite "macro" prédéfinie. Ce mécanisme permet de lancer des sous-programmes complets à l'aide d'une instruction à caractère unique. Par exemple, la commande `i` déclenche la procédure de reprise d'origine (*homing*) des axes X, Y.

3. Extraction des paramètres de vol (*Parsing*) Pour les déplacements géométriques (ex : `x50y-20v60`), la trame textuelle doit être convertie en grandeurs physiques flottantes. Plutôt que d'imposer une longueur fixe ou une structure rigide à l'utilisateur, l'algorithme repère la position (l'index) des lettres clés (x, y, z, v) via la méthode `indexOf()`.

```
int xi = inp.indexOf('x');
int yi = inp.indexOf('y');
int vi = inp.indexOf('v');

// Validation de la syntaxe
if (vi == -1 || (xi == -1 && yi == -1)) {
    Serial.println(F("Ex: x50v60 | y-30v40 | x50y50v60"));
    sendErr();
    return;
}

// Extraction des sous-chaines (substring) et conversion (toFloat)
float dx = 0, dy = 0;
if (xi != -1) dx = inp.substring(xi + 1, (yi != -1 && yi > xi) ? yi : vi).
    ↪ toFloat();
if (yi != -1) dy = inp.substring(yi + 1, vi).toFloat();
float rpm = inp.substring(vi + 1).toFloat();
```

Listing 2 – Logique de découpage (parsing) des coordonnées et de la vitesse.



La procédure `sendErr()` est une simple macro servant à affichier ERR (via `Serial.println(F("ERR"));`) dans le cadre du protocole Handshake strict entre l'interface graphique –dont la conception est détaillée dans la section suivante– et le code embarqué.

Cette méthode d'extraction dynamique permet une grande flexibilité d'utilisation. Par exemple, l'instruction ternaire `(yi != -1 && yi > xi) ? yi : vi` gère intelligemment la fin de la sous-chaîne x : elle s'arrête au caractère y si celui-ci est présent après x, ou directement au caractère v (vitesse) dans le cas d'un mouvement purement horizontal. Une fois les valeurs physiques (millimètres et tours par minute) validées, elles sont transférées au deuxième étage pour le calcul de la rampe dynamique et du *mapping* cinématique.

4.3 Analyse du deuxième étage — Planification et sécurité



Cette sous-section sera principalement axée sur l'analyse de la procédure `deplacerXY()`. Une analyse (moins complète) de la procédure `deplacerZ()` est aussi présente en fin de sous-section.

4.3.1 Mapping cinématique

L'exécution du mouvement se fait directement sur les incréments **moteur** (plutôt que les incréments cartésiens Δx et Δy). Cette conversion est requise pour corriger deux failles inhérentes à l'interpolation cartésienne :

- **Conservation de la résolution spatiale :** Dans l'espace cartésien, exiger que X et Y soient des entiers impose mathématiquement que la somme $A + B = 2X$ soit paire. Par conséquent, A et B doivent strictement partager la même parité, ce qui rend 50% des positions physiques inatteignables. L'application de l'algorithme dans le référentiel moteur permet d'exploiter la totalité de la grille matérielle, doublant ainsi la précision mécanique effective (Fig. 38).
- **Suppression du Jitter temporel :** Une interpolation cartésienne produit une répartition temporelle asynchrone des pas moteurs (arythmie), engendrant des accélérations irrégulières. Le calcul direct dans l'espace moteur garantit une distribution isochrone des impulsions, éliminant les sources de vibrations, de résonances structurelles et de perte de précision dynamique.

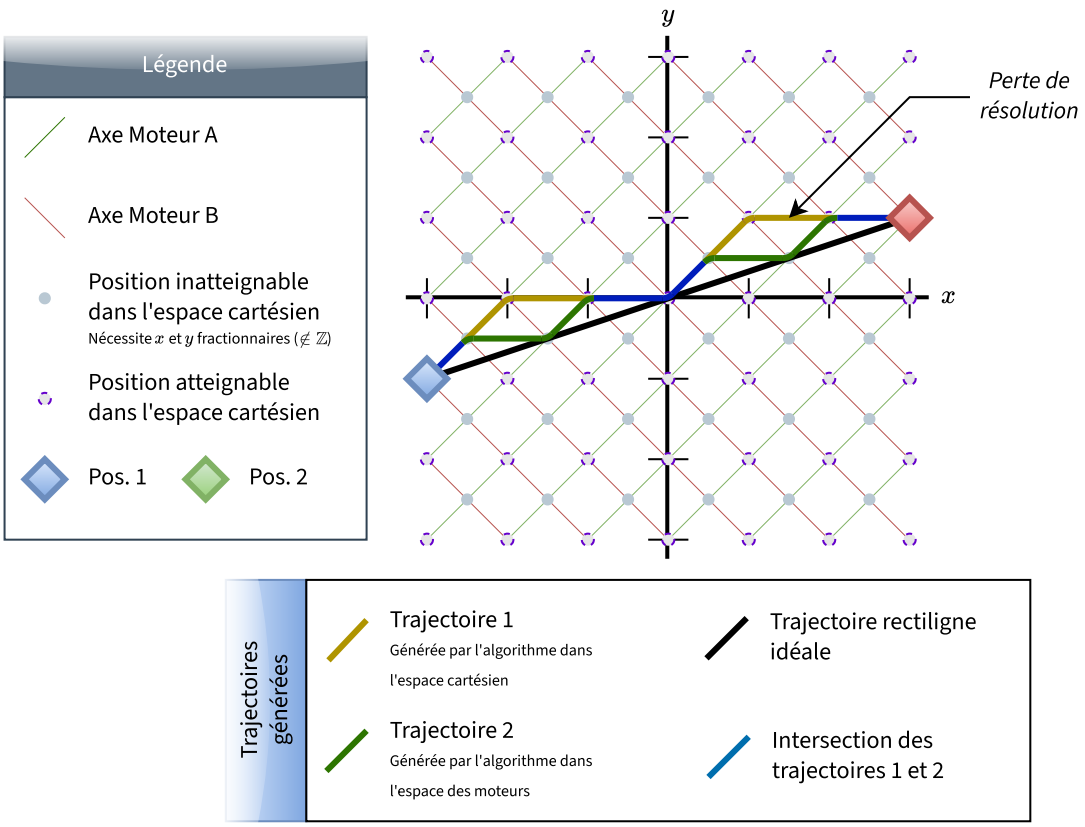


FIGURE 38 – Comparaison globale des trajectoires rectilignes générées via l'interpolation cartésienne et l'interpolation moteur.

Les lois cinématiques établies précédemment ont été réécrites différemment dans le code pour prendre en compte des modifications dans le système réel sans changer profondément la nature et le fonctionnement du système (via redéfinition des variables, entre autres...). Il aurait aussi été possible de les laisser inchangées en redéfinissant arbitrairement le "moteur 1" et "moteur 2" dans le code pour qu'ils correspondent exactement à ceux dans le système mécanique.

$$\begin{cases} \Delta\texttt{pasM1} = \texttt{mmToSteps}(\Delta\texttt{y}) - \texttt{mmToSteps}(\Delta\texttt{x}) \\ \Delta\texttt{pasM2} = -\texttt{mmToSteps}(\Delta\texttt{y}) - \texttt{mmToSteps}(\Delta\texttt{x}) \end{cases} \tag{45}$$

L'extrait du code source de la procédure `deplacerXY()` 3 détaille comment est réalisé la conversion *coordonnées cartésiennes* $\leftrightarrow$ *Pas moteurs*.

4.3.2 Bornage dynamique

Avant l'exécution du mouvement, le système déploie deux couches de sécurité pour prévenir toute collision mécanique : une couche "mathématique" préventive, et une couche matérielle active.

TABLEAU 12 – Architecture fonctionnelle des deux couches de surveillance du traceur

Couche de Sécurité	Logique de Commande et Implémentation
Bornage mathématique *(Préventif)*	Les coordonnées cibles sont systématiquement évaluées par rapport à l'enveloppe de travail maximale du traceur. L'algorithme s'appuie sur la fonction `constrain()` pour tronquer mathématiquement toute trajectoire théorique qui s'aventurerait en dehors de la grille autorisée $[0, \texttt{MAX_STEPS}]^2$, où `MAX_STEPS` est une constante définie en amont.
Surveillance matérielle *(Active)*	Avant chaque impulsion moteur, l'état matériel des capteurs de fin de course (`LIMIT1`, `LIMIT2`) est lu et on scrute le buffer série à la recherche d'une commande d'arrêt d'urgence. En cas d'interruption, le mouvement est avorté instantanément. La position spatiale exacte au moment de l'impact est recalculée et sauvegardée.

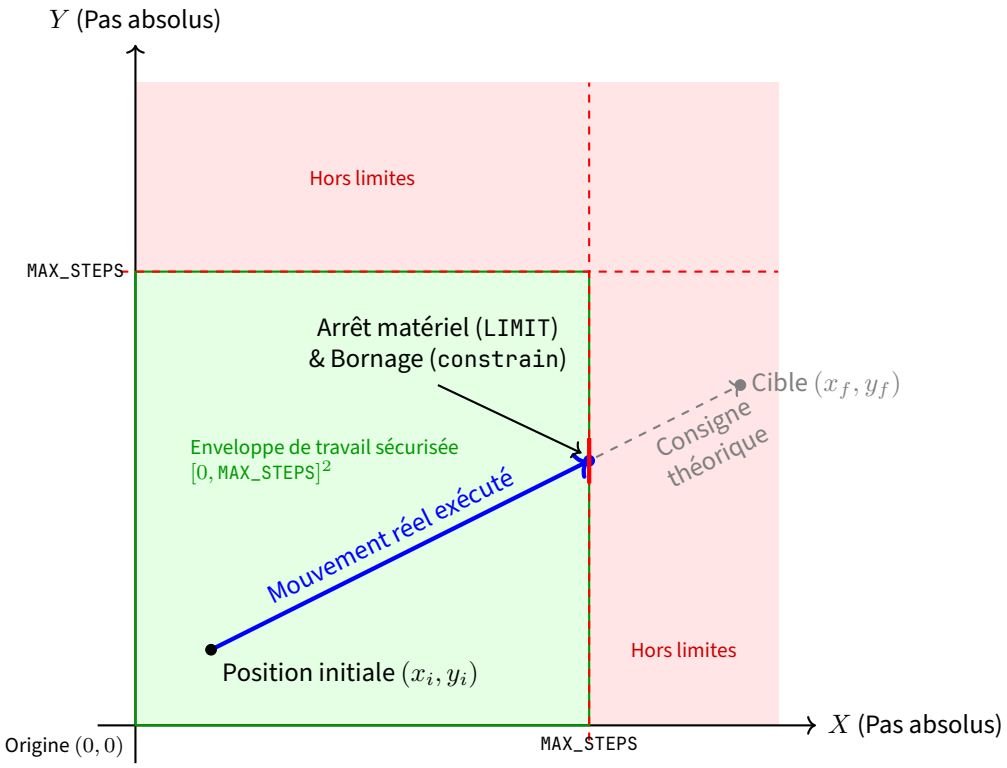


FIGURE 39 – Principe de sécurisation spatiale sur le plan XY. Toute trajectoire traversant les frontières de l'enveloppe déclenche une interruption matérielle immédiate, couplée à un recalcul borné des coordonnées absolues pour préserver le référentiel.

```
// Bornage dynamique, détaillé dans la sous-sous-section suivante.

long clampX = constrain(stepsX+mmToSteps(dx), 0L, MAX_STEPS);
long clampY = constrain(stepsY+mmToSteps(dy), 0L, MAX_STEPS);

```

```
// stepsX et stepsY font référence à la position actuelle du chariot.
// clampX et clampY représentent les nouvelles positions en pas (bornées).
// realDX et realDY représentent les incréments convertis en pas.

long realDX = clampX-stepsX,  realDY = clampY-stepsY;

// Bornage dynamique (2).

if (realDX==0 && realDY==0) { Serial.println(F("Deja en limite.")); afficherPosition(); return; }
if (clampX != stepsX+mmToSteps(dx)) { Serial.print(F("Limite X: ")); Serial.print(clampX*MM_PER_STEP,1); Serial.println(F("mm")); }
if (clampY != stepsY+mmToSteps(dy)) { Serial.print(F("Limite Y: ")); Serial.print(clampY*MM_PER_STEP,1); Serial.println(F("mm")); }

// Conversion en pas moteur via les lois cinématiques définies ci-dessus.

long pasM1=-realDX+realDY,  pasM2=-realDX-realDY;

// Initialisation d'une variable de direction pour ne prendre en compte que les pas absolus.

bool dir1=(pasM1>=0),      dir2=(pasM2>=0);
digitalWrite(CW1, dir1?HIGH:LOW);
digitalWrite(CW2, dir2?HIGH:LOW);

// Définition d'une variable de pas absolus pour chaque moteur.
// La variable "total" sera utilisée lors de la synchronisation des moteurs via DDA.

long absM1=abs(pasM1), absM2=abs(pasM2), total=max(absM1,absM2);
```

Listing 3 – Conversion des instructions x, y en pas moteurs après bornage.

4.3.3 Pré-calcul de la rampe du profil trapézoïdal de vitesse

Pour des raisons qui sont détaillées et expliquées de manière *très exhaustive* dans la section 6.3 (et plus globalement, sa section mère), l'implémentation d'un profil trapézoïdal de vitesse s'est rapidement imposée pour garantir un mouvement fluide et un tracé précis.

Afin de pallier l'absence de coprocesseur arithmétique (FPU) sur le microcontrôleur 8 bits, le calcul continu de la rampe est entièrement réalisé en arithmétique entière. Les paramètres cinématiques (nombre de pas d'accélération, bornes temporelles) sont **précalculés en amont** de la commande de déplacement.

Durant l'exécution du tracé, la boucle d'asservissement (pilotée par l'algorithme DDA détaillé à la section 4.4) se concentre exclusivement sur l'interpolation temporelle. Elle évalue le délai d'impulsion (`currentHP`) en microsecondes via une arithmétique entière stricte. Ce délai dynamique est ensuite appliqué directement aux déclenchements matériels générés par le DDA, lissant la dynamique du chariot sans bloquer le processeur.

La Figure 40 illustre la consigne cinématique générée par cette logique d'interpolation temporelle en fonction de la progression du mouvement.

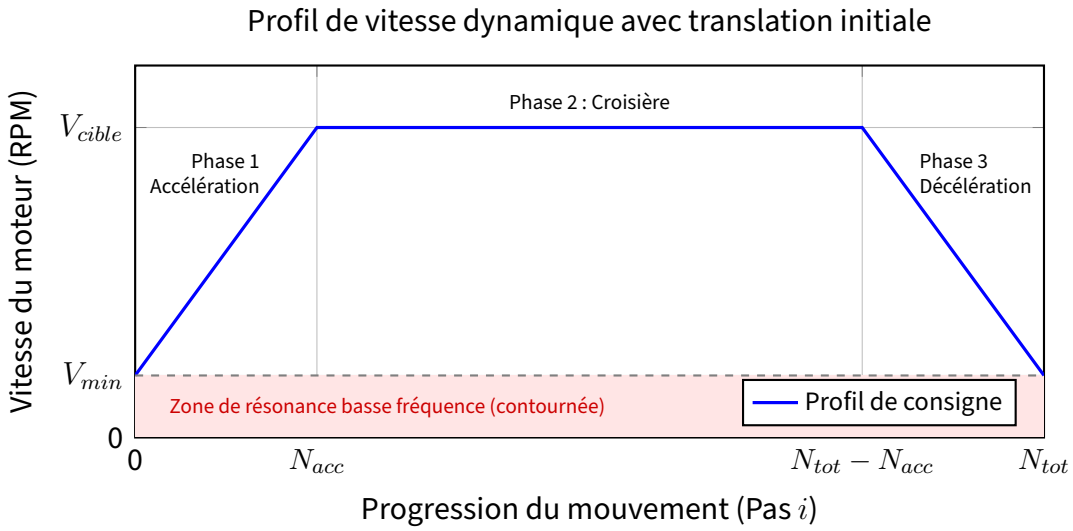


FIGURE 40 – Profil cinématique généré par la fonction `rampedHP`. Le démarrage à V_{min} évite les pertes de pas liées au couple de détente du moteur.

Discrétisation temporelle du profil trapézoïdal L'implémentation algorithmique du profil de vitesse repose sur la modulation dynamique du délai d'attente entre chaque front montant envoyé au driver. La variation de la vitesse est accomplie par une interpolation linéaire de la demi-période (`currentHP`) du signal d'horloge. Cette approche permet de s'affranchir des calculs en virgule flottante au sein de la boucle principale, maximisant la fréquence d'exécution grâce à l'arithmétique entière.

La durée de la demi-période $HP(i)$ pour l'itération i est régie par les équations aux différences suivantes :

— **Phase d'accélération** ($i < N_{ramp}$) :

$$HP(i) = HP_{max} - \left(\frac{HP_{max} - HP_{min}}{N_{ramp}} \right) \cdot i \tag{46}$$

— **Phase de décélération** ($i \geq N_{total} - N_{ramp}$) :

$$HP(i) = HP_{min} + \left(\frac{HP_{max} - HP_{min}}{N_{ramp}} \right) \cdot (i - (N_{total} - N_{ramp})) \tag{47}$$

Entre ces deux phases, la consigne est maintenue constante à HP_{min}.

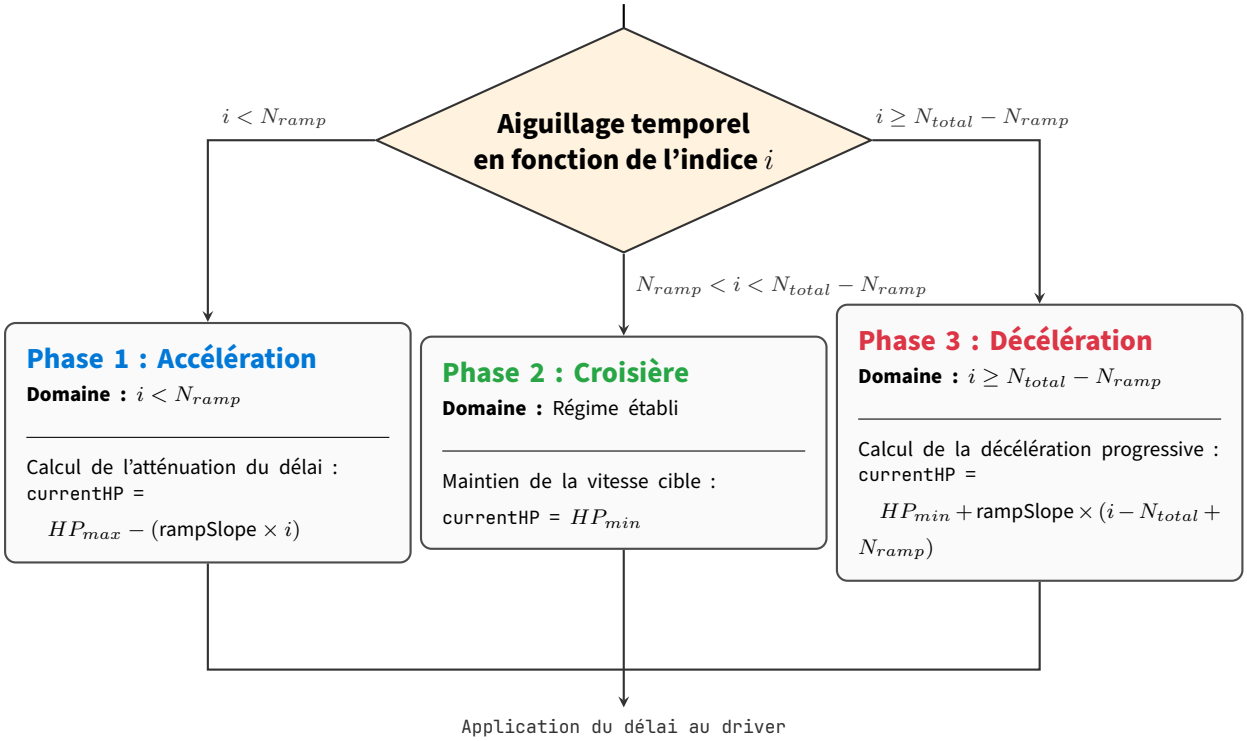


FIGURE 41 – Extrait de l'algorigramme d'aiguillage cinématique au sein de la boucle d'asservissement temporelle DDA.

La figure 42 illustre la traduction de cette loi de commande en signaux physiques. La modulation temporelle de la période ($2 \times HP$) génère une variation de la fréquence des pas, entraînant l'accélération mécanique du rotor.

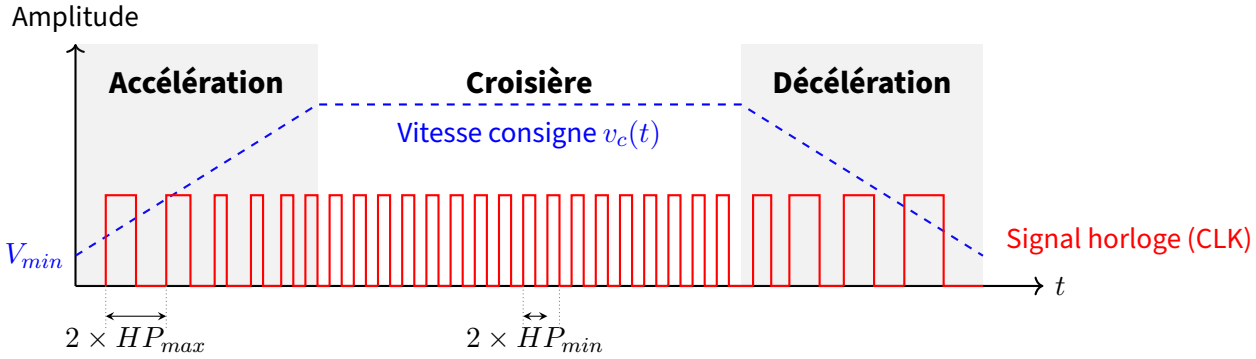


FIGURE 42 – Modulation de la largeur d'impulsion matérielle traduisant la consigne trapézoïdale de vitesse. L'échelle des impulsions représentées est exagérée et ne correspond pas exactement à celles envoyées au moteur en pratique (à but illustratif uniquement).

4.4 Analyse du troisième étage — Exécution du mouvement (DDA)

L'interpolation linéaire du mouvement, indispensable pour tracer des trajectoires rectilignes, est assurée par un Analyseur Différentiel Numérique (DDA) en calculs entiers (*Integer DDA*).

Principe de l'interpolation DDA L'algorithme identifie le déplacement dominant pour établir une base de temps matérielle synchronisée. La variable `total` (`total` $= \max(|\Delta$`pasM1`$|, |\Delta$`pasM2`$|)$) détermine le nombre d'itérations exact de la boucle de tracé. Cette approche garantit que l'axe exigeant le plus grand nombre de pas avance à chaque cycle. L'axe secondaire est asservi **proportionnellement à ce dénominateur commun**. Le DDA est implémenté en **arithmétique entière stricte**.

Deux accumulateurs (`eM1` et `eM2`) sont initialisés au centre du pas (`total/2`) pour garantir un arrondi symétrique. À chaque itération de la boucle de base de temps, le besoin en déplacement de chaque axe est ajouté à son accumulateur. Dès que cet accumulateur déborde de la valeur `total`, un pas matériel est déclenché et l'accumulateur est retranché.

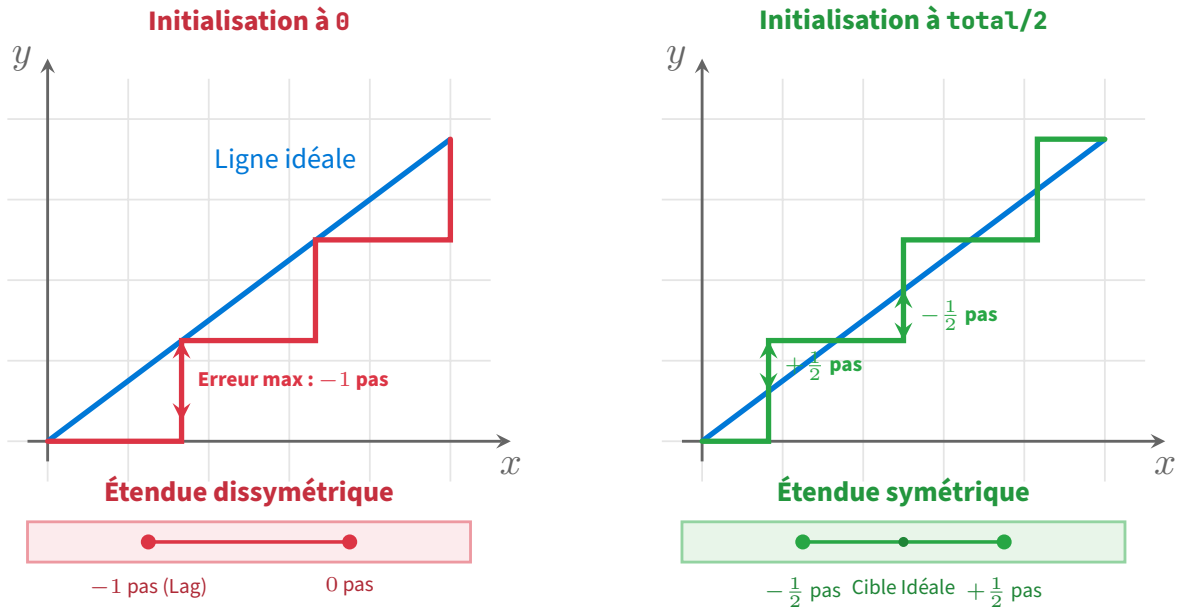


FIGURE 43 – Analyse géométrique de l'erreur de quantification spatiale du traceur. L'injection de la constante `total/2` convertit l'opération de troncature en un arrondi au plus proche, centrant l'erreur sur $[-\frac{1}{2}, +\frac{1}{2}]$.

Analogie du réservoir d'accumulation Le principe de la DDA s'apparente à un système de dosage volumétrique. Chaque moteur possède un intégrateur discret (les variables `eM1` et `eM2`) dont la capacité maximale correspond au déplacement dominant (`total`).

À chaque cycle de calcul (itération), on verse dans ces intégrateurs une « dose » proportionnelle au déplacement total requis par l'axe (`absM1` pour le moteur A, `absM2` pour le B). Dès qu'un intégrateur atteint sa pleine capacité, il déborde : un pas physique est déclenché, et l'intégrateur est vidé de la valeur `total`, conservant l'éventuel reste pour le cycle suivant.

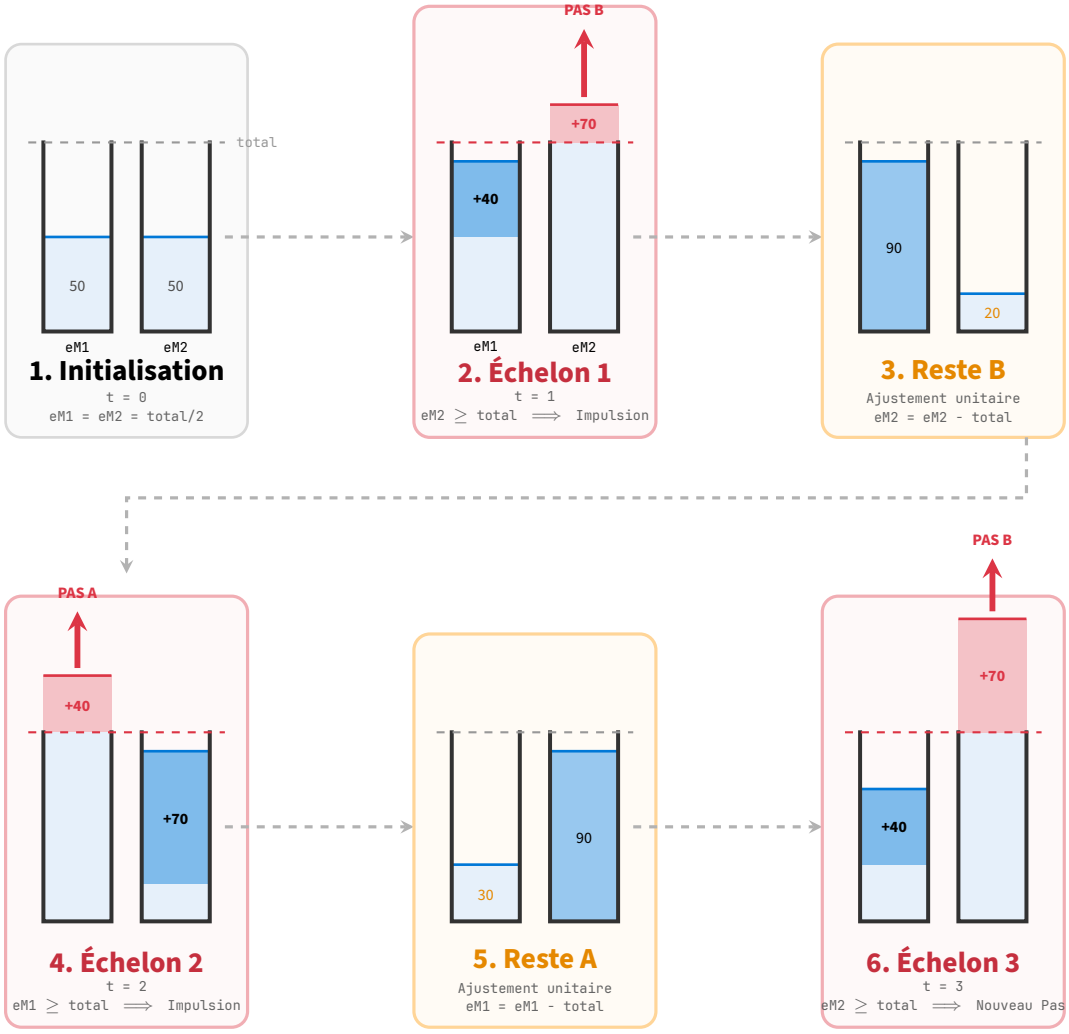


FIGURE 44 – Chronogramme dualisé de l'Analyseur Différentiel Numérique (DDA) sur douze seaux d'intégration fractionnelle de capacité arbitraire $= 100$. L'évaluation simultanée de `eM1` et `eM2` met en évidence la **distribution asynchrone** des impulsions physiques envoyées aux drivers moteurs.

À chaque itération de la boucle de base de temps, le déplacement requis par chaque axe est ajouté à son accumulateur. Dès que cet accumulateur déborde de la valeur `total`, un pas matériel est déclenché via le masque binaire (`mask`) et l'accumulateur est retranché.

```
byte mask = 0;
// Accumulation d'erreur proportionnelle pour l'axe A
eM1 += absM1;
if (eM1 >= total) {
    mask |= 0x01;  // Armement du pas moteur A
    eM1 -= total;  // Réinitialisation relative
}
// Idem pour l'axe B
eM2 += absM2;
if (eM2 >= total) {
    mask |= 0x02;
    eM2 -= total;
}
```

Note d'optimisation : Le type `byte` (8 bits) est employé pour le masque afin de garantir l'exécution des opérations bit à bit en un seul cycle d'horloge sur l'architecture AVR (Advanced Virtual RISC, Harvard modifié) 8 bits, réduisant au maximum la latence.

4.5 Informations additionnelles

Fonctions et procédures intermédiaires utilitaires

Notre solution implémente une architecture logicielle **extrêmement modulaire** structurée en cinq couches distinctes, allant de l'abstraction matérielle basse niveau (HAL) jusqu'au moteur d'exécution cinématique. Cette organisation garantit une séparation stricte entre le traitement arithmétique des profils de vitesse, l'interpolation spatiale et la gestion des flux d'interruption série. L'intégralité des fonctions et des routines de services associées est récapitulée dans les tableaux 13, 14, 15, 16 et 17.

TABLEAU 13 – Fonctions de communication et télémétrie (Interface Série).

Nom de la Fonction	Rôle et Description Logique
`sendOk()`	Émet le signal d'acquittement synchrone `OK` requis par le protocole de handshake pour valider la bonne réception de la commande.
`sendErr()`	Notifie l'IHM distante d'une erreur de syntaxe ou d'un échec d'évaluation lors du parsing.
`sendLimit(axis)`	Déclenche une alerte textuelle critique enrichie vers l'IHM lors du blocage ou de l'activation d'un capteur de fin de course.
`afficherPosition()`	Calcule et retourne en temps réel la télémétrie absolue de la machine (coordonnées X, Y, Z converties en mm) ainsi que les limites physiques de l'enveloppe de travail.

TABLEAU 14 – Utilitaires et conversions mathématiques (Helpers).

Nom de la Fonction	Rôle et Description Logique
`recalculerDerives()`	Actualise de manière globale les facteurs d'échelle cinématiques (`MM_PER_STEP`, `MM_PER_STEP_Z`) et met à jour la borne discrète `MAX_STEPS`.
`mmToSteps(mm)`	Convertit une coordonnée métrique linéaire continue (X ou Y) en un nombre entier discret d'impulsions (pas machine).
`mmToStepsZ(mm)`	Convertit une coordonnée métrique linéaire continue dédiée à l'axe vertical Z en pas machine à partir de la constante révisée `MM_PER_STEP_Z`.
`rpmToHP(rpm)`	Convertit une vitesse de rotation cinématique en une période d'horloge discrète (demi-période `hp` en μs) exploitable par les timers.
`rampedHP(i, total, rpm)`	Modélise mathématiquement le profil trapézoïdal de l'axe vertical Z en évaluant l'indice i par rapport aux rampes d'accélération et de décélération.

TABLEAU 15 – Couche d'abstraction matérielle et sécurité (HAL).

Nom de la Fonction	Rôle et Description Logique
`stepMotors(mask, hp)`	Commande l'état physique des broches d'horloge (`CLK1`, `CLK2`, `CLK3`) par masquage binaire direct (Bit 0 : M1, Bit 1 : M2, Bit 2 : Z) pour assurer une commutation synchrone des axes.
`libererCapteur(axe, hp)`	Algorithme de dégagement de sécurité : force l'axe spécifié (1, 2 ou 3) à reculer pour ouvrir le contact logique d'une butée matérielle avant d'autoriser un mouvement.

TABLEAU 16 – Moteur d'exécution cinématique (Core).

Nom de la Fonction	Rôle et Description Logique
`homing()`	Exécute le calage d'origine séquentiel et cartésien des axes planaires (axe X puis axe Y) en coordonnant les moteurs de transmission du traceur.
`homingZ()`	Réalise de manière autonome la prise de référence de l'axe vertical en gérant la montée de l'effecteur jusqu'à la commutation du capteur `LIMIT3`.
`deplacerXY(dx, dy, rpm)`	Cœur du micrologiciel. Cadence l'interpolation linéaire DDA b axe et génère *inline* par arithmétique entière stricte les rampes du profil de vitesse trapézoïdal (`rampSlope`) tout en surveillant les interruptions série et de fin de course.
`deplacerZ(dz, rpm)`	Pilote le mouvement vertical de l'outil d'écriture sur l'axe Z en exploitant la fonction de lissage `rampedHP` pour éliminer les vibrations de la structure.

TABLEAU 17 – Cycle de vie et configuration.

Nom de la Fonction	Rôle et Description Logique
`editerParametres()`	Menu de configuration interactif sur l'UART permettant la mise à jour dynamique des registres mécaniques et d'asservissement sans nécessiter de recompilation.
`setup()`	Routine d'amorçage : configure la direction des entrées/sorties et précalcule les variables d'échelle.
`loop()`	Scrutateur asynchrone principal : traite le buffer d'entrée de la liaison série UART (115200 bauds), isole les jetons syntaxiques et oriente le flux d'exécution vers le Core cinématique.

Constante / Variable	Valeur	Unité	Rôle et Justification Technique
⚙ Configuration Mécanique et Résolution			
`STEPS_PER_REV`	1600	pas/tr	Résolution configurée en pas fractionnaires (microstepping 1/8) pour lisser le mouvement et supprimer les résonances mécaniques.
`MM_PER_REV`	40	mm/tr	Avance linéaire par révolution sur le plan planaire (Poulie GT2 20 dents, pas de 2 mm pour les axes X et Y).
`MM_PER_REV_Z`	8	mm/tr	Avance linéaire par révolution dédiée à la vis de guidage hélicoïdale de l'axe vertical Z.
⛔ Limites de l'Espace de Travail			
`MAX_TRAVEL_MM`	300,0	mm	Enveloppe limite de sécurité et course utile maximale sur les axes X et Y. Inférieure à 400 (définie par le CDC) pour prendre en compte la taille du chariot.
`MAX_TRAVEL_Z_MM`	100,0	mm	Borne logicielle de garde pour la distance de translation verticale de l'axe Z.
`MAX_STEPS_Z`	20000,0	pas	Course maximale de sécurité discrète autorisée pour le **homing** de l'outil selon l'axe Z.
🚀 Dynamique et Profil de Vitesse			
`MIN_RPM`	1,0	tr/min	Vitesse plancher absolue. Impérativement non nulle afin de prévenir toute exception de division par zéro lors du calcul d'inversion des délais dans l'algorithme du profil trapézoïdal.
`MAX_RPM`	50,0	tr/min	Plafond cinématique nominal configuré pour préserver le couple dynamique et éliminer l'overshoot lors des phases de décélération.
`ACCEL_RPM_PER_STEP`	0,2	tr/min/pas	Pente d'accélération et de décélération linéaire appliquée aux rampes temporelles.
🛡 Sécurité et Procédures de Homing			
`HOMING_HALF_PERIOD`	200	μs	Demi-période des impulsions d'horloge fixant la vitesse d'approche initiale vers les capteurs de fin de course.
`RELEASE_STEPS`	2000	pas	Course de dégagement requise pour libérer mécaniquement les contacteurs et basculer de manière fiable leur état logique.
`VERBOSE`	`false`	—	Invalidation des flux de débogage série pour éliminer la latence d'interruption et garantir le déterminisme temporel de la boucle DDA.

TABLEAU 18 – Récapitulatif des paramètres de configuration matériels et des constantes cinématiques d'asservissement.

4.6 Gestion du stylet (Axe Z)

L'axe Z, via la procédure `deplacerZ()`, commande la levée et la descente de l'effecteur (dans notre cas, le stylet). Contrairement au plan XY régi par l'interpolation DDA, l'axe Z opère de manière totalement indépendante, unidimensionnelle et séquentielle. Son exécution suspend tout mouvement dans le plan pour garantir que le chariot reste strictement immobile lors de la procédure de remise à zéro ou d'un quelconque mouvement du stylet.

L'algorithme de commande (`deplacerZ()`) est structurellement plus simple, car il ne nécessite pas de synchronisation multi-axes. Néanmoins, il intègre une couche de sécurité préventive et active absolue pour éviter toute collision mécanique ou casse de l'outil, conceptuellement identiques à celles de la procédure `deplacerXY()`.

La loi de commande utilisée pour contrôler l'axe Z est déduite simplement des propriétés de la solution vis-écrou utilisée. Une rotation d'un tour du moteur de cet axe induit un déplacement de 8 mm ; on en déduit facilement la résolution en mm/pas.

```
MM_PER_STEP_Z = MM_PER_REV_Z / (float)STEPS_PER_REV;
```

4.7 Procédures de Homing/Remise à zéro

La réinitialisation géométrique du système est indispensable pour synchroniser les coordonnées logiques du micrologiciel avec la position physique réelle du chariot. Cette opération s'exécute de manière séquentielle en boucle ouverte jusqu'à l'interruption par les capteurs de fin de course mécaniques.

4.7.1 Homing planaire

La cinématique impose un mouvement combiné des deux moteurs planaires pour isoler un déplacement orthogonal pur vers les boutons-poussoirs de référence. La procédure `homing()` s'exécute en deux étapes indépendantes :

- **Initialisation de l'axe X :**
  - Une routine de dégagement préventive (`libererCapteur`) recule le chariot si le contacteur est déjà enfoncé.
  - Les deux moteurs tournent dans le même sens (`CW1=HIGH`, `CW2=HIGH`) pour déplacer l'effecteur uniquement selon l'axe X.
  - Le mouvement s'arrête dès que la broche `LIMIT1` bascule à l'état `LOW`.
  - Une seconde phase de dégagement dégage lentement la butée pour valider précisément la transition logique.
- **Initialisation de l'axe Y :**
  - Le micrologiciel applique une inversion de sens sur le premier moteur (`CW1=LOW`, `CW2=HIGH`) pour contraindre le déplacement suivant selon l'axe Y.
  - Le chariot progresse jusqu'à l'impact avec le second capteur mécanique (`LIMIT2`).
  - Après un ultime recul de sécurité via la fonction de libération, les compteurs logiciels de position de la couche DDA (`stepsX` et `stepsY`) sont forcés à zéro, établissant l'origine absolue $(0, 0)$.

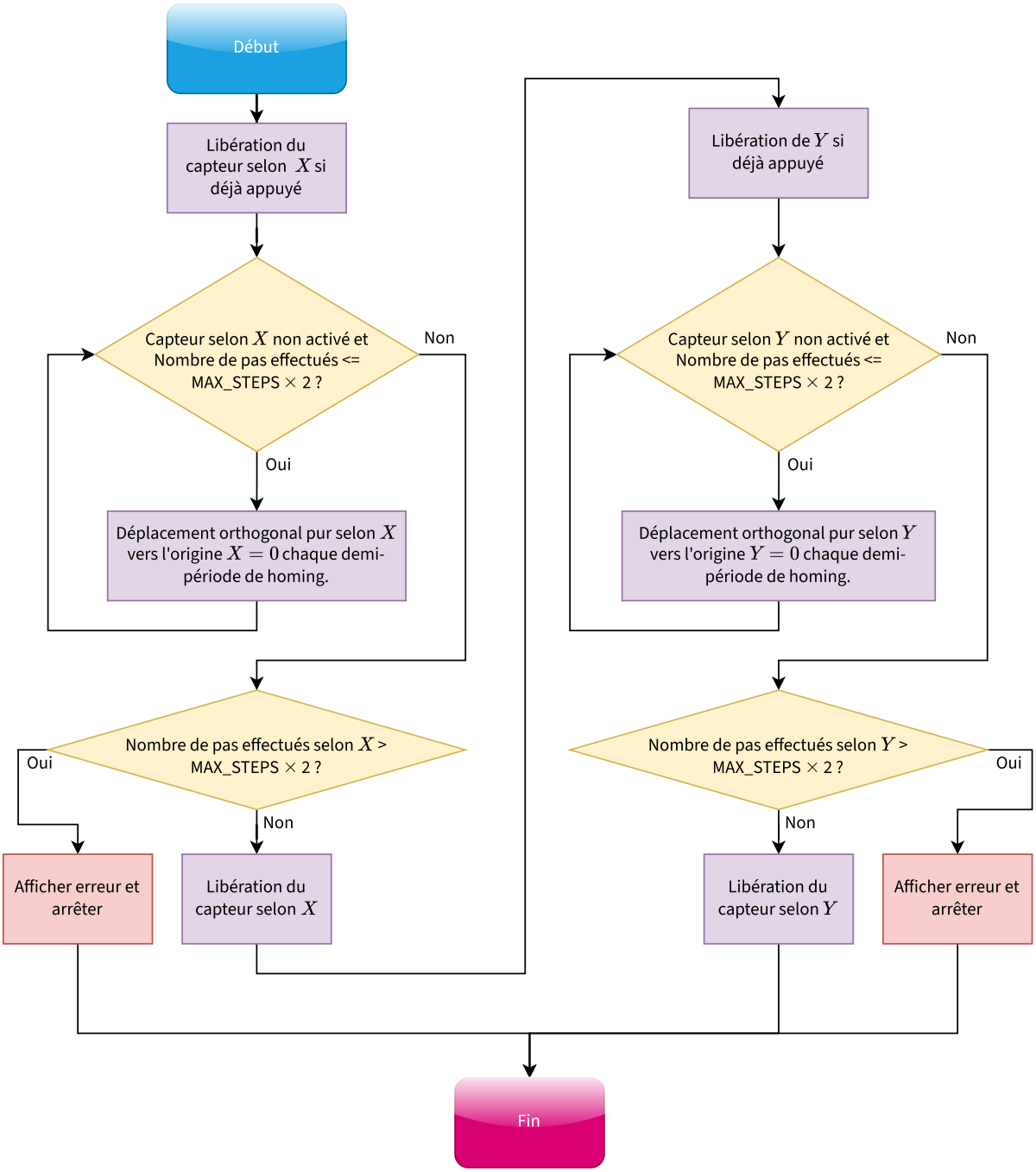


FIGURE 45 – Algorigramme de fonctionnement de la procédure de Homing planaire

4.7.2 Homing selon Z

L'axe vertical est traité, via `homingZ()`, de façon totalement indépendante des axes planaires pour éviter toute collision de l'outil avec la surface de travail lors de la phase de recalage.

— **Sécurisation initiale :** Le système vérifie l'état de la broche `LIMIT3`. Si l'outil est déjà en position haute, une commande de descente forcée libère le contacteur.
— **Séquence de montée :** Le driver reçoit une consigne de direction constante (`CW3=LOW`) associée à des impulsions cadencées par la constante temporelle `HOMING_HALF_PERIOD`

($200\ \mu$s). L'arbre du moteur entraîne la vis hélicoïdale pour faire monter l'ensemble du bloc porte-stylet.

— **Définition de la référence d'appui :** Dès que l'impact mécanique sur `LIMIT3` est détecté, la boucle s'interrompt. La fonction `libererCapteur` applique une série de $2\,000$ pas en sens inverse (`CW3=HIGH`) pour relâcher la pression sur le ressort de complaisance. La variable de position `stepsZ` est alors réinitialisée à 0.

4.8 Intégration à l'UI et Handshake

Pour garantir l'intégrité du flux de données, la communication IHM-Microcontrôleur s'appuie sur un protocole de type *Question-Réponse* (**Handshake** logiciel).

L'interface de pilotage externe (i.e. l'interface graphique) doit impérativement scruter et respecter les signaux de contrôle émis par le micrologiciel :

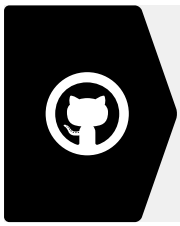
— **Acquittement Nominal (`OK`) :** Émis systématiquement à la fin d'une procédure système réussie. Ce signal déverrouille l'envoi de la trame suivante par l'IHM.

— **Rejet Syntaxique (`ERR`) :** Déclenché par la couche d'analyse logicielle si la trame est malformée, incomplète, ou si les paramètres cinématiques sont aberrants (vitesse négative, déplacement nul). La consigne est purgée sans déclenchement mécanique.

— **Interruption Matérielle (`LIMIT [Axe]`) :** Transmis de manière asynchrone si un capteur de fin de course est activé lors de l'exécution du mouvement. Ce signal indique une collision imminente.

— **Télémétrie (`>> X=...`) :** Émise soit à l'arrêt, sur requête manuelle (s $\equiv$ "statut") ou lors d'un avortement de mouvement d'urgence (a $\equiv$ "arrêt" d'urgence), fournissant la position physique exacte du chariot.

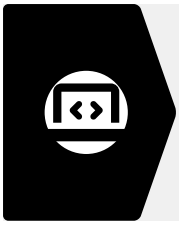
Ce protocole strict assure qu'aucune requête ne soit perdue ou désynchronisée lors du transfert massif de commandes générées par l'IHM.



DÉPÔT DE CODE SOURCE
L'intégralité du projet (Firmware Arduino, scripts du Web Worker et logique de l'IHM) est disponible en open-source pour consultation et versionnage.
URL : github.com/Samy-Y/projet2_groupe14

5 INTERFACE HOMME-MACHINE

```
++++++++++[>+++++++>++++++++++>+++++++++++>++++++++++>+++++
++++>++++++++++>++++++++++>++++++++++>++++++++++>+++>++++++
+>++++++++++>++++++++++>++++++++++>+++++>++++++++>+++++++++
+>++++++++++>++++++++++>++++++++++>++++++++++>++++++++++<<<
<<<<<<<<<<<<<<<<<<<-]>+++.>.>----.>+.>++++.>++.>---.>-.>+.>++
.>++.>+.>-..>+.>-----.>---.>---.>-.>++++.>-----.>.>+.
```



ACCÈS INTERFACE DE PILOTAGE
Le système de supervision est déployé sur GitHub Pages. Il permet l'exploitation du traceur via l'API WebSerial directement depuis un navigateur compatible.
Accès : samy-y.github.io/projet2_groupe14/

L'interface utilisateur du programme de contrôle du traceur est une partie indispensable du système, qui permet de *grandement* simplifier son utilisation et —considérant la nature très modulaire du code Arduino que nous avons conçu—, d'enrichir son fonctionnement de divers autres fonctionnalités. L'interface a été conçue pour répondre à quatre exigences clés :

1. **Être ergonomique, accessible et simple d'utilisation.** Idéalement, il n'y aurait pas besoin de manuel/tutoriel pour pouvoir utiliser correctement l'interface ; l'emplacement des boutons, la nomenclature des fonctionnalités etc. doit alors suivre les standards déjà existants et doivent avoir l'air "naturels" aux yeux de l'utilisateur.
2. **Être cross-platform.** L'interface doit être accessible depuis n'importe quel ordinateur (dans le sens pur du terme, qui englobe aussi les tablettes et smartphones...).
3. **Être simple à installer.** Il s'agit d'une "exigence-conséquence" naturelle découlant des deux premières. Le programme contenant l'interface graphique ne doit pas nécessiter l'installation de multiples dépendances de la part de l'utilisateur (ex.: un programme Python réalisé via PyQt, un programme Processings, etc...).
4. **Être visuellement agréable.** Tous les standards modernes en terme de conception graphique doivent être respectés.

5.1 Identité Graphique

Le design global de l'interface graphique est dans un **style "flat"**, avec **simulation de profondeur mécanique** (voir Fig. 46) pour différencier facilement les éléments cliquables principaux des éléments de fond. Cette simulation de profondeur est réalisée techniquement via une ombre portée sans flou (entièrement plate) à angle $0°$ sur laquelle vient se translater l'élément cliquable (généralement un bouton) pour simuler un enfoncement physique, sans avoir recours à du skeuomorphisme/neumorphisme qui alourdirait énormément le programme.

La famille de polices choisie est (sans surprise, compte tenu de l'aspect "simple" du projet) une famille sans-serif : **Source Sans 3** (la même utilisée sur ce rapport).

La palette de couleurs est classique et prévisible :

— Un bleu principal (`0056b3`) d'ombre (`0d345d`).
— Un gris principal (`ccc`) d'ombre (`aaa`).
— Un rouge principal (`c65050`) d'ombre (`912525`).
— Un vert principal (`2e8b57`) d'ombre (`1e5c3a`).
— Un jaune principal (`e0a800`) d'ombre (`b38600`).
— Tous les textes (étiquettes, headings etc.) sur fond blanc sont de couleur (`333`), légèrement plus clair que le noir pur (`000`), qui est généralement perçu comme étant plus agréable à lire.
— Les textes "muets" (ex.: boutons grisés...) sont en (`666`).

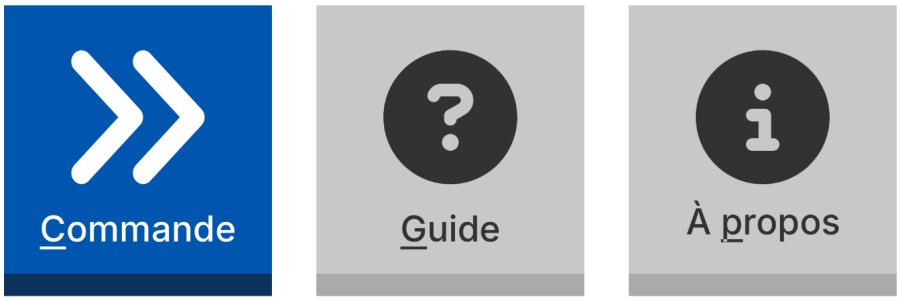


FIGURE 46 – Exemples de boutons à “aspect mécanique” (ombre portée sans flou)

5.2 Architecture Technique

5.2.1 Stack Technologique

Le stack technologique utilisé pour le développement de l’interface graphique est un **stack web vanilla** (HTML/CSS/Vanilla JS), sans framework lourd type React/Vue. Le résultat obtenu est donc une site web entièrement statique, qui peut être utilisé localement ou hébergé sur une plateforme dédiée (ex.: GitHub Pages), sans avoir à configurer une installation locale de Node.Js, Vite, ni à gérer des dépendances.

Pour se connecter à l’Arduino Mega, l’interface communique exclusivement en utilisant **l’API WebSerial**. Depuis mai 2026, WebSerial est supporté nativement sur les navigateurs basés sur Firefox/Gecko, cette technologie devient alors de plus en plus répandue.

5.2.2 Sécurité et fiabilité

Comme mentionné dans la section **??**, un **protocole Handshake** a été implémenté pour éviter la saturation du buffer de réception de l’Arduino et éviter toute désynchronisation et/ou perte d’instructions. L’envoi de données est strictement conditionné à la réception d’une chaîne de type `OK`. Si l’interface reçoit un message commençant par `LIMIT` (déclenchement physique d’un capteur), l’interface doit instantanément basculer en état d’erreur, purger la file d’attente et émettre une alerte sonore.

L’interface implémente un **gestionnaire d’états** strict (`IDLE`, `HOMING`, `RUNNING`, `ERROR`). Tant que le Homing initial n’a pas été exécuté et validé par l’Arduino, toutes les commandes de déplacement manuel et le lancement d’un tracé doivent être désactivés (visuellement grisés et non cliquables). L’état du port série est constamment surveillé, pour mettre le tracé en pause, sauvegarder l’index du dernier vecteur et actualiser l’interface en cas de fermeture inattendue.

Les traitements mathématiques lourds (comme le parsing et la discrétisation d’un fichier SVG, qui seront détaillés prochainement) doivent impérativement être déportés dans un **Web Worker fonctionnant en arrière-plan**. Les calculs sont en revanche, toujours effectués **côté client**, même si l’interface est hébergée sur une plateforme externe (nature serverless).

Avant de déverouiller les boutons de démarrage de tracé, l’utilisation doit cocher une “pre-flight checklist” de vérification matérielle pour s’assurer que le système est prêt pour une

mise en marche. Ces boutons seront grisés avant la vérification.

5.2.3 Documentation et déploiment automatisé

Le déploiement de l'interface est automatisé via un workflow GitHub Actions, qui génère et publie l'interface sur GitHub Pages. Elle est aussi accompagnée d'une documentation complète, hébergée sur GitHub Pages via MkDocs. Cette documentation couvre :

- L'architecture du projet (arborescence des fichiers, description des fonctions JS, etc.).
- La liste complète des fonctionnalités de l'interface.
- La liste des raccourcis clavier et leur description.

La génération des fichiers HTML hébergés sur GitHub Pages est gérée par MkDocs (conversion de fichiers Markdown vers HTML), qui est intégré dans un workflow GitHub Actions pour assurer un déploiement automatisé à chaque push sur la branche principale.

Le déploiement est donc **continu** tout au long du développement, garantissant que la documentation et l'interface sont toujours à jour et accessibles.

5.3 Accessibilité et Ergonomie (UX)

5.3.1 Ergonomie et expérience utilisateur

L'interface graphique a été conçue pour :

- **Être entièrement navigable au clavier** via la touche Tabulation ⇄ (via utilisation propre des index de tabulation HTML).
- **Posséder des raccourcis clavier utiles** basés sur des combinaisons `Shift` ⬆ + `LETTRE`. La lettre servant de déclencheur doit être visuellement soulignée dans le nom du bouton de l'interface (ex.: Homing). Les raccourcis sont aussi indiquées lors du passage du curseur sur un bouton (avec description du rôle du bouton) dans une infobulle.
- **Posséder des retours Audio.** Elle intègre des notifications sonores discrètes via l'API HTML Audio pointant vers la bibliothèque hébergée "Google Material Sound" (pas de fichiers locaux) (ex.: un son doux pour le succès, un son sec pour les erreurs).
- **Être Mobile-First.** L'interface est ergonomique sur écrans tactiles (tablettes/téléphones sous Chrome Android gérant WebSerial). Les textes sont lisibles sans zoom et les zones de clic (touch targets) sont suffisamment larges. Des îcones sont aussi utilisées via la bibliothèque FontAwesome pour simplifier la lecture.

5.3.2 Accessibilité

Dans le cas où l'interface est exécutée localement sans accès à Internet, des polices de secours sans empattement standard sont prévues (celles utilisées par défaut dans le système, `system-ui`). Des attributs `alt` sont utilisés pour remplacer les îcones Font-Awesome.

5.4 Fonctionnalités et Logique Opérationnelle

Le fonctionnement de l'interface est segmenté en plusieurs modules logiques gérant le cycle de vie complet d'un tracé, de l'importation du fichier source à l'exécution physique.

5.4.1 Workflow Automatique et Traitement SVG

Le cœur de l'application réside dans sa capacité à traiter des fichiers vectoriels et à les convertir en instructions machines (impulsions/pas moteur) optimisées. Ce processus s'articule autour des étapes suivantes :

Discrétisation et Aplatissement (Flattening) Les formes géométriques complexes (courbes de Bézier, ellipses) contenues dans le fichier SVG sont mathématiquement converties en une série de segments linéaires. La précision est ajustable via l'erreur cordale. Un filtrage spatial fixe la longueur minimale d'un segment à 0,2 mm (résolution de la machine) pour éviter la saturation de la liaison série. Une alerte de complexité se déclenche si le fichier génère plus de 10 000 segments.

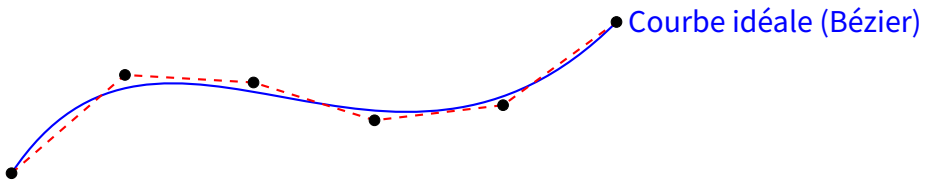


FIGURE 47 – Illustration de l'aplatissement d'une courbe de Bézier. Les segments rouges représentent les instructions linéaires (G1) générées.

Génération Autonome de l'Axe Z L'interface délègue la gestion des levées et descentes du stylet au client Web. Lors d'une transition entre deux chemins vectoriels isolés, le script JavaScript insère automatiquement une instruction de levée (`Z_UP`), suivie d'un déplacement rapide vers la nouvelle coordonnée, puis d'une instruction de descente (`Z_DOWN`). Un booléen agit comme garde-fou pour constamment s'assurer de l'état du stylet, et éviter qu'il ne s'enfonce ou se lève plusieurs fois de suite.

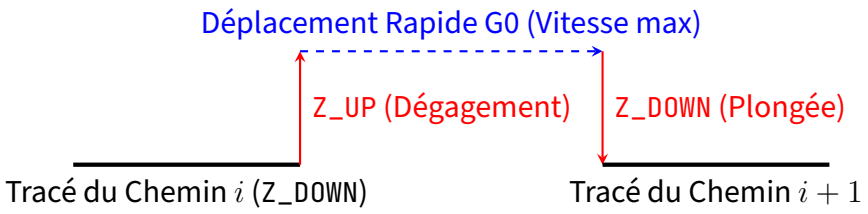


FIGURE 48 – Profil de mouvement cinématique lors d'une transition entre deux chemins vectoriels isolés.

Optimisation du Routage (Nearest Neighbor) Pour réduire le temps total d'usinage, l'ordre d'exécution des différents tracés SVG est réorganisé dynamiquement en utilisant l'algorithme du plus proche voisin. Cette méthode minimise la distance parcourue à vide.

Environnement Virtuel et Sécurité Spatiale Le tracé est prévisualisé sur un calque représentant les dimensions réelles du support (A4, A3 ou personnalisé). Le système évalue les coordonnées matricielles en temps réel : si le tracé déborde des dimensions définies ou dépasse l'enveloppe de travail mécanique (`MAX_TRAVEL_MM`), le démarrage est strictement verrouillé.

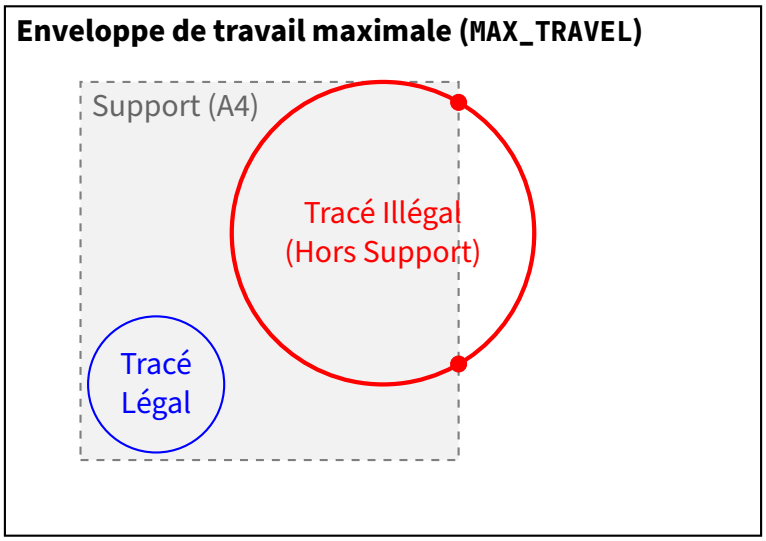


FIGURE 49 – Modélisation de la sécurité spatiale logicielle interdisant la sortie du stylet de la zone de dessin (sans échelle).

Jumeau Numérique et Télémétrie Lors de l'exécution, le canevas de prévisualisation agit comme un jumeau numérique. Le tracé global s'affiche en fantôme, et chaque segment physiquement tracé (confirmé par un accusé de réception `OK` de la carte) est surligné en temps réel. Le système calcule dynamiquement le temps restant estimé (ETA).

Remplissage par hachures La lecture des balises natives des fichiers SVG nous permet aussi d'utiliser les balises `<fill>` pour effectuer des remplissages par hachures. Une conversion en **grayscale** des couleurs de *fill* nous permet d'ajuster dynamiquement la densité de remplissage du tracé (en plus d'être entièrement paramétrable par l'utilisateur).

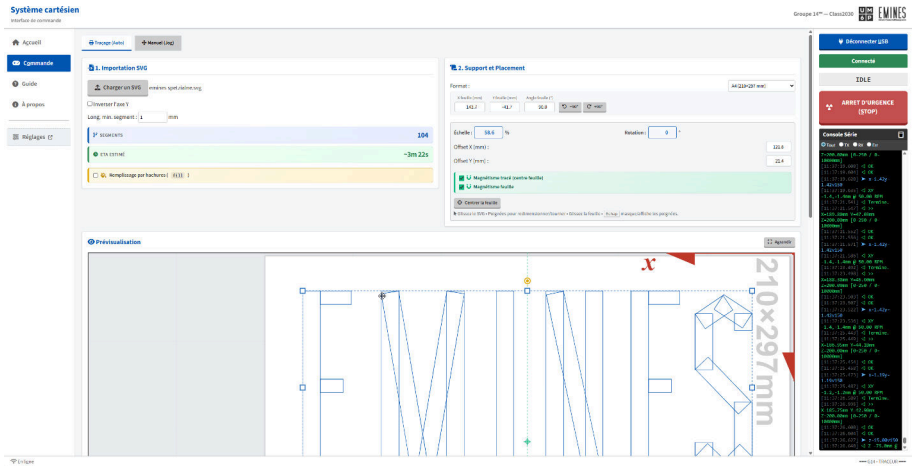


FIGURE 50 – Vue du jumeau numérique sur l'interface graphique.

5.4.2 Contrôle Manuel et Configuration Système

Pour les opérations de maintenance et le réglage initial, l'interface propose un contrôle direct sur les actionneurs et les paramètres globaux.

- **Panneau de Jogging :** Un contrôleur directionnel permet le déplacement manuel sur les trois axes avec des incréments spatiaux prédéfinis (de 0,1 mm à 50 mm).
- **Calibration Z :** Une procédure dédiée permet de faire descendre le stylet pas-à-pas jusqu'au point de contact physique avec le papier. Cette coordonnée est enregistrée localement sous la variable `Z_PROBING_ZERO`.
- **Arrêt d'Urgence Logiciel :** Bouton critique persistant qui, une fois déclenché, purge instantanément la file d'attente d'instructions côté JavaScript et transmet un signal d'interruption asynchrone au firmware. Le jumeau numérique fige immédiatement son état pour refléter la position d'arrêt exact.
- **Paramétrage Dynamique Persistant :** L'utilisateur peut ajuster les vitesses de tracé, les limites de l'enveloppe de travail et un coefficient de calibration mathématique (pour compenser une potentielle tension inégale des courroies). Ces paramètres sont sauvegardés dans le `localStorage` du navigateur pour garantir leur persistance entre chaque session, évitant ainsi de devoir reflasher le microcontrôleur.

5.5 Méthodologie de Conception Assistée par IA

Le développement de cette interface a suivi un flux de travail rigoureux mais axé sur une description exhaustive en amont (en vue de l'utilisation de LLM pour le développement de cette dernière) plutôt que sur l'écriture manuelle du code, ce qui a permis de réduire *considérablement* le temps de développement.

1. **Spécification Technique Exhaustive :** L'ensemble des fonctionnalités, l'architecture logicielle, les comportements de sécurité (Software Limits, E-Stop) et le protocole de communication série ont été documentés sous forme de cahier des charges détaillé.

2. **Maquettage Visuel :** La réalisation de croquis d'interface statiques sur *Photopea* a permis de figer la hiérarchie de l'information, l'ergonomie (Mobile-First) et l'identité graphique sans écrire de code.

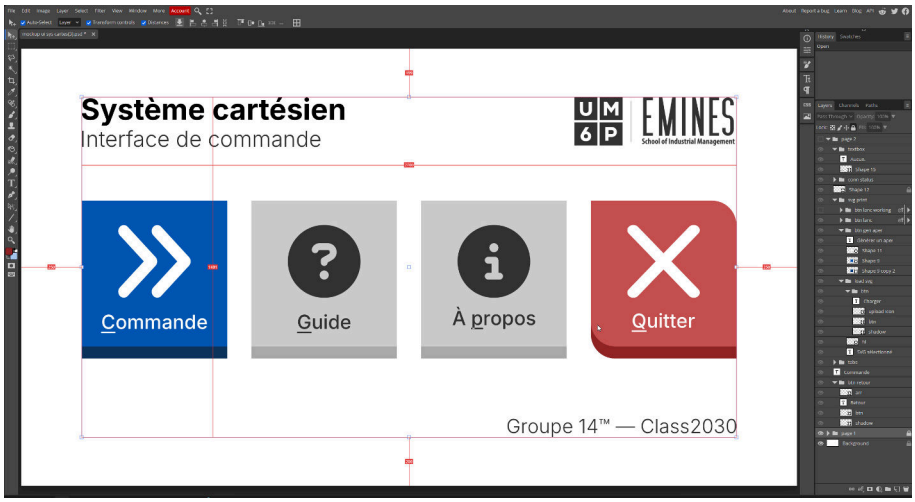


FIGURE 51 – Conception de l'interface sur Photopea (le brouillon initial prévoit une interface mobile).

3. **Ingénierie de Prompt :** Le cahier des charges ainsi que les contraintes visuelles ont été structurés pour former une instruction (prompt) claire, compartimentée et hautement technique.

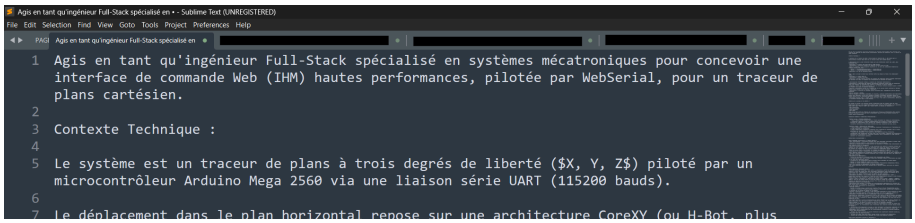


FIGURE 52 – Création du prompt.

4. **Génération de Code (IA) :** Ce prompt a été soumis à des modèles d'intelligence artificielle générative avancés, notamment Gemini 3.1 Pro via l'assistant GitHub Copilot intégré à l'environnement VSCode.
5. **Validation et Déploiement :** La précision et la complétude de la consigne initiale ont permis de générer un code HTML, CSS et JavaScript vanilla **quasi-parfait dès la première itération**. La phase de débogage a été pratiquement inexistante, se limitant à de simples ajustements structurels mineurs.



Le prompt utilisé pour la génération de l'interface graphique est disponible sur le répertoire GitHub de l'interface (`PROMPT.txt`).

6 TESTS ET VALIDATION

```
++++++++++[>++++++++>++++++++++>++++++++++++>++++++++++++>+++
+++++++++>+++>++++++++++>++++++++++++>+++>++++++++++++>++++++
++++>++++++++++>++++++++++>++++++++++>++++++++++>++++++++++
++>++++++++++>++++++++++>++++++++++<<<<<<<<<<<<<<<<<<<-]>+
+++.>+.>-----.>----.>-----.>++.>+.>----.>++.>--.>---.>--.>---
--.>.>---.>----.>-----.>+.>.
```

Cette section récapitule les tests et protocoles de validation suivis pour s'assurer du bon fonctionnement du traceur. Avant d'amorcer l'utilisation du code embarqué final détaillé dans la section précédente, nous devons évaluer le fonctionnement du système global sur des composantes "unitaires" de base pour :

1. mettre en évidence d'éventuels problèmes mécaniques lors du fonctionnement, principalement les **drifts/dérives** (Fig. 53) et les **overshoots** (Fig. 54).
2. mettre en évidence d'éventuels problèmes d'ordre logiciel (mauvaises lois de commande, constantes erronées...).

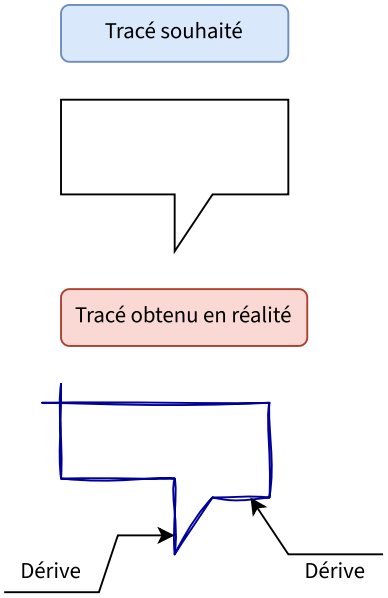


FIGURE 53 – Diagramme simplifié expliquant les drifts.

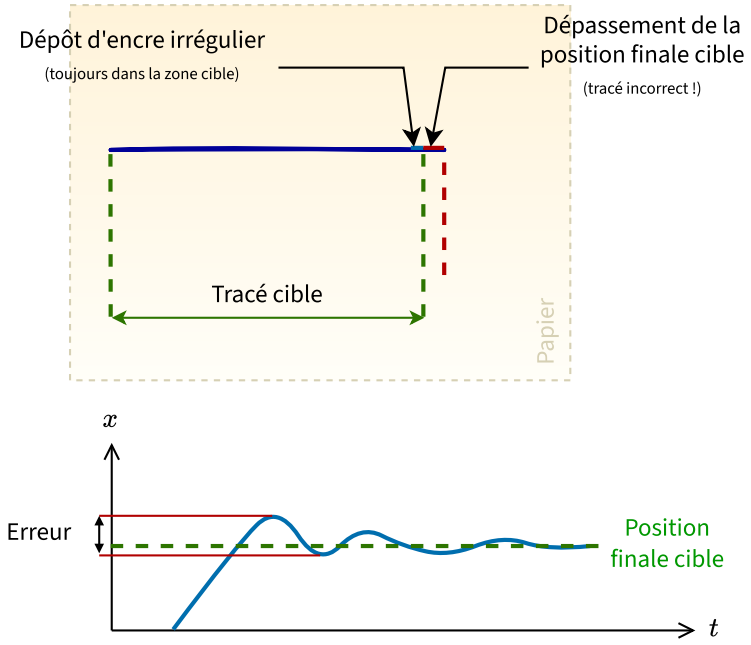


FIGURE 54 – Diagramme et graphique simplifiés expliquant les overshoots.

6.1 Protocoles des tests unitaires et d'intégration

Divers tests unitaires ont été appliqués sur le système pour isoler son fonctionnement sur des composantes réduites et s'assurer qu'il se comporte comme prévu et que le micrologiciel créé est correctement intégré. L'étape de validation a donc été structurée autour de trois tests unitaires majeurs :

1. **Test d'intégration physique :** Ce test consiste en une vérification manuelle et purement mécanique de la liberté de mouvement sur les trois axes (X, Y et Z) afin de détecter d'éventuels blocages structurels évidents après l'assemblage final.
2. **Tests unitaires cinématiques :** Des commandes de déplacement orthogonaux purs isolés selon l'axe X, Y puis Z sont envoyées avant l'exécution de trajectoires combinées. Ces tests nous ont aussi permis d'évaluer la précision du système ainsi de quantifier de manière préliminaire les drifts et les overshoots.
3. **Tests de trajectoires complexes/combinées :** L'exécution de trajectoires combinées plus ou moins complexes (tracé de formes simples) ont permis de valider la loi de commande globale. Ces tests ont parfaitement mis en évidence les drifts/dérives et les overshoots lors du tracé.

6.2 Synthèse des résultats et anomalies observées

Les tests unitaires ont révélé plusieurs anomalies, lors du fonctionnement du système.

1. **Déplacements fantômes à très haute vitesse :** À très grande vitesse ($\gg 120$ tr/min), on observe l'apparition de déplacements diagonaux fantômes lors de l'exécution de consignes de mouvements orthogonaux purs (causés par des points durs liés au fonctionnement des moteurs, entre autres...). Ces déplacements fantômes nous ont aussi empêché d'exécuter des arrêts brusques rigoureux, ce qui rendait toute mesure vibratoire quantitative via accéléromètre impossible et **limite notre étude à des analyses purement qualitatives des phénomènes mis en jeu**.
2. **Overshoots et drifts à très haute vitesse :** Lors du tracé de formes complexes à très haute vitesse, des overshoots et dérives *très importantes* sont observés.
3. **Sifflement aigu au niveau de l'axe Z :** L'actionnement de l'axe Z générait constamment un sifflement très aïgu.
4. **Fléchissement du stylet lors d'un enfoncement trop important :** Le ressort utilisé dans la tête d'outil (porte-stylet) était inadéquat pour rappeler le stylet lors d'un enfoncement trop important, ce qui provoquait des flexions très importantes et une rupture/cassure du stylet sous ces efforts.

Nous écartons l'hypothèse d'une défaillance matérielle liée à une saturation du CPU ou à une mauvaise gestion des interruptions côté micrologiciel.

6.3 Solutions et correctifs appliqués

Pour restaurer la précision géométrique et la répétabilité du tracé, quatre actions correctives ont été implémentées :

— **Bridage brutale de la plage de vitesse usuelle :** La vitesse nominale a été bridée de manière drastique, passant de 150 tr/min à un maximum strict de 50 tr/min. Ce compromis a été nécessaire pour garantir la faisabilité même du tracé. La plage de vitesses fixée par le cahier de charges n'est donc pas respectée.

— **Implémentation du profil de vitesse trapézoïdal :** L'introduction logicielle de rampes progressives d'accélération et de décélération, détaillées dans la section précédente, a permis de lisser le mouvement. L'overshoot dynamique est considérablement atténué, bien que des drifts résiduels cumulatifs restent observables.

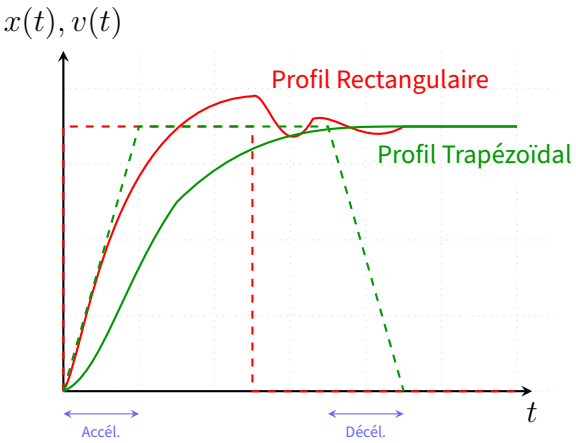


FIGURE 55 – Comparaison de l'overshoot en profil rectangulaire et en profil trapézoïdal. En pointillés rouge (resp. vert) : le profil de vitesse rectangulaire (resp. trapézoïdal). En trait continu rouge (resp. vert) : la position $x(t)$ résultante et l'overshoot (resp. sans overshoot).

— **Configuration en microstepping (1/8) :** L'activation du pas fractionnaire à l'échelle 1/8 sur les drivers a lissé le mouvement. La résolution spatiale globale a été multipliée au prix d'une réduction considérable du couple de maintien. Ce traitement a également permis de supprimer définitivement le son aïgu généré au niveau de la structure de l'axe Z.

— **Utilisation d'un ressort à raideur plus faible et impression d'une nouvelle pièce de maintien du stylet avec nervures :** Un ressort à raideur plus faible a été inséré dans le système originel pour compléter ce qui représente *techniquement* un asservissement en position du stylet par rapport à la surface rigide. La pièce de maintien de ce dernier a aussi été remplacée par une autre (Fig. 56) disposant de nervures circulaires pour encaisser les efforts latéraux engendrés lors des déplacements.

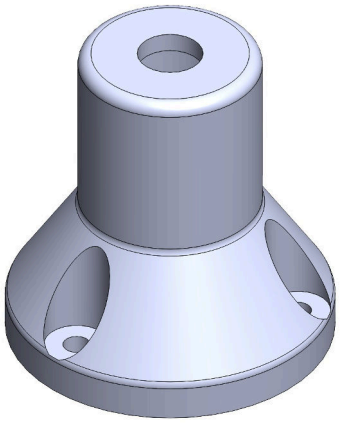


FIGURE 56 – Nouvelle pièce de maintien du stylet.

6.4 Évaluation des performances du système final

Bien que les problèmes les plus évidents et importants ait été résolus dans la phase de tests et validation préliminaire, de nombreux autres défauts et défaillances subsistent encore.

6.4.1 Problèmes d'ordre logiciel

— **Conversion SVG–Tracé inadéquate et peu fiable :** Le système de conversion SVG–Tracé construit "from scratch" en Javascript server-less via Webworkers discrétise mal de nombreux fichiers SVG (particulièrement ceux créés en utilisant `draw.io`). Cette mauvaise discrétisation se manifeste surtout sous forme de tracés parasites et d'une segmentation aberrante sur des trajets rectilignes supposés être simples.

⇒ **Solution : Utiliser un middleware externe (plus performant) pour assurer la conversion SVG–Tracé.**



FIGURE 57 – Tracé SVG original à convertir

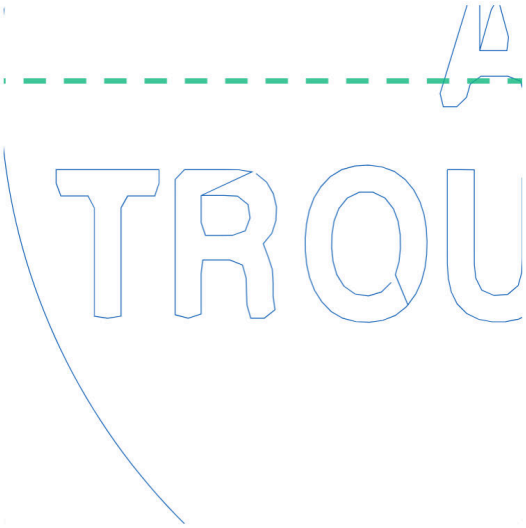


FIGURE 58 – Résultat de la conversion (tracés parasites visibles sur R et O)

— **Profil trapézoïdal réellement hyperbolique :** L'implémentation de notre profil trapézoïdal repose sur une modulation linéaire de la demi-période des impulsions envoyés aux moteurs. Mais comme la vitesse de rotation de ces derniers est inversement proportionnelle à cette période $V \propto \frac{1}{T}$, le profil réellement obtenu est donc un profil *hyperbolique*, qui n'est pas idéal pour obtenir un mouvement lisse.

⇒ **Solution : Implémenter un *vrai* profil trapézoïdal de vitesse en modulant la période de manière hyperbolique.** Une telle modification ajoutera une autre couche de complexité au système de gestion de vitesse, et pourrait induire d'autres problèmes liés aux délais des calculs de profil de vitesse.

— **Syntaxe inconsistante dans le code Arduino :** Bien que ce problème soit sans conséquences réelles sur le tracé du dessin, il rend l'opération de maintenance un peu plus difficile. En effet, la langue n'est pas uniformisée au sein du code Arduino, qui varie de l'anglais au français très souvent.

⇒ **Solution : Standardiser la langue au sein du code et utiliser uniquement l'anglais pour faire correspondre la langue de syntaxe à celle du langage de programmation.**

— **Définition des moteurs 1 et 2 inversée :** Idem, celui-ci ne provoque aucun effet indésirable, mais il rend juste la maintenance et la compréhension du code plus difficile.

⇒ **Solution : Inverser les définitions des moteurs 1 et 2 au sein du code.**

6.4.2 Problèmes d'ordre mécanique

Le problème majeur résiduel d'ordre mécanique est lié aux dérives cumulatives (drifts), particulièrement critiques lors des phases de tracé à haute vitesse ($\gg 120$ tr/min)—que nous avons éliminées de la plage de fonctionnement. Ce phénomène provient majoritairement de l'effondrement du couple dynamique des moteurs pas-à-pas (saut de pas).

Le bridage de la vitesse à 50 tr/min applique un **correctif symptomatique** qui dégrade les performances globales. Une résolution rigoureuse implique d'agir directement sur les causes électriques de la perte de pas plutôt que de restreindre la dynamique du système (plus particulièrement : le micro-stepping en 1/8).

Conclusion Générale

Ce projet d'ingénierie a permis de valider la conception, la fabrication et la commande d'un système cartésien traceur de plan opérationnel.

Sur le plan mécanique, la structure globale est robuste et fonctionnelle. Sur le plan électronique et automatique, l'implémentation de l'algorithme DDA en arithmétique entière et l'introduction du profil de vitesse trapézoïdal ont permis de supprimer les overshoots dynamiques transitoires. Le passage au microstepping 1/8 a définitivement lissé le mouvement, bien que la baisse de couple associée ait imposé un bridage de la vitesse maximale pour prévenir le saut de pas.

Le prototype final répond ainsi aux exigences majeures du cahier des charges.

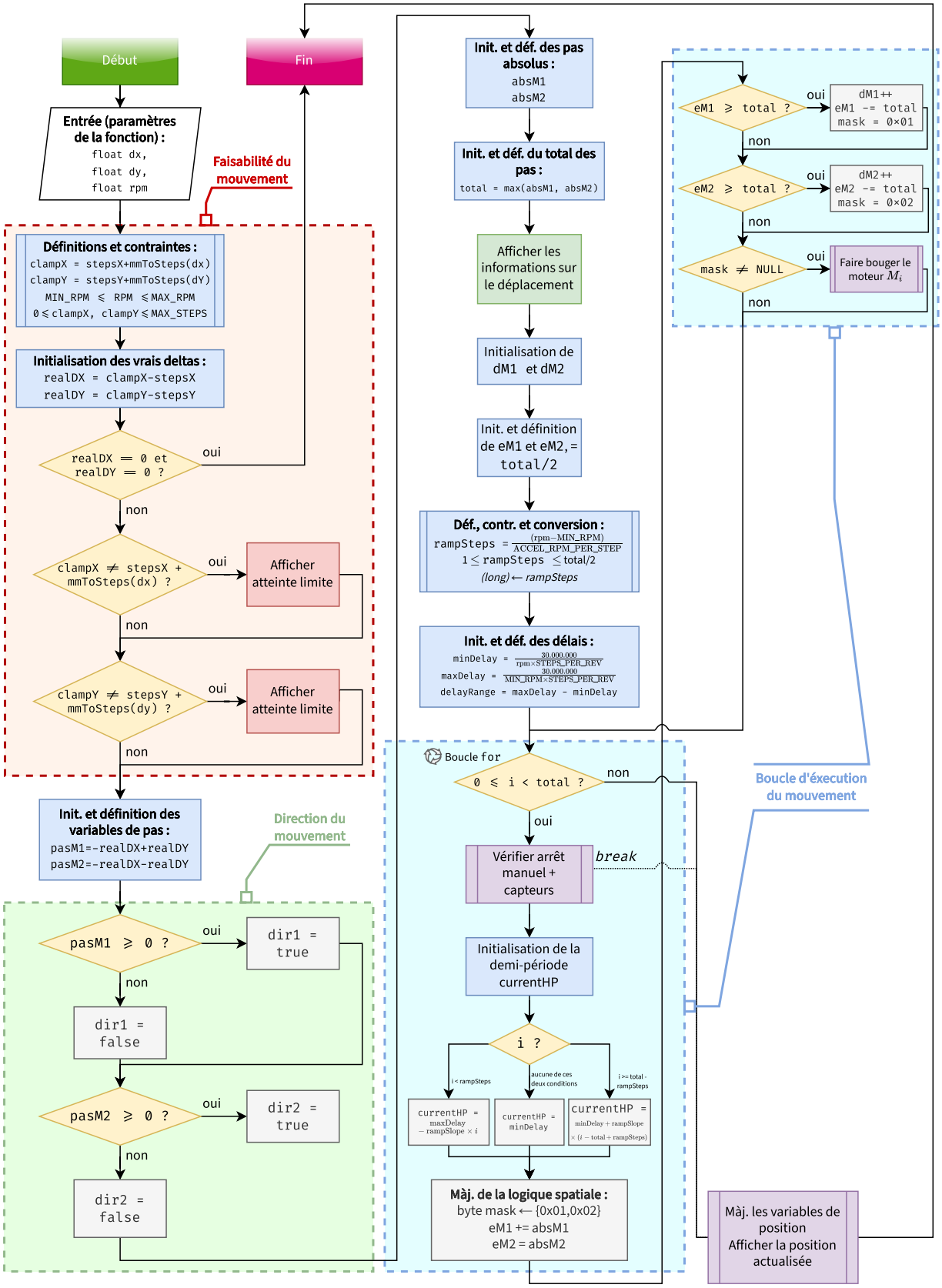

```
++++++++++[>++++++++>++++++++++>++++++++++>++++++++++>++++++++++>+++>++++++++++>++++>++++++++++>++++++++++>
++++++++++>++++++++++>++++++++++>+++>++++++++++>++++++++++>+++>++++++++++>++++++++++>+++>++++++++++>++++
++++++>++++++++++>++++++++++>++++++++++>++++++++++>+++++>+++>++++++++>++++++++++>++++++++++>++++++++++
+>+++>++++++++++>++++++++++>++++++++++>++++++++++>++++++++++>++++++++++>++++++++++>++++++++++>+++>+++++
++++++>++++++++++>++++++++++>+++>++++++++++>++++++++++>++++++++++>++++++++++>+++>++++++++++>++++++++++
++>++++++++++>++++++++++>++++++++++>+++>++++++++++>++++++++++>++++++++++>++++++++++>++++++++++>++++++++++
>++++++++++>+++>++++++++++>++++++++++>++++++++++>+++>++++++++++>++++++++++>++++++++++>++++++++++>++++++++
++++>++++++++++>+++>++++++++++>++++++++++>++++++++++>++++++++++>++++++++++>++++++++++>++++++>
+++>++++++++>++++++++++>++++++++++>++++++++++>+++>++++++++++>++++++++++>+++>++++++++++>++++++++++>++++++
+++++++>++++++++++>++++++++++>++++++++++>+++>++++++++++>++++++++++>++++++++++>+++>++++++++++>++++++++++
++++++++++>++++++++++++++++++>++++++++++>++++++++++>+++>++++++++++>++++++++++>++++++++++>+++>++++++++++>++
++++++++>+++>++++++++++>++++++++++>++++++++++>++++++++++>++++++++++>++++++++++>++++++++++>++++++++
+++++>+++>++++++++++>++++++++++>++++++++++>++++++++++>++++++++++>++++++++++>++++++++++>++++++++++++++++++
++>++++++++++++++++++>+++>++++++++++>++++++++++>++++++++++>+++>++++++++++>++++++++++>+++>++++++++++>++++++++
+++>++++++++++>++++++++++>++++++++++>+++>++++++++++>++++>++++++++++++++++++>++++++++++++++++>++++++++++
++>++++++++++>++++++++++>++++++++++>+++>++++++++++>++++++++++>++++++++++>+++>++++>++++++++++>++++++++++
+>++++++++++>++++>+++>++++++++++>++++++++++++++++++>++++++++++++++++>++++++++++>++++++++++>++++++++++>++++
++++++>++++++++++>++++++++++>++++++++++>+++++>+++>++++++++++>++++++++++>++++++++++>++++++++++>+++>++++
++++++>++++++++++>+++>++++++++++>++++++++++>++++++++++>++++++++++>++++++++++>++++++++++>++++++++++>++
+>++++++++++>++++++++++>++++++++++>++++++++++>+++++>++++++++++++++++++>++++++++++++++++>++++++++++>++
+++++++++>++++++++++>+++>++++++++++>++++++++++>++++++++++>++++++++++>++++++++++>++++++++++>+++>+++<<<<<<<<<
<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<
<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<-]>---.>+.>+++
+.>-.>-----.>++.>.>-.>---.>--.>+.>-----.>++++.>++.>--.>---.>++.>-.>+.>++.>++++.>---.>++..>+.>++++.>----.>----.>
++.>--.>+.>---.>-----.>++.>+.>-----.>++.>+.>++++.>+.>.>-----.>++.>+++.>---.>+.>++.>--.>+.>---.>-----.>++.>---.>
---.>++++.>+.>++.>++.>---.>++..>++++.>+.>-.>-----.>+.>++.>-.>+.>-----.>++.>++.>+.>-----.>-----.>----.>-----.>++.
>.>+.>-----.>---.>-----.>--.>-----.>----.>++.>--.>+.>---.>-----.>++.>.>+.>++.>-----.>---.>--.>+.>.>-----.>++.>++
.>---.>-----.>++.>-.>-----.>.>-.>+.>++.>++.>---.>-----.>++.>-----.>-----.>++.>--.>+.>---.>-----.>++.>---.>--.>+
.>++.>++.>++++.>+.>-.>---.>++++.>+++.>---.>-----.>-.>++.>+++.>---.>+.>++.>-.>+.>++.>-----.>+.>.>----.>+.>++.>.>-
.>-----.>-.>-----.>---.>-----.>----.>++.>++.>---.>-----.>++.>.>+++.>---.>+.>+.>++.>.>-----.>-.>-.>+.>++++.>---.>
----.>-----.>++.>----...>++.>.>+.>---.>-----.>++.>.>+.>++.>-----.>---.>---.>++++.>+.>.>-----.>++.>++.>+.>---.>-
---.>-----.>-----.>.>----.>++++.>+.>++.>----.>---.>-.>---.>-----.>-----.>++.>+++.
```

Fin du rapport.

Groupe 14

Annexe

A Algorigramme de la fonction de déplacement



B Schéma du câblage

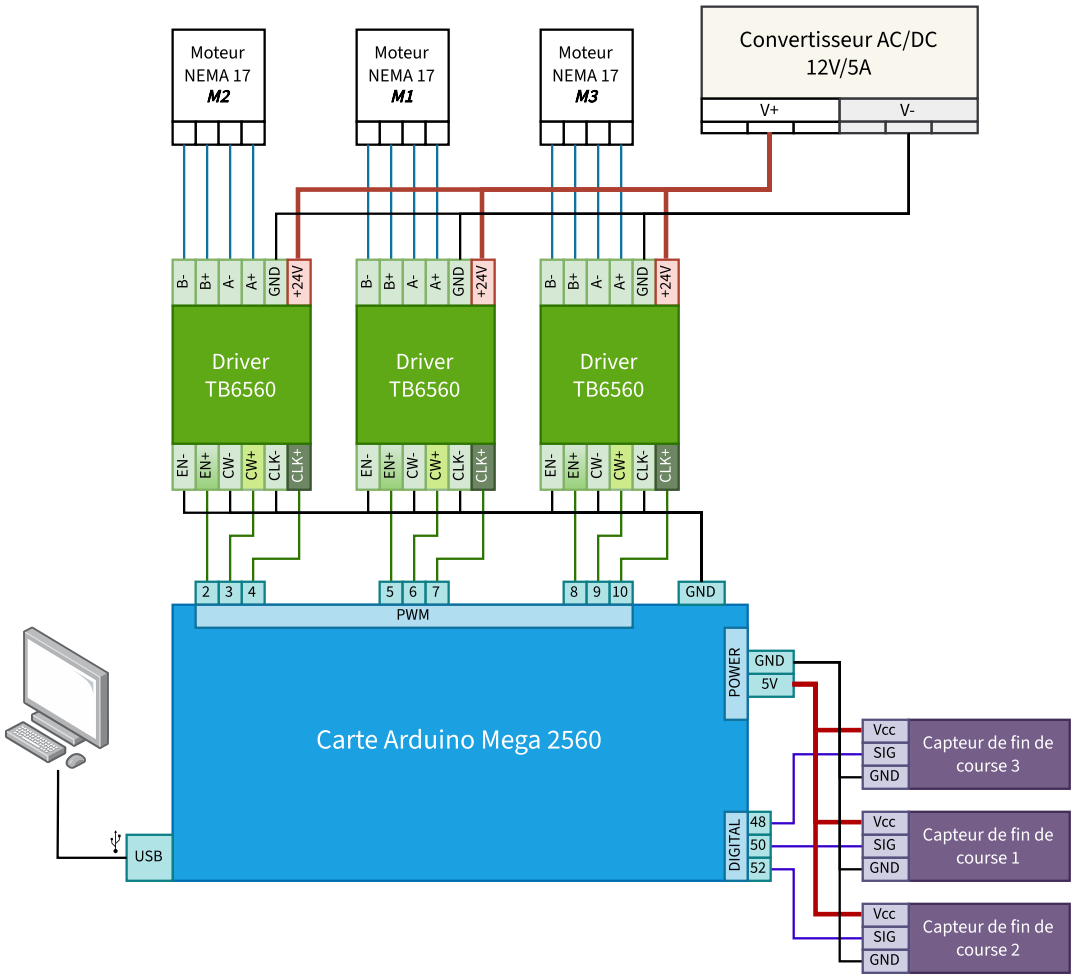


FIGURE 59 – Schéma électrique simplifié du système entier

C Déclaration d'utilisation de l'intelligence artificielle générative

Par transparence et rigueur académique, cette section récapitule l'usage d'outils d'intelligence artificielle générative au cours du développement du projet et de la rédaction du présent rapport. Les choix architecturaux majeurs relèvent intégralement de notre responsabilité et de notre travail conceptuel. Le tableau 19 dresse le registre exhaustif des composantes impactées.

TABLEAU 19 – Registre d'utilisation des outils d'IA

Composante du projet	Nature de la contribution
Rédaction et syntaxe du rapport	Correction grammaticale ; Densification des phrases ;
Mise en forme du rapport	Génération de structures de tableaux complexes ; Génération des squelettes de blocs en code LaTeX ;
Représentations graphiques	Génération de code TikZ de base **à partir de croquis sur Paint, draw.io ou Photopea** (Figures 39, 41, 40, 43, 44, 55, 58, 47, 48, 49, 34)
Développement logiciel	Aide à l'écriture de fonctions utilitaires et de structures de parsing de chaînes de caractères. Développement de l'interface graphique (création du middleware brut de conversion SVG). **Aucune IA n'a été utilisée pour la présentation Powerpoint de la soutenance**